

CANOPY Functional Requirements Document

Brief to Booth / Full-Platform Audit v1.0
August 12, 2026

Table of Contents

CANOPY — Functional Requirements Document

Version 1.0 · 2026-08-12 · full end-to-end audit of the platform as built

This document is the master inventory of everything Canopy is: every surface, menu item, workflow, input, and output — what each exists to do (**intent**) and what it actually does today (**current state**). It was produced by a five-domain code audit of the repository (`src/ + supabase/`), with the hosted schema (`src/integrations/supabase/types.ts`) treated as ground truth. The full per-screen audits live in `audit/` and are the authoritative detail behind every claim here:

Appendix	Domain
01-shell-access.md	Routing, auth, roles, layout shell, public/marketing surfaces, suspension mechanics
02-project-flow.md	The six-step project flow, Files, Knowledge Base, step gating & status model
03-agency-workspace.md	All agency site-level pages
04-super-admin.md	Platform-owner surfaces, invites, preview mode
05-backend-data-ai.md	Data model, 29 edge functions, prompt pipeline, RAG, integrations

Status vocabulary used throughout: LIVE works as intended · PARTIAL works with real limits · STUB UI exists, no behavior · INERT written but never read · LEGACY superseded system still shipping · BROKEN observably defective · PLANNED documented intent, no code.

1. What Canopy is

Canopy is a **spatial operating system for environment design**: a multi-tenant SaaS where experiential/exhibit agencies turn a client brief into a costed, rendered, client-ready activation package in under a day. The core promise chain:

- 1. **Ingest** a brief (file, paste, or guided Q&A) → structured `ParsedBrief`.
- 2. **Ground** it in accumulated intelligence: client brand facts (approval-gated), agency knowledge, activation-type playbooks, venue/show data — weighted, retrievable memory that compounds across projects.
- 3. **Strategize**: eight AI-generated concept elements (big idea → budget logic).
- 4. **Spatialize**: percentage zones become real-unit, drag-editable booth geometry — the ground truth every render is composed against.
- 5. **Render**: a hero image first (with a human hanging-element approval gate), then a consistent fan-out to every view — each image carrying the exact prompt that produced it.
- 6. **Deliver**: AI-designed deck, PPTX/PDF proposal, materials & cost list, 3-D modeling brief, Figma spec, ZIP — and feed learnings back into client intelligence.

Tenancy: agency → members (owner/admin/member/viewer) → clients → projects (optionally grouped into multi-activation **suites**). Platform owners (super admins) operate a separate console over all agencies. Six industry verticals swap vocabulary, brief sections, input mode, and default render angles.

Stack: React 18 + Vite + TS + Tailwind/shadcn (Flow C design system) · Supabase (Postgres + RLS + pgvector + Storage + 29 edge functions) · Zustand + React Query · AI via Lovable gateway/Google (Gemini family), OpenAI (gpt-image-2), Anthropic (Sonnet for decks), Firecrawl (scraping), Runway/Kling/Veo (video). Deployed via Lovable (pushes to `origin/main` as `gpt-engineer-app[bot]`); Vercel analytics wired.

2. Roles & access model

Intent

Two orthogonal role systems: **platform roles** (`user_roles`: `admin`, `super_admin`) and **agency roles** (`agency_members`: `owner` > `admin` > `member` > `viewer`). Every signed-in user must belong to an agency (mandatory onboarding). Platform owners administer agencies (suspend/disable/trial/flags/quotas) and can preview the agency experience. Suspended agencies keep read access but lose writes; disabled/expired agencies are locked out.

Current state

Requirement	Status	Reality
Auth (invite-only beta; sign-in, recovery, waitlist)	LIVE	Public sign-up deliberately retired; <code>beta_waitlist</code> table captures requests (no notification path — dashboard polling only)
Post-login redirect to intended destination	BROKEN	<code>ProtectedRoute</code> passes <code>state.from</code> and <code>AcceptInvite</code> passes <code>?redirect=</code> — <code>/auth</code>

Requirement	Status	Reality
		ignores both; every login lands on <code>/projects</code> , which dead-ends the signed-out invite flow
Mandatory agency onboarding (accept invite or create agency, industry locked at creation, orphan backfill)	LIVE	<code>create_my_agency</code> RPC; industry lock enforced by DB trigger
Route-level role guards	PARTIAL	Only <code>/admin/super-admins</code> , <code>/admin/agencies</code> , <code>/admin/industries*</code> self-guard. <code>/admin</code> and <code>/account/:userId</code> are reachable by any authed user and rely on RPC-level filtering; role hooks swallow errors as <code>false</code> (a network blip silently demotes a super admin)
Agency membership resolution	PARTIAL	Highest-priority membership wins; multi-agency membership is modeled but there is no agency switcher , and suspension gating follows the primary agency, not the agency whose data is touched
Suspension lifecycle (suspend/disable/trial → RLS write-block → banner → landing page)	PARTIAL	Server: <code>agency_has_access()</code> gates writes on <code>clients</code> , <code>projects</code> , <code>knowledge_documents</code> , invites — but not pricing, overrides, agency row, or any edge function (AI spend is not blocked). Client: <code>canWrite</code> is computed and never consumed — no button disables; suspended users hit raw RLS errors. Only disabled/trial-expired actually redirect to <code>/access-suspended</code>

Requirement	Status	Reality
Preview-as-agency mode	STUB+	A single client-side boolean: swaps nav and the /admin tab set. The banner's "Read-only agency view" is false — nothing consults it for writes; it binds to no target agency (previewing from a user drill-in shows the super admin's <i>own</i> agency); state is lost on refresh
Audit trail	PARTIAL	<code>agency_access_log</code> captures all agency lifecycle actions (except admin-notes edits); super-admin grant/revoke is unaudited

Invitation systems — three coexist, one works end-to-end:

1. `pending_invites` (agency member + super admin) — **LIVE**: RLS-gated insert, accepted via `accept_pending_invite` on the onboarding page. No email is ever sent; "they'll join automatically on sign-up" copy is aspirational (the invite is only applied if the invitee reaches onboarding — an already-onboarded super-admin invitee never gets the role).
2. `platform_invites` + `admin-invite-user` edge function — **PARTIAL**: sends a real Supabase auth email, but its role grant is dead (`invited_role` metadata is read by nothing) and the function 403s for pure super admins (`role='admin'` check).
3. `project_invites` (token share links, `/invite/:token`) — **LIVE** server-side (`accept_project_invite`), but the page still says "**BriefEngine**", and the Projects-page "Share..." menu that should mint links is a toast stub.

3. Information architecture

Navigation (intent = final Flow C model; current = shipped)

- **Site-level left sidebar** (always site scope): Agency set — All Projects, Clients, Activation Types, Agency Knowledge, Pricing, Company Profile, Team (+ Admin Settings for agency admins). Platform set (super admins) — Accounts, Agencies, Industries, Super Admins, Invites.
- **Project-level top nav**: breadcrumb + spec pill + measurement-system dropdown + step pill rail.

Current-state defects in the IA

- **/platform-invites is a dead sidebar link** — no route exists; super admins clicking "Invites" get the 404 page (the invites UI actually lives in `/admin` → All Accounts).

- **Orphan routes with no entry point:** /rhino (fully duplicated by Files → 3D tab), /knowledge-base (legacy project KB), /team (legacy team model). /explore (360° viewer) is commented out but its components still ship.
- **Step model disagreement:** the pill rail has **seven** pills (Files is #6, Export #7) while every sheet says “Step 0n / 06”; pill completion is **positional** (everything left of the current URL shows ✓ regardless of actual state) and the designed **blocked** red-dot state is produced by no caller.
- The public /architecture page is unlinked from anywhere (share-by-URL only); the landing page’s “4-scope RAG” vs “5-scope” copy conflicts with itself.

Full route map

See audit/01-shell-access.md §1 for the complete table (path → component → guard → audience), including eager/lazy split and dead routes.

4. Public & entry surfaces

Surface	Intent	Current state
/ landing	Top-of-funnel: position the spatial OS, route to sign-in	LIVE (dark theme kept deliberately). Gaps: no mobile nav at all; primary CTA routes anonymous visitors to a guard redirect rather than sign-up; stats hardcoded
/industries/:slug	Per-vertical marketing deep-dive with live vocabulary	PARTIAL — anonymous visitors lose the project-types section (RLS blocks the read); interior_design has no hero image or narrative; new DB industries render empty
/architecture	Public, honest intelligence-architecture explainer (live vs planned)	LIVE — manually kept in sync with docs/intelligence/; exposes schema names publicly by design
/auth	Invite-only door: sign-in, recovery, beta waitlist	LIVE with the redirect-loss defect (§2); terms/privacy referenced but not linked; no MFA/SSO
/invite/:token	Redeem project share link	LIVE mechanically; stale “BriefEngine” branding; dead-ends for signed-out

Surface	Intent	Current state
		users
/onboarding/create-agency	Mandatory tenancy gate	LIVE; logo is a URL field (not an upload); no decline-invite affordance
/access-suspended	Non-punitive lockout landing	LIVE; contact email is <code>hello@exhibit.us.com</code> (brand mismatch); suspended (vs disabled) users are never actually routed here
* 404	Terminal not-found	Off-brand: no Canopy styling, raw <code><a></code> , no route back for signed-in users

5. Agency workspace (site-level)

Full per-screen spec: `audit/03-agency-workspace.md`.

5.1 Projects — /projects

Intent: workspace home; every visible project as a visual grid or client-grouped table with pipeline progress, search/filters, create project/suite. **Live:** create (with brand URL captured), grid/table toggle (persisted), hover-scrub render thumbnails, client grouping, suite parents with expandable children, status-routed open, delete, admin all-users mode. **Gaps: Share... and Archive are toast stubs** (no archived column; the fully-built `project_invites` system is the natural backing for Share but isn't wired). Create-dialog copy promises an immediate brand scrape that actually happens later on Upload. Non-admin query re-filters to `user_id = me`, so teammates' projects are invisible despite RLS permitting them — the “agency workspace” is currently per-user. Pipeline steps “Render Prompts Ready” and “Exported” can never light up (see §6.7).

5.2 Clients — /clients + /clients/:clientId

Intent: roster of brands; each client is the container for brand intelligence, brand guidelines, scoped knowledge, and its projects. **Live:** create/edit client, logo display (URL-based), accurate project counts (fixed to count RLS-visible projects by `client_id` — see the `agency_id` root cause in §9), per-client dashboard with Overview/Brand/Knowledge/Projects tabs and deep-linkable `?tab=`, auto-saving notes, **brand deep-dive** from URL or brand-book PDF (three size-tiered paths incl. client-side rasterization at ≤ 40 pages), dedupe-aware persistence into `brand_intelligence` (unapproved, confidence 0.85), `brand_guidelines` projection, approve/delete per entry. **Gaps:** no logo upload (URL string only); brand colors are only ever set by the deep dive (no editor, so wrong colors can't be corrected); no client delete on the dashboard; `brand_guidelines` upsert failures are swallowed; approval is enforced at generation call sites (LIVE) but the *dashboard* surfaces don't communicate that unapproved entries are inert.

5.3 Activation Types — /agency/activation-types (+ /:typeId)

Intent: the catalogue of formats the agency builds; each type teaches the AI its grammar (template, must-have/avoid, size envelope) and carries scoped knowledge. **Live:** built-ins + agency custom types, industry isolation filtering, category grouping, favorites (localStorage), create custom type, per-type dashboard with template editing (override for built-ins via `activation_type_overrides`, direct update for custom), restore defaults, dual knowledge panels (agency-scoped + platform foundation). **Gaps:** the **activation_type_agency knowledge scope is embedded but never retrieved** (rag scope list omits it) — agency-specific type knowledge is a dead end; favorites are device-local; custom types can't set scale/sqft/emphasis from this UI; no delete/rename for custom types; template “injected into every prompt” claim depends on generation paths that read `element_emphasis` (partial).

5.4 Agency Knowledge — /agency/knowledge

Intent: agency-wide document layer feeding all generation. **Live:** upload → embed (embed-document: extract, chunk ~1000 chars, `gemini-embedding-001 768-d`) → auto-tag (auto-tag-document: `doc_type/tags/summary`); status lifecycle with retry; pin/unpin (pins force-inject into retrieval); preview/download/delete; search. **Gaps:** the 3-second delay before auto-tagging races extraction on large PDFs; `priority_weight`, `title`, `user_tags` are writable by the hook but have no UI; images are indexed by a synthetic filename caption only (no vision); `summarize-document` and `extract-pricing` edge functions are deployed but never invoked. **And the biggest one: retrieval itself rarely runs — see §8.3.**

5.5 Pricing — /agency/pricing + /pricing?project=

Intent: agency rate cards + supplier feeds resolving a per-project CSI-division BOM with provenance-labeled prices and regional factors. **Live:** BOM CRUD (`plan_items`), override pricing, CSI grouping + division roll-up (`project_pricing_summary`), quality tier + region parameters, `price_plan` resolution chain (override > agency quote > global quote > unpriced), the agency-level index of projects-with-BOMs. **Gaps:** **no UI writes pricing_sources or pricing_quotes anywhere**, so every line is override-or-unpriced; the four roadmap cards (rate-card import, AI estimate, blueprint → BOM, supplier feeds) are STUB; **price_plan raises for any project created after agency onboarding** because `projects.agency_id` is never stamped on insert (§9); region is free text; snapshots/export are absent.

5.6 Company Profile — /company

Intent: the agency's own identity for proposals/exports + a show/venue cost database. **Live:** profile CRUD, logo uploads (light/dark) to `company-assets`, brand colors, contact block, show-cost CRUD with preset copy-on-edit; `company_name/logo_url` reach decks/exports; `name/industry/booth-sizes/notes` + show costs reach `generate-element`. **Gaps:** brand colors, tagline, and all contact fields are saved but **read by nothing** (the Branding tab's promise is aspirational); `logo_dark_url` unconsumed; the profile is **per-user, not per-agency** — teammates maintain divergent “agency” profiles; no unsaved-changes guard.

5.7 Team — /agency/team (LIVE) vs /team (LEGACY)

Intent: one agency roster with owner/admin/member/viewer roles and email invites. **Live (/agency/team):** member list via `list_agency_members` RPC, role changes, removal, pending `pending_invites` with cancel; primary-owner protections (client-side). **Legacy (/team + the /admin “Team” tab):** the pre-agency `team_members` model — invites never email anyone, `accepted_at` is never set (everyone renders “Pending” forever), and the placeholder `user_id = owner` makes rows meaningless. **Two unrelated team systems are both shipping**, reachable through different doors.

5.8 Admin Settings (agency view) — /admin

Intent: agency-level configuration: project-type AI behavior, clients & brand intelligence, knowledge, KB health, team. **Live:** the `ProjectTypeManager` editing surface (labels, descriptions, render context, 8 strategic elements with AI guidance, enable/disable), `ClientsManager`, KB panel, KB health dashboard (re-embed, legacy migration trigger). **Critical gap: `project_type_configs` is write-only.** No generation path, prompt builder, or edge function reads it — editing render context or AI guidance here changes nothing about output. It is also keyed per-user, not per-agency. Four parallel “type” systems exist (`code registry` → `per-user configs (INERT)` → `AI-detected custom_project_types` → `agency activation_types+overrides`); only the last is wired end-to-end. Disabled types still appear in brief-side selectors (they read the registry directly).

6. The project flow (six steps + Files + Knowledge Base)

Full per-step spec: `audit/02-project-flow.md`. Project identity rides `?project=<uuid>`; `useProjectSync` hydrates the Zustand store; persistence is per-field projects UPDATES.

6.1 Step 01 · Brief — /upload

Intent: capture the brief (file/paste/guided Q&A) → `ParsedBrief`, plus the supporting context: brand website, brand logo, inspiration, project documents — before advancing. **Live:** PDF/DOCX/TXT parsing (`parse-brief`: Gemini Flash, PDF-vision escalation to Pro, JSON-repair), guided builder (`synthesize-brief` + per-field `suggest-brief-field`), AI project-type inference + fuzzy client matching + new-client capture with brand-intelligence extraction, brand website persist + deep-dive trigger, brand logo into the KB (`brand-logo` tag → reference image on every render), inspiration images/links, project KB (RAG-backed panel), the deliberately-below-the-fold Continue CTA. **Gaps:** brief file signed URL has a 1-year TTL with no refresh (silent 404 later); no `image/.doc` brief support; the store’s brand-intelligence capture failure is silent.

6.2 Step 02 · Review — /review

Intent: verify/correct every parsed field and close the gaps the prompt composer needs, **before** tokens are spent. **Live:** six editable section cards with inline editors; gap clarification cards (venue = blocking, hero dimensions = blocking, color hex / audience / hero form / hanging = helpful) writing answers back to the exact fields the normalizer reads; hanging-element authoring

(creative direction = EXACT contract); existing-space photo + polygon annotations for interior verticals (analyze-existing-space vision merge that preserves in-flight user edits); Creative Direction embrace/avoid. **Gaps: blocking gaps don't block** — Confirm is always enabled; hanging width/depth/drop edits in the card are **discarded** (only qualitative fields round-trip); input mode resolves from the agency's industry (no per-project industry column); five of six confidence badges are hardcoded; whole-blob parsed_brief writes are last-write-wins.

6.3 Step 03 · Concept — /generate

Intent: generate the eight strategy elements from the brief + weighted context; read, edit, regenerate with feedback. **Live:** auto-generate on first arrival; sequential batch through generate-element (Gemini 2.5 Pro, forced tool calls, element-tuned temperatures, spatial zone post-processing/clamping); per-element persist-on-completion; module-level job survives navigation; retry w/ timeout; regeneration feedback auto-captured as approved past_learning intelligence; intelligence entry selector (approved-only); detail panel with dot-path editable fields. **Gaps: manual edits in the detail panel are never persisted** (store-only — lost on reload); the step reads the legacy knowledge_base_files for context, so documents uploaded on Step 01's RAG panel never reach element generation as inline content; generating/error statuses are memory-only.

6.4 Step 04 · Spatial — /spatial

Intent: AI zone allocation → real-unit drag-editable geometry (the render ground truth), enriched with structural form, intent, materials, features, hanging. **Live:** the absolute-unit geometry model with legacy round-trip; drag/resize/shape/height/form/material editing; features; hanging elements on canvas; AI enrichment (enrich-spatial) preserving user edits; layout variations (idempotent, pre-fixed); validation panel + auto-fix + metrics + cost estimator; per-zone prompt override that both Spatial and Render respect; per-config selection persisted via activeConfigKey and shared with Render. **Gaps: Export SVG button has no handler;** enrich-spatial always enriches configs[0] server-side (wrong zones for non-first configs — silently no-ops on merge); **hanging elements authored here never reach the render prompt** (only parsed_brief.hangingElements does — documented KNOWN GAP); the removed floor-plan render feature's state is still half-mounted; validation never blocks progression.

6.5 Step 05 · Render — /prompts

Intent: compose the exact prompt from normalized brief + geometry; hero first; conversational refinement; hanging-element human gate; consistent fan-out; every image keeps its prompt. **Live:** the full client-side composer (§8.2) with preflight checklist, prompt inspect/edit, two-step compose → render; hero conversation thread with branching; EDIT-MODE refinement (locked instruction template); hanging approval keyed to (config × hero image) and persisted on the image row; per-config versioning (__v__/_cfg__ angle-id scheme, legacy fallback, orphan recovery); “Generate all sizes” sequential batch with per-config save handlers; all-views fan-out (exteriors then interiors, concurrency 3, retry on transients); per-image prompt_artifacts + RenderPromptDialog everywhere; geometry reference capture; model fallback chain with a visible model badge + failure reason. **Gaps:** hanging gate **warns but never blocks**; style presets are dead on the composed path (preference order means they apply only to the legacy builder);

preflight reads `configs[0]` zones (wrong on multi-config); hero thread and prompt edits are memory-only (images survive, conversation doesn't); prompt versions persist to localStorage only (`projects.prompt_versions` column doesn't exist); `hanging_elements_aloft` compliance is permanently "unknown" (no post-render check).

6.6 Step 06 · Export — /export

Intent: package everything into client deliverables and close the learning loop. **Live:** AI-designed deck (Claude Sonnet HTML slides, per-slide regenerate, PDF/PPTX export, key-diagnostic ping); classic PPTX/PDF proposal; materials & cost list (`generate-materials`) + CSV; 3-D modeling brief + Meshy prompts (`generate-3d-brief`) + .md; Figma spec JSON; full ZIP (renders + Rhino + `content.md/json`); Save Learnings (`extract-learnings` → selected entries into `brand_intelligence`). **Gaps: nothing ever writes status: "completed"** — the "Exported" pipeline step can never light; decks live in localStorage only (not shared, not durable); materials/3-D/Figma outputs are un-persisted component state; **every export uses configs[0]** — the active size selected in Spatial/Render is ignored; `renderPrompts` sent to the 3-D brief is always null.

6.7 Step gating & status model (the structural finding)

Four disagreeing notions of progress coexist: (1) the **positional** pill rail; (2) `activeStep` in the store (written, never read for gating); (3) per-screen soft empty-states (the only real gates — advisory dead-ends, never redirects); (4) persisted status — `projects.status` only ever receives "draft" and "reviewed", while "parsing", "generating", "completed" are read-only fiction, and `render_prompts/hero_prompt/hero_style_confirmed` are read but never written. Element status is derived from column non-nullness. **Requirement going forward:** one authoritative per-step status model, persisted, driving pills (including the unused blocked state), routing, and the Projects-card pipeline.

6.8 Files — /files · Knowledge Base — /knowledge-base · Rhino — /rhino

Files (LIVE): all renders across versions/sizes with angle filters, current-only, per-size grouping, config labels, lightbox with Download + View prompt; 3-D tab (Rhino upload + AI polish); video tab (Runway/Kling/Veo via `generate-video`) seeded from image selection. Gaps: read-only (no delete/rename/set-current); occupies step slot 6 vs Export's "06" title; downloads always named .png. **Knowledge Base (LEGACY, orphaned):** the pre-RAG `knowledge_base_files` page — no nav link, client-side text extraction only, public-URL storage. Two parallel project KBs exist; `migrate-legacy-kb` exists but no UI calls it from here. **Rhino (LEGACY, orphaned):** fully duplicated by Files → 3D tab.

7. Platform (super admin)

Full per-screen spec: `audit/04-super-admin.md`.

7.1 /admin — Platform console

Intent: accounts, role grants, platform invite ledger, activation-type CRUD, venue intelligence, per-agency image-model routing, fleet AI usage, invites. **Live:** account search/stats/drill-in; grant/revoke agency-admin via the audited `admin-manage-role` function; AI usage dashboards (fleet stats, agency + user leaderboards, feature×model breakdown from `ai_usage_events`); activation-type manager; image-model manager. **Gaps: blank first paint for super admins** (initial tab state race selects a tab that doesn't exist in the platform tab set); the “Venues & Shows” and “Invites & Team” tabs are **not platform-scoped** (they show the signed-in admin's personal rows via legacy hooks); the invite dialog's `super_admin` option grants nothing; `admin-invite-user` 403s for pure super admins; no revoke/resend on platform invites.

7.2 /admin/agencies — Agency lifecycle console

Intent: commercial control plane — status, trials, feature flags, quotas, industries, notes, audit log per agency. **Live:** the full drawer (Status/Industries/Features/Quotas/Log), all lifecycle RPCs audited into `agency_access_log`, effective-status derivation, 30-day AI spend inline, member/client/project counts, last-activity ordering, admin industry re-assignment (the only escape hatch for the creation-time lock). **Gaps: feature_flags and quotas have zero readers in the product** — complete authoring UIs over inert data; trial date input is timezone-lossy; no trial → active path other than “Reactivate”; the “Suspended” summary tile silently includes trial-expired; notes edits skip the audit log.

7.3 /admin/industries (+ /:slug)

Intent: manage verticals: identity/ordering, activation-type tagging, globally-curated industry knowledge, vocabulary. **Live:** list with live counts (RPC w/ constants fallback + schema-setup banner), create/delete, detail tabs (overview, cross-tagging of activation types, industry-scope KB uploads, vocabulary editor with the 9-key reference). **Gaps:** the auto-seed of built-ins is **dead in exactly the empty-table case it exists for** (placeholder-UUID guard bug; `interior_design` missing from the map); `briefSections/inputMode/defaultRenderAngles` exist only in the TS constant — not columns, not editable, so custom industries get vocabulary only; built-ins are deletable with `force: true` always passed; `interior_design` is a consistent second-class citizen (icon map, hero image, “5 verticals” copy).

7.4 /admin/super-admins

Intent: grant/revoke platform-owner access with safeguards. **Live:** roster via self-gated RPC, invite via `pending_invites`, revoke with self-revoke warning, client-side last-admin lock. **Gaps: the last-super-admin protection is client-only** (a direct RPC call or two concurrent revokes can lock the platform out entirely); invites send no email and only apply at onboarding; no audit trail; nothing expires pending invites.

7.5 /account/:userId

Intent: single-user dossier: identity, role, owned agency, roster, activity, plan. **Live:** header/role badge, stat cards, roster (owner-only), recent projects, entry to preview mode. **Gaps:**

subscription tier pills are a **decorative stub** (“wired to real billing later”); no access-status surface (must cross-reference /admin/agencies); owned-agency is singular/owner-only; no role controls here; “Active” status is a literal.

8. Backend & AI pipeline

Full detail: audit/05-backend-data-ai.md.

8.1 Data model (34 tables, by domain)

- **Identity/tenancy:** agencies (access status, trial, flags, quotas, image model, industries), agency_members, agency_access_log, user_roles, profiles, industries, company_profiles; LEGACY team_members.
- **Clients/brand:** clients, brand_guidelines (structured facts), brand_intelligence (approval-gated memory, 6 categories), venue_intelligence, show_costs; brand_assets (**missing from generated types** — accessed via casts).
- **Projects:** projects is the central record — brief text/file/parsed JSONB, all **eight element JSONB columns**, spatial_strategy (zones/features/materials/hanging/activeConfigKey), suite hierarchy (parent_id, is_suite, inheritance flags), render_prompts/hero_prompt (write-orphaned); custom_project_types, project_type_configs (INERT), activation_types + activation_type_overrides.
- **Renders:** project_images (angle-id versioning, is_current, prompt_artifacts = prompt/negative/geometry/compliance/references/model/config tags/hangingApproved), rhino_renders.
- **Pricing:** plan_items, pricing_sources + pricing_quotes (**no writers exist**), regional_factors; RPCs price_plan, project_pricing_summary.
- **Knowledge/RAG:** knowledge_documents (scoped, pinned, priority-weighted) + knowledge_chunks (pgvector 768) + rag_query_log; LEGACY knowledge_base_files, activation_type_kb_files, kb_migration_log.
- **Invites/telemetry:** pending_invites, platform_invites, project_invites, ai_usage_events (+ usage RPCs), beta_waitlist (not in types).
- **Storage buckets:** project-images, knowledge-documents (private/signed), knowledge-base (LEGACY, public), brand-assets, rhino-renders, company-assets, briefs.

8.2 Prompt composition (client-owned, src/lib/normalizedBrief.ts)

normalizeBrief → validateBrief (hard constraints + gap catalog) → composePrompt, dispatching on industry input mode:

- **Spatial-canvas renderer**, section order: # SCENE (per-type photographic lineage) · # SPACE (rectangular carpet allocation; “design organically above the floor”) · # HANGING ELEMENTS (venue-rigged; creative direction = EXACT lock) · # STRUCTURAL APPROACH · # ZONE PROGRAM (deduped functional purposes only — coordinates

deliberately removed) · # BRAND (role-tagged hex, signage visibility rules) · # BUDGET REALITY (standard/premium/ultra material vocabulary + the one-accent-upsell rule; no budget → premium) · # CONTEXT · # ENVIRONMENT (always; the void-ban's positive half) · # DESIGN INTENT · # HARD CONSTRAINTS · # NEGATIVE (inline; includes the blank-background/void ban).

- **Existing-space path** for interior verticals: SCENE / EXISTING SPACE / PRESERVE / REDESIGN / INTENT / CONSTRAINTS / NEGATIVE (SPACE and ZONE PROGRAM deliberately omitted).
- **Consistency model:** hero rendered from the full composed prompt (used **verbatim** by the edge function) → hanging-element human gate (approval keyed to config × hero image, persisted to the image row; refinements go through a locked EDIT-MODE template with no composedPrompt so the edge takes the edit branch) → view fan-out with deliberately **terse** ~100-token prompts + hero-as-only-reference (long prompts made gpt-image-2 regenerate from scratch) → per-config save handlers bind renders to their size.
- **Image chain:** gpt-image-2 primary (edits endpoint with ≤4 refs + optional mask) → Gemini image fallback chain; SVG refs stripped; floor-plan PNGs deliberately never attached (label-bleed); server-side upload returns short URLs; modelUsed/primaryError surface in the UI.

8.3 Knowledge & intelligence retrieval

- **Approval gate (LIVE):** both element and render call sites filter `is_approved` before sending brand intelligence. Confidence scores are display-only.
- **Deterministic brand context (LIVE):** `brandRAGBuilder` assembles guidelines + approved intelligence + assets + venue + legacy KB into a token-budgeted prose block passed to element/hero/view calls (capped 600/500 chars in image prompts, placed last by design).
- **Vector RAG (PARTIAL — the flagship gap):** upload → embed → auto-tag → hybrid retrieval (`match_knowledge_chunks`: 0.7 vector + 0.3 BM25, × per-doc priority, × scope weights project 1.0 > client 0.85 > activation_type 0.75 > agency 0.6, dedupe, optional rerank, pinned force-include, `rag_query_log` analytics) is fully built — **but every `buildRagContext` call is gated on `agency_id`, and the `brief/element/hero/view` call sites never send it.** Retrieval is live only for materials, 3-D briefs, and decks. The main promise — “every new activation has context of previous and stored data” — is currently only carried by the deterministic brand block, not the document corpus.
- **Learning loop:** capture (PLANNED — no `learning_events`), distill (PARTIAL — manual Save Learnings + regen-feedback capture), approve (LIVE), retrieve (PARTIAL per above), generate (LIVE). Matches `docs/intelligence/` and the public `/architecture` page.

8.4 Edge functions (29)

Full table in `audit/05-backend-data-ai.md` §2. Highlights: `parse-brief`, `synthesize-brief`, `suggest-brief-field`, `generate-element`, `enrich-spatial`, `generate-hero/generate-view/save-render-image` (the render triple, with the PGRST204 artifacts

fallback and server-side uploads), generate-panorama, polish-rhino-render, generate-video, generate-materials, generate-3d-brief, generate-presentation (Claude designed-deck + pptx + ping), analyze-existing-space, deep-dive-brand, scrape-*, extract-learnings, embed-document, auto-tag-document, rag-retrieve, migrate-legacy-kb, admin-invite-user, admin-manage-role.

Deployed but never invoked from the app: rag-retrieve (standalone), summarize-document, extract-pricing, brand-compliance-check, best-practices-suggest — five finished capabilities with no product surface. summarize-document and brand-compliance-check are additionally **schema-mismatched** against brand_intelligence (columns that don't exist). **Security posture:** ~14 functions run `verify_jwt = false`; several re-auth internally, but generate-hero performs no authentication and the shared `access-gate.ts` (suspension check for edge functions) is imported by zero functions.

8.5 Models & telemetry

Task → model roster: Gemini 2.5 Flash (parse/synthesize/tag/learnings/rerank), Flash-Lite (field suggestions), 2.5 Pro (elements, enrichment, vision, deep-dive), Gemini 3 Flash preview (materials/3-D), `gemini-embedding-001` (768-d), Claude Sonnet (decks/summaries/compliance), `gpt-image-2` → Gemini image previews (renders), Runway/Kling/Veco (video). Client-facing model names (Signature/Studio/Draft/Typographic) hide providers — but **the per-agency image-model preference is INERT** (both render functions `void imageModel`). Every AI call fire-and-forget logs to `ai_usage_events` with a hardcoded price table; the platform AI-usage dashboards read it (LIVE).

9. Consolidated gap register

The single prioritized list, deduplicated across all five audits. **P0 = broken promise or data-integrity risk** · **P1 = built-but-disconnected intent** · **P2 = duplicated/legacy systems to consolidate** · **P3 = polish**.

P0 — broken or risky now

1. **projects.agency_id never stamped on insert.** Weakens agency RLS for all new projects, broke client counts (worked around), and makes `price_plan` raise for every post-onboarding project. One-line fix at insert + backfill; unlocks pricing and honest agency scoping.
2. **RAG retrieval disconnected from the core flow** — no `agency_id` sent by `parse-brief/element/hero/view` call sites; the knowledge corpus never reaches briefs, concepts, or renders.
3. **Element edits and deck state are volatile** — detail-panel edits never persist; designed decks are `localStorage`-only.
4. **Multi-config leaks:** `enrich-spatial` and all Export payloads hardcode `configs[0]`, ignoring the selected size.
5. **Hanging-element split-brain:** canvas-authored hanging elements never reach the prompt; Review discards dimension edits.

6. **Route/auth confusions:** dead `/platform-invites` nav item → 404; blank first paint on `/admin` for super admins; login discards redirect intent (breaks invite links); `revoke_super_admin` can empty the roster server-side; `admin-invite-user` 403s for pure super admins.
7. **Suspension is soft where it matters:** `canWrite` unused (raw RLS errors instead of disabled UI), edge functions unmetered by access status (AI spend continues), RLS coverage partial.
8. **Step/status fiction:** `status: "completed"` and `render_prompts` never written → pipeline UI lies; pill completion positional.
9. Brief file signed URLs expire after one year with no refresh path.

P1 — authored intent with no effect (wire or remove)

10. `project_type_configs` (render context + AI guidance editor) read by nothing.
11. `agencies.feature_flags` and `quotas` — full admin UI, zero readers.
12. Per-agency image model preference voided by both render functions.
13. Style presets dead on the composed prompt path.
14. Company-profile brand colors/tagline/contacts and `logo_dark_url` unconsumed.
15. Subscription tier pills decorative; no billing model anywhere.
16. Five deployed edge functions never invoked (`rag-retrieve`, `summarize-document`, `extract-pricing`, `brand-compliance-check`, `best-practices-suggest`) — two of them schema-mismatched.
17. `pricing_sources/pricing_quotes` have no writers — the provenance chain resolves to override-or-unpriced only.
18. `activation_type_agency` + `industry` KB scopes embedded but never retrieved.
19. Preview mode: no read-only enforcement, no agency binding, no persistence.
20. Invites that never email (`pending_invites` everywhere; super-admin grant only applies at onboarding).

P2 — parallel systems to consolidate

21. Two project knowledge bases (`knowledge_documents` vs legacy `knowledge_base_files`) — Step 03 reads only the legacy one; `migrate-legacy-kb` exists but is buried.
22. Two team systems (`agency_members` vs `team_members`) — both shipping, different doors.
23. Three invite systems (§2).
24. Four project-type systems (§5.8).
25. Orphan surfaces: `/rhino`, `/knowledge-base`, `/team`, commented `/explore`; duplicated Rhino UI.
26. Per-user vs per-agency confusion: `company_profiles`, `project_type_configs`, venue intelligence — all “agency” concepts keyed to `user_id`.

P3 — polish

27. Brand identity drift: “BriefEngine” on invite page, `hello@exhibitus.com` on suspension, “4-scope vs 5-scope” copy, off-brand 404, no mobile nav on landing.

28. `interior_design` second-class across icon/hero/copy/seed maps.
 29. Dead buttons/fields: Export SVG, `badge_scan_cost`, Share/Archive toasts (see P0/P1 for their real fixes).
 30. Measurement system `localStorage`-only; prompt versions `localStorage`-only (schema column missing).
 31. Dark-theme token classes on `SuspensionBanner` inside the light shell.
-

10. Requirements statement (forward intent)

Condensing the above into the platform's requirement set — each item is already designed and partially built; none requires invention:

1. **One tenancy truth.** Stamp `agency_id` at project insert; migrate per-user “agency” tables (`company_profiles`, `project_type_configs`) to agency scope; one team system (`agency_members`); one invite system with real email delivery and acceptance outside onboarding.
 2. **Memory that actually compounds.** Pass agency/client/activation/project scope into every generation call so the built RAG layer runs; merge the legacy KB; retire dead scopes or wire them; land `learning_events` (Phase 1 of `docs/intelligence/roadmap.md`) so capture stops being manual.
 3. **One status model.** Persisted per-step status driving pills (incl. `blocked`), routing, and project cards; write `completed`; make blocking gaps block.
 4. **Config fidelity end-to-end.** The active booth size selected once must govern enrichment, preflight, renders (already does), exports, and pricing.
 5. **Honest enforcement.** `canWrite` consumed by the UI; `access-gate` imported by spend-incurring edge functions; server-side last-super-admin lock; role guards at the routing layer.
 6. **Close the authoring ↔ effect loops.** Either wire `project_type_configs`, feature flags, quotas, image-model preference, style presets, and company branding into their consumers — or remove the editors. A control that does nothing is debt with a UI.
 7. **Ship the buried capabilities.** Share links (`project_invites` → the Share menu), compliance check and best-practices as surfaces, pricing sources/quotes authoring (rate-card import), document summarize/extract-pricing on the KB rows.
-

Maintained alongside the code in `docs/frd/`. The audit appendices are point-in-time (2026-08-12); update them by re-running the domain audits after major changes. Companion documents: `docs/intelligence/` (intelligence-layer architecture + roadmap), `design/flows/` (Flow C artboards 24–50 + SPEC), the public `/architecture` page.

\newpage

CANOPY — Domain Audit: Application Shell, Access Control & Public Surfaces

Read-only audit. All findings verified against source at
/Users/bryanscott/Desktop/Brief to Booth.

Appendix A — Application Shell, Access Control & Public Surfaces

Provider stack (outermost → innermost): `AppErrorBoundary` → `QueryClientProvider` → `AuthProvider` → `CacheClearGuard` (`useClearCacheOnUserChange`) → `PlatformOwnerProvider` → `TooltipProvider` → `Toaster` + `Sonner` → `BrowserRouter` → `Suspense` (`LazyFallback`) → `Routes`. Vercel Analytics + SpeedInsights mount alongside the router.

Eager (in main bundle): `Index`, `Auth`, `NotFound`. Everything else is `React.lazy` code-split.

Path	Component	Guard	Load	Reachable by
/	Index	none	eager	anyone (public)
/auth	Auth	none	eager	anyone (public)
/architecture	Intelligence Architecture	none	lazy	anyone (public)
/invite/:token	AcceptInvite	none (self-guards)	lazy	anyone; self-redirects to /auth if signed out
/industries/:slug	IndustryDetail	none	lazy	anyone (public)
/projects	Projects	ProtectedRoute (full)	lazy	authed + agency + not locked out
/upload/review/generate/spatial/prompts/files/export	Upload Review Generate Spatial Prompts Files Export	ProtectedRoute (full)	lazy	as above
/pricing	Pricing	ProtectedRoute	lazy	as above (per-project BOM editor, ?project=)

Path	Component	Guard	Load	Reachable by
/agency/pricing	AgencyPricing	ProtectedRoute	lazy	as above
/rhino	Rhino	ProtectedRoute	lazy	as above — no nav entry point anywhere
/knowledge-base	KnowledgeBase	ProtectedRoute	lazy	as above — no nav entry point anywhere
/company	CompanyProfile	ProtectedRoute	lazy	as above
/admin	AdminSettings	ProtectedRoute only	lazy	any authed user — no role guard in the route; the page self-branches on <code>isSuperAdmin && !previewMode</code>
/account/:userId	AgencyAccount	ProtectedRoute only	lazy	any authed user (no role check found)
/team	Team	ProtectedRoute	lazy	as above — no nav entry point (superseded by <code>/agency/team</code>)
/clients	Clients	ProtectedRoute	lazy	as above
/clients/:clientId	ClientDashboard	ProtectedRoute	lazy	as above
/agency/knowledge	AgencyKnowledge	ProtectedRoute	lazy	as above
/agency/activation-types	ActivationTypes	ProtectedRoute	lazy	as above
/agency/activation-types/:typeId	ActivationTypeDashboard	ProtectedRoute	lazy	as above
/agency/team	AgencyTeam	ProtectedRoute	lazy	as above
/admin/super-admins	SuperAdmins	ProtectedRoute	lazy	super admins

Path	Component	Guard	Load	Reachable by
		te + page-level if (! isSuperAdmin) <Navigate to="/project s">		
/admin/ agencies	AdminAgencie s	ProtectedRou te only	lazy	any authed user client-side; server RPCs enforce super- admin
/admin/ industries	AdminIndustr ies	ProtectedRou te only	lazy	any authed user client-side
/admin/ industries/: slug	AdminIndustr yDashboard	ProtectedRou te only	lazy	any authed user client-side
/access- suspended	AccessSuspen ded	ProtectedRou te enforceAcces sGate={false } enforceOnboa rding={false }	lazy	any authed user
/onboarding/ create- agency	OnboardingCr eateAgency	ProtectedRou te both gates off	lazy	any authed user
/suite	Suite	ProtectedRou te (full)	lazy	as above; entered only via SuiteContext Bar
*	NotFound	none	eager	anyone

Commented-out / dead routes:

- `const Explore = lazy(...)` (line 54) and the `/explore <Route>` block (lines 363–368) are commented out: “360° Explorer hidden — kept for future re-enable”.
`src/pages/Explore.tsx` and
`src/components/explore/{BoothExplorer,PanoramaViewer}.tsx` still ship in the repo but are unreferenced. The matching step is also commented out in `ProjectHeader.tsx` (Compass icon import + `/explore` step).
- `Calculator/pricing` is commented out of `PROJECT_STEPS` with a note that pricing is now agency-level, not a project step.

Dead nav target: `AppSidebar.platformOwnerNavItems` includes `{ path: "/platform-invites", label: "Invites" }` — **no such route exists**, so a super admin clicking “Invites” lands on the 404 page. (The invites UI actually lives inside `/admin` → “Invites & Team” tab, and query keys `admin-platform-invites` exist in `useAdminRole`.)

Error/reload behavior (`AppErrorBoundary` + `main.tsx`): distinguishes chunk-load failures (stale bundle → auto-reload once per 10s cooldown keyed on `sessionStorage["canopy:stale-chunk-reload-attempted"]`, plus a cache-busting `?_canopy_reload=` manual refresh) from render errors (offers “Try again” soft reset and “Go to dashboard”). `main.tsx` duplicates this for `vite:preloadError` and `unhandledrejection` before React mounts.

2. Screens

[/ — Landing \(`src/pages/Index.tsx`\)](#)

Route & access · Public, eager-loaded, no guard. Dark theme (`canopy-dark`).

Intent · Top-of-funnel marketing page positioning Canopy as “a spatial operating system for environment design” — sell the multi-industry, RAG-grounded brief → render → deck pipeline and push visitors into `/projects` (which bounces them to sign-in) or `/auth`.

UI anatomy (top → bottom)

1. **Fixed glass header** — `CanopyLogo` (horizontal, byline) → `/`; anchor nav `#showcase` “Work”, `#industries` “Industries”, `#system` “System”, `#intelligence` “Intelligence”; “Sign in” ghost → `/auth`; “Enter” primary → `/projects`. Nav is hidden `md:flex` — **no mobile menu at all**.
2. **Hero** — 13-image cross-fading background (`HERO_ROTATION`, 5.5s interval, parallax via `useScrollly`), pill eyebrow, H1 “Design the space / before you build it.”, body paragraph, CTAs “Start a project” → `/projects` and “See the work” → `#showcase`, plus a 3-item trust bar: “Multi-tenant for agencies + teams”, “4-scope RAG with `pgvector`”, “AI skills on every upload”.
3. **#showcase** — 6-tile bento grid (`SHOWCASE`): Uber, Samsung, Tesla, Topps + two generic (Energy/Sport, Hospitality). Copy explicitly disclaims: examples are experiential-vertical; other verticals “coming online”.
4. **#system** — 4 `PILLARS` cards: Spatial intelligence, 5-scope RAG engine, AI skills built-in, Brief to render in a day.
5. **#industries** — 5 image cards linking to `/industries/:slug` (experiential, architecture, landscape, entertainment, `audio_visual`) + a dashed “Beyond the launch verticals” placeholder tile (themed entertainment, retail, hospitality, sports venues, public art).
6. **Spatial planning feature** — 3D-tilted floorplan image with animated scan line and 4 floating metric chips (“Demo · 142 sqft”, “Reception · flow A+”, “VIP Lounge · 96 sqft”, “Meeting · adj ✓” — all hardcoded decoration), plus a 4-item checklist.

7. **#intelligence** — RAG network image with a 5-scope weight strip (Industry 0.70, Agency 1.00, Project Type 0.92, Client Brand 0.85, Project 1.00) + 4 INTELLIGENCE cards (Brand intelligence, Project type playbooks, Brand compliance audit, Cost intelligence).
8. **#workflow** — 6-step rail: Brief → Parse → Ground → Generate → Plan → Deliver.
9. **Stats strip** — < 24h brief-to-first-render · 5 knowledge scopes · 5 industries · 60+ project types.
10. **Parallax gallery band** — 8 scrolling render images.
11. **Final CTA** — “Upload your first brief” → /projects; “Sign in” → /auth.
12. **Footer** — logo, tagline, links to /auth and /projects (“Dashboard”).

Inputs · Zero user input, zero data loading. All content is module-level constants (SHOWCASE, PILLARS, INDUSTRIES, WORKFLOW, INTELLIGENCE, STATS) plus static @/assets imports. Only reactive state: mounted (60 ms entrance delay), heroIdx (rotation), useScrollly().

Workflows · Anchor scrolls; “Enter”/“Start a project”/“Upload your first brief”/“Dashboard” → /projects → ProtectedRoute → /auth for anonymous visitors; “Sign in” → /auth; industry card → /industries/:slug.

Outputs & side effects · None. No DB, no storage, no edge functions. Vercel Analytics/Speed Insights fire globally from App.

Current-state gaps

- **Claim inconsistency on the same page:** hero trust bar says “4-scope RAG with pgvector”; pillars say “5-scope RAG engine”; stats say “5 Knowledge scopes”; intelligence section lists 5 scopes. `index.html` OG description also says “4-scope RAG” and “Claude skills built-in” (product copy elsewhere says “AI skills”).
 - The landing never links to /architecture — that public page is effectively unlisted/share-by-URL only.
 - Anonymous CTAs point at /projects, so the primary CTA path is CTA → guard redirect → /auth, not a direct sign-up. Combined with invite-only auth, “Start a project” is not actually actionable for a new visitor.
 - No mobile navigation (hidden `md:flex`), so on phones the header has no Sign in / Enter affordance.
 - Stats (“5 industries”, “60+ project types”) are hardcoded and can drift from the `industries / activation_types` tables that `IndustryDetail` reads live.
-

/industries/:slug — Industry Detail (`src/pages/IndustryDetail.tsx`)

Route & access · Public, lazy. Renders for anonymous visitors via a constants fallback.

Intent · Per-vertical marketing deep-dive that proves Canopy “speaks your industry’s language” — vocabulary swap, capabilities, deliverables, supported project types.

UI anatomy · Fixed glass header (logo, “All industries” → /#industries, Sign in, Enter) · Hero (industry image, icon pill “Canopy for {label}”, tagline H1, pitch, CTAs) · **Vocabulary**

grid (one card per vocabulary key/value) · **Capabilities / What you ship** two-column lists · **Project types** grid (count + cards) · CTA band with Badge "Built for {label}" · Footer ("© Canopy — A spatial OS for environment design", back to home).

Inputs

- Route param `slug`.
- Live read: `industries` table (`slug`, `label`, `description`, `icon`, `vocabulary`, `.maybeSingle()`).
- Live read: `activation_types` where `industries @> [slug]` and `is_builtin = true`, ordered by label.
- Fallback: `BUILTIN_INDUSTRIES` from `src/lib/builtinIndustries.ts` when the DB read returns nothing (anonymous visitors — the `industries` table is RLS-gated to authenticated users).
- Static: `NARRATIVE` map (tagline/pitch/capabilities/deliverables per slug for all 5 verticals), `HERO_BY_SLUG` images, `ICON_MAP`.

Workflows · Copy interpolates vocabulary: CTA reads "Start a {vocab.project}", "Bring us a {vocab.brief}". Unknown slug → minimal "Industry not found" screen with a Back-to-home button (not the global 404). CTAs → `/projects`, `/auth`, `/#industries`.

Outputs & side effects · Read-only Supabase selects. Mutates `document.title` and injects/updates a `meta[name=description]` tag for SEO.

Current-state gaps

- SEO title/description are set but never reset on unmount, so navigating back to `/` leaves the industry title in place (SPA, no head manager).
 - For anonymous visitors the **project-types section silently disappears** (RLS blocks the read, `projectTypes` stays empty) — the vertical looks thinner to the exact audience the page targets.
 - Trailing `void cn;` at line 508 with a comment admitting `cn` is imported only "defensively" — dead import.
 - `NARRATIVE` is hardcoded per slug; a new industry seeded in the DB renders with no tagline/capabilities/deliverables and no hero image.
-

[/architecture — Intelligence Architecture](#)
(`src/pages/IntelligenceArchitecture.tsx`)

Route & access · Public, lazy, no guard, no auth. Light theme (`bg-cloud text-navy`, forced `colorScheme: light`).

Intent · A shareable "living document" that explains how Canopy stores, retrieves, and learns from agency/client data — explicitly the visual twin of `docs/intelligence/`, intended for prospects/stakeholders and honest about what is live vs. planned.

UI anatomy · Top bar (CanopyMark + “CANOPY” + “Intelligence architecture · living document”, link “canopy.gofightwin.co →” → /) · Navy hero band (“Memory that compounds”) with two cards: Client memory, Agency memory · **The learning loop** — 5 cards (Capture *planned*, Distill *partial*, Approve *live*, Retrieve *partial*, Generate *live*, marked generative ♦) + a pink invariants callout (“nothing auto-extracted reaches generation without human approval”; “client scope is a wall”) · **Data model** — 6 table cards with status pills and field chips: brand_intelligence (live), brand_guidelines (live), knowledge_documents + embeddings (live), project_images.prompt_artifacts (live), Agency-scope data (live), learning_events (**planned**) · **Learning signals** — 10-row table (signal → what it teaches → status; 4 live/partial, several planned) · **Roadmap** — 4 phases, all *planned* · Footer (“Maintained alongside the code in docs/intelligence · Canopy by Exhibitus”, back to Canopy).

Inputs · None — entirely static local constants (LOOP_STEPS, TABLES, SIGNALS, PHASES). Deliberately “dependency-light” with local StatusPill / SectionLabel primitives instead of the shell kit.

Workflows / outputs · Two links back to /. No writes, no reads, no side effects.

Current-state gaps

- Publicly exposes internal schema names, roadmap, and “planned” gaps to anyone with the URL, with no noindex and no access control.
 - Content is manually duplicated from docs/intelligence/ — the file header itself instructs “keep both in sync”, i.e. drift is unmanaged.
 - It defines its own SectionLabel that shadows the shell-kit SectionLabel (different API: swatch/color vs accent).
 - Unreachable from any nav or footer in the app or on the landing page.
-

/auth — Sign In / Recovery / Beta Waitlist (src/pages/Auth.tsx)

Route & access · Public, eager. Single-card surface with four mutually exclusive modes.

Intent · The only login door. Account creation is **invite-only** by design (explicit comment: signUp is deliberately *not* destructured; the public sign-up path was retired in favor of a beta waitlist).

UI anatomy · Dark ambient background (grid pattern + two CanopyAmbientGlow orbs) · CanopyLogo (stacked, byline) → / · one canopy-panel card rendering one of:

1. **Recovery** (?type=recovery): “Set a new password” — New password, Confirm password, “Update password”.
2. **Forgot password** (showForgotPassword): back-link, Email, “Send reset link”.
3. **Beta request** (showBetaRequest): back-link, Work email (required), Your name, Agency or company, “What kind of work are you bringing to Canopy?” (textarea, maxLength 1000), “Request access”; on success swaps to a “You’re on the list” confirmation panel.

4. **Sign in** (default): Badge "Invite-only beta", Email, Password with "Forgot?" toggle, "Sign in", and "Don't have an account? Request beta access". · Footer line: "By continuing you agree to Canopy's terms and privacy policy." (plain text — **not linked**).

Inputs · `useSearchParams().get("type")`; uncontrolled `FormData` on every form; `useAuth().signIn`; direct `supabase.auth.resetPasswordForEmail / updateUser`; `supabase.from("beta_waitlist").insert`.

Workflows

- **Sign in** → `signIn(email, password)` → on error, destructive toast "Sign in failed" + `error.message`; on success toast "Welcome back" and `navigate("/projects")` — **unconditionally**, ignoring any intended destination.
- **Forgot** → `resetPasswordForEmail(email, { redirectTo: origin + "/auth?type=recovery" })` → "Check your email" toast, returns to sign-in.
- **Recovery** → client-side validation (match + min length 6) → `supabase.auth.updateUser({ password })` → toast + `navigate("/projects")`.
- **Beta request** → `insert { email, name, agency, reason, source: "auth_page" }` into `beta_waitlist`; Postgres 23505 (unique-violation on `lower(email)`) is treated as success ("You're already on the list").

Outputs & side effects · Supabase Auth session (persisted by the client, picked up by `AuthProvider.onAuthStateChange`, which then upserts profiles); `beta_waitlist` row insert (RLS: INSERT granted to anon, authenticated; SELECT/UPDATE/DELETE super-admin only, per `20260515000001_beta_waitlist.sql`); navigation to `/projects`.

Current-state gaps

- **Post-login destination is hardcoded.** `ProtectedRoute` sends `state: { from: location }` and `AcceptInvite` sends `?redirect=/invite/:token` — Auth reads **neither**. Deep links are always lost; invite acceptance after sign-in is broken (user lands on `/projects` and must re-open the invite URL).
 - No password strength rules beyond `length >= 6`; no rate limiting or captcha on the waitlist insert (anon-writable table).
 - Beta waitlist has **no notification path** — the code comment states submissions are only visible by polling the Supabase dashboard; a Resend/edge-function notification to `bryan@gofightwin.co` is noted as a future enhancement.
 - `useAuth.signUp` is still exported and implemented (`emailRedirectTo: window.location.origin`) but has zero call sites — dead code that contradicts the invite-only posture.
 - Terms/privacy referenced but no documents or links exist.
 - No email-verification, MFA, or SSO/OAuth path anywhere.
-

`/invite/:token` — Project Invite Acceptance (`src/pages/AcceptInvite.tsx`)

Route & access · Public route; **not** wrapped in `ProtectedRoute` — it self-redirects unauthenticated users. Intentionally outside the onboarding/access gates.

Intent · Redeem a tokenized **project collaboration** invite link and drop the accepter straight into the project.

UI anatomy · Centered card: Link2 icon well, title “Project Invite”, description “You’ve been invited to collaborate on a project in **BriefEngine**”, and one of three bodies — idle (explainer + “Accept Invite” button), success (green check, “Invite accepted!”, “Redirecting to project...”), error (red X, message, “Go to Projects” button).

Inputs · `useParams().token`; `useAuth()` for gating; `useAcceptInvite()` from `src/hooks/useTeam.tsx` → RPC `accept_project_invite(_token)` (server-side token lookup; returns a `ProjectInvite` row with `project_id` and scope: `upload_only` | `view_comment` | `full_edit`).

Workflows · If `!authLoading && !user` → `navigate('/auth?redirect=/invite/' + token)`. “Accept Invite” → `mutateAsync(token)` → success toast from the hook (“You now have {scope} access...”) → `setTimeout(1500)` → `navigate('/upload?project=' + invite.project_id)`. Button is disabled after first attempt (`hasAttempted`).

Outputs & side effects · Server-side invite redemption via RPC (marks accepted, grants project access); navigation to `/upload?project=...`

Current-state gaps

- **Stale branding:** the card says “BriefEngine”, not Canopy — the only place in the shell that still uses the old product name.
 - The `?redirect=` param it constructs is **never honored** by `/auth` → the invite flow dead-ends for signed-out invitees.
 - **Two parallel invite systems coexist:** token-based *project* invites (`accept_project_invite`, this page, `useTeam`, `ProjectInviteManager`) and id-based *agency member / super-admin* invites (`pending_invites`, `accept_pending_invite`, `useAgencyTeam`, surfaced only on the onboarding page). There is no email link that lands on a page for the second kind.
 - Legacy `useTeam.useInviteTeamMember` writes a `team_members` row with the **inviter’s own** `user_id` as a placeholder and a code comment that the accept flow will fix it — no such accept flow exists for `team_members`.
 - No unauthenticated preview of what project/agency the invite is for; no expiry messaging beyond the generic error string.
-

/onboarding/create-agency — Mandatory Agency Onboarding (src/pages/OnboardingCreateAgency.tsx)

Route & access · `ProtectedRoute enforceAccessGate={false}`
`enforceOnboarding={false}` — authenticated users only; both gates disabled because the page is the gate. Every non-super-admin without an agency is force-redirected here.

Intent · Guarantee the tenancy invariant: every signed-in user belongs to an agency. Either accept a pending agency invitation or create your own agency (becoming its owner), which also backfills any orphaned data the user created before agencies existed.

UI anatomy

1. Ambient dark background, CanopyLogo → /.
2. **Pending invitations panel** (only if agency-type invites exist) — “You have N invitation(s)”, one clickable row per invite showing agency name, “Joining as {role}”, and relative invited-at (`formatDistanceToNow`), chevron affordance.
3. **Create-agency CanopyPanel** — eyebrow flips between “One last step” and “Or start fresh”; H1 “Create your agency”; explainer that you’ll be owner.
 - **Agency name** (required, 2–80 chars, autofocus)
 - **Logo URL** (optional, `type=url`)
 - **Industry picker** — 2-col grid of live industries with icon, label, description, selected check; warning copy: “This is **locked once your agency is created** — contact platform support to change later.”
 - **Extra industries** — multi-select chips for cross-vertical agencies, “rare”, also locked after onboarding.
 - **Primary / Secondary brand color** — native color inputs + hex text fields (defaults `#A78BFA`, `#0B1B2B`).
 - Submit “Create agency →”.
4. Footer strip: “Signed in as {email}” + Sign out.

Inputs · `useOnboardingState()` (`hasAgency/isLoading`), `useMyPendingInvites()` → RPC `my_pending_invites()` (matches `lower(auth.jwt()->>'email')`, `status='pending'`, not expired), `useIndustries()` → industries table, `useAuth()`.

Workflows

- Mount effect: if `hasAgency` → `navigate("/projects", { replace: true })`.
- **Accept invite** → `useAcceptInvite` (agency variant) → RPC `accept_pending_invite(_invite_id)` → inserts `agency_members` (`agency_id`, `auth.uid()`, `role`) (or `user_roles.super_admin` for super-admin invites), marks the invite accepted → toast → /projects.
- **Create agency** → client validation (`trimmed name ≥ 2`) → `useCreateMyAgency` → RPC `create_my_agency(_name, _logo_url, _brand_colors, _primary_industry, _industries)`; server creates the agency, makes the caller owner, backfills orphaned projects/clients/KB docs → invalidates `agency`, `clients`,

projects, my-pending-invites → toast “Welcome to Canopy, {name}” → /projects.

- **Sign out** → `signOut()` → /.

Outputs & side effects · DB: agencies insert + agency_members owner row + backfill of orphaned rows (all inside the SECURITY DEFINER RPC); or agency_members insert via invite acceptance. Navigation to /projects.

Current-state gaps

- **Logo is a URL text field, not an upload** — despite the file header claiming “optionally upload a logo”. No validation that the URL resolves.
 - Industry lock is enforced at the DB trigger level (20260427233000_lock_industry_isolation.sql) with the only escape hatch being `admin_set_agency_industries` (super admin, via `useAdminSetAgencyIndustries`) — but the page gives the user no in-product way to request that change beyond “contact platform support”.
 - `useCreateMyAgency` passes 5 args while the migration’s GRANT/COMMENT reference a 3-arg `create_my_agency(text, text, jsonb)` signature — the industry-aware overload comes from a later migration; worth verifying only one signature is live.
 - Brand colors are collected here but there is no preview of where they’re applied.
 - No “leave/decline invite” action — invites can only be accepted or ignored.
-

/access-suspended — Access Suspended Landing (src/pages/AccessSuspended.tsx)

Route & access · `ProtectedRoute` with **both** gates disabled, so locked-out users can actually reach it without a redirect loop.

Intent · Give a suspended / disabled / trial-expired agency a clear, non-punitive dead end: what happened, why, that their data is safe, and how to get unblocked.

UI anatomy · Ambient background + `CanopyLogo` → / · Panel with state-dependent icon (Lock / `AlertTriangle` / `Clock`) and label (“Account disabled” / “Access suspended” / “Trial ended”) · Headline · “{Agency} can no longer create or modify data in Canopy.” · **Reason block** (when `suspensionReason` present) · **Trial ended** block with `formatDistanceToNow(trialEndsAt)` · Reassurance paragraph (state-specific: data preserved / convert to paid) · `mailto:hello@exhibit.us.com` · Footer actions: “Home” → / and “Sign out”.

Inputs · `useAgency()` → { agency, access, isLoading } (from `agency_members` join `agencies`); `useAuth().signOut`.

Workflows · If not loading and access is missing or **not restricted**, it hard-redirects via `window.location.href = "/projects"`. Sign out → `signOut()` then `window.location.href = "/"`.

Outputs & side effects · No writes. Full-page navigations (not React Router).

Current-state gaps

- Contact email is `hello@exhibitus.com`, while the brand elsewhere is Canopy and the operator email is `bryan@gofightwin.co / canopy.gofightwin.co` — three different identities across the shell.
 - **State mismatch with the gate:** the page renders for any `isRestricted` state, but `ProtectedRoute` only routes users here when `isLockedOut` (disabled / trial_expired). A merely *suspended* agency is never redirected here — it keeps full route access and only sees the banner (see §6).
 - No self-serve remediation (no billing/upgrade CTA), no ticket form — email only.
 - Uses `window.location.href` instead of router navigation, discarding SPA state.
 - Renders outside `AppLayout`, so there's no way back into any read-only view of past work.
-

* — [Not Found \(src/pages/NotFound.tsx\)](#)

Route & access · Public catch-all, eager.

Intent · Terminal 404.

UI anatomy · Centered “404” / “Oops! Page not found” / plain `<a href="/">Return to Home</a>` on bg-muted.

Inputs · `useLocation().pathname` only.

Workflows / outputs · `useEffect` logs `console.error("404 Error: User attempted to access non-existent route:", pathname)`. No telemetry, no DB.

Current-state gaps

- Completely off-brand: no `CanopyLogo`, no shell kit, no theme tokens beyond bg-muted; the only page that ignores the design system.
 - Uses a raw `<a>` (full page reload) instead of `<Link>`.
 - Signed-in users get no route back into the app (no “Go to projects”).
 - This is where the broken `/platform-invites` sidebar link lands super admins.
 - 404s are only `console.error`'d — not reported to Analytics.
-

[/suite](#) — [Suite Overview \(src/pages/Suite.tsx\)](#)

Route & access · `ProtectedRoute` (full gates), lazy. Reached only from `SuiteContextBar` (“Suite Overview” button / parent breadcrumb) — no sidebar or step-nav entry.

Intent · Parent-project view for multi-activation “suites”: see the parent, its child activations, add another activation, and roll up budget.

UI anatomy · Inside `AppLayout` · “All Projects” back button · Parent summary Card (name, project type, brand name from `parsed_brief`, status badge from `STATUS_BADGES` map, “N

activations” badge) · “Activations” heading + “Add Activation” button + `<SuiteOverview>` grid · **Budget Rollup** card (only when total > 0) · `<AddActivationPanel>` sheet.

Inputs · `useProjectSync()` → `projectId` (from `?project=`);
`useProject(projectId); useProjectSuite(projectId, null)` → { `parent`,
`children`, `siblings` }; `parentProject.budget_logic.totalPerShow`.

Workflows · Add Activation → panel → on created → `navigate('/review?project=' + newId)`. Child card click (in `SuiteOverview`) → project routes. Not-found → “Project not found.” + back button.

Outputs & side effects · Writes happen inside `AddActivationPanel` (new child project row); this page itself only navigates.

Current-state gaps

- **Budget rollup is a stub.** The code comment states it plainly: “We’d need full child data for `budget_logic` — for now use parent if available”. The “rollup” sums exactly one number (the parent’s), so a suite’s children contribute nothing.
 - Non-null assertions `projectId!` are passed to `SuiteOverview / AddActivationPanel` even though the page can render with a missing id.
 - Uses raw shadcn Cards rather than the Flow C shell kit (`WorkSheet`, `PageHeader`), so it’s visually out of step with the newer pages.
 - `STATUS_BADGES` is a locally duplicated status map (also duplicated in project list surfaces).
-

3. Auth, Roles & Permissions

AuthProvider / `useAuth (src/hooks/useAuth.tsx)`

- **Contract:** { `user`, `session`, `isLoading`, `signIn`, `signUp`, `signOut` }.
- **Session handling:** registers `supabase.auth.onAuthStateChange` **before** calling `getSession()` (deliberate ordering to avoid missing the initial event). Both paths set `session/user` and clear `isLoading`.
- **User-switch safety:** tracks `prevUserId` in a ref; when the id changes it calls `useProjectStore.getState().resetProject().signOut()` also resets the store before `supabase.auth.signOut()`. Complemented by `CacheClearGuard` → `useClearCacheOnUserChange`, which calls `queryClient.clear()` on any user-id change (explicitly “prevents data from one user leaking into another user’s session”).
- **Profile side effect:** on every auth event and on initial session, upserts profiles { `user_id`, `email`, `display_name` (`user_metadata.display_name || full_name`), `avatar_url` } with `onConflict: user_id`, so admins can see names/emails. `useEnsureProfile` in `useAdminRole` duplicates this.
- **Post-login:** the provider does nothing navigational; `Auth.tsx` hardcodes `/projects`.

- **Gaps:** `signUp` exists but is unused; the `profiles` upsert result is unchecked (failures are silent in `AuthProvider`, only logged in `useEnsureProfile`); no session-expiry UX beyond the guard redirect.

`useAdminRole (src/hooks/useAdminRole.tsx)`

Hook	What it does	Source
<code>useIsAdmin()</code>	Selects all <code>user_roles</code> rows for the user; true if roles include <code>admin</code> or <code>super_admin</code> . 5-min <code>staleTime</code> . Errors resolve to false.	<code>user_roles</code>
<code>useIsSuperAdmin()</code>	<code>user_roles</code> where <code>role='super_admin'</code> , <code>.maybeSingle()</code> → boolean. 5-min <code>staleTime</code> . Errors → false.	<code>user_roles</code>
<code>useAdminProfiles()</code>	Gated on <code>isAdmin</code> . RPC <code>get_all_user_profiles()</code> + all projects, grouped by <code>user_id</code> into <code>ProjectSummary[]</code> .	RPC + projects
<code>useAdminUsers()</code>	Legacy: projects grouped by user.	projects
<code>useInviteUser()</code>	Edge fn <code>admin-invite-user</code> with Bearer token.	edge function
<code>usePlatformInvites()</code>	<code>platform_invites</code> list, gated on <code>isAdmin</code> .	<code>platform_invites</code>
<code>useManageAdminRole()</code>	Edge fn <code>admin-manage-role</code> with <code>grant_admin</code> <code>revoke_admin</code> <code>grant_super_admin</code> <code>revoke_super_admin</code> .	edge function
<code>useGrantAdminRole / useRevokeAdminRole</code>	Direct <code>user_roles</code> insert/delete, “kept for backward compat”.	<code>user_roles</code>
<code>useEnsureProfile()</code>	Idempotent profiles upsert, <code>staleTime: Infinity</code> .	profiles

Role model observed: two orthogonal axes — **platform roles** in `user_roles` (`admin`, `super_admin`) and **agency roles** in `agency_members` (`owner` > `admin` > `member` > `viewer`, priority-ordered in `useAgency`). Nothing in the routing layer consumes agency role; only `useIsAdmin`/`useIsSuperAdmin` affect UI.

Gaps

- **ProtectedRoute checks no roles at all.** Admin routes (`/admin`, `/admin/agencies`, `/admin/industries`, `/admin/industries/:slug`, `/account/:userId`) are reachable by any authenticated, onboarded, non-locked-out user. Only `/admin/super-admins` self-guards. Defense rests entirely on server RPC/RLS enforcement, so non-privileged users see chrome and empty/erroring panels rather than a clean denial.
- **admin-invite-user edge function requires role = 'admin' exactly** (`.eq("role", "admin").maybeSingle()`), so a pure `super_admin` (no `admin` row) gets 403 Forbidden: `admin only` — inconsistent with `useIsAdmin`, which treats super admins as admins.
- Both role queries **swallow errors as false**, so an RLS/network failure silently downgrades a super admin to a regular user (they'd be bounced from `/admin/super-admins` and, if agency-less, pushed into onboarding).
- 5-minute `staleTime` means role changes take up to 5 min to reflect unless a mutation invalidates.
- `useGrantAdminRole/useRevokeAdminRole` bypass the audited edge function entirely (direct table writes) — parallel, unaudited privilege paths.

PlatformOwnerContext (`src/contexts/PlatformOwnerContext.tsx`) — Preview Mode

- **API:** { `previewMode: boolean`, `setPreviewMode` }, backed by plain `useState` in a provider mounted inside `App`. Default context value is a no-op, so consumers outside the provider silently do nothing.
- **Consumers:** `AppSidebar` (nav-set switch, banner, enter/exit buttons), `AdminSettings` (`isPlatformView = isSuperAdmin && !previewMode` chooses between the Platform Admin and Agency Settings tab sets), `AgencyAccount` (`handlePreviewAsAgency` → `setPreviewMode(true)` → `/projects` + an info toast “Preview mode active — browsing as Agency Admin ... Use the banner to exit”).
- **Intent:** let a super admin see the product the way an agency admin sees it.

Gaps

- **Preview mode is presentation-only.** The sidebar banner promises “**Read-only agency view**”, but nothing in the codebase consults `previewMode` to disable writes — no mutation, RLS policy, or form is gated by it. A super admin in preview mode has full write privileges.
- **It has no target agency.** Entering preview from `/account/:userId` does not scope anything to that user's agency; the app just shows the super admin's *own* `useAgency()` context. “Preview as Agency Admin” is generic, not impersonation.
- State is not persisted (no `localStorage/URL` param), so a refresh silently drops the super admin back into Platform Admin nav mid-task.

ProtectedRoute (`src/components/layout/ProtectedRoute.tsx`)

Props: `enforceAccessGate = true`, `enforceOnboarding = true`.

Evaluation order:

1. **Loading** — while `useAuth().isLoading || useAgency().isLoading || useOnboardingState().isLoading`, render a full-screen spinner (blocks paint until *all three* resolve).
2. **Auth gate** — `!user → <Navigate to="/auth" state={{ from: location }} replace />`.
3. **Onboarding gate** — if `enforceOnboarding && !isSuperAdmin && !pathname.startsWith("/onboarding") && pathname !== "/access-suspended" && onboarding.needsOnboarding` → `/onboarding/create-agency`.
4. **Access gate** — if `enforceAccessGate && !isSuperAdmin && pathname !== "/access-suspended" && access?.isLockedOut` → `/access-suspended`.

`useOnboardingState()` composes `useAuth + useAgency + useIsSuperAdmin + useMyPendingInvites`; `needsOnboarding = user && !isLoading && !hasAgency && !isSuperAdmin && !hasPendingAgencyInvites` (a pending agency invite defers onboarding so the user accepts rather than creating a duplicate agency).

Gaps

- `state.from` is captured but never consumed (see `/auth`).
- **Suspended ≠ locked out**: only disabled and `trial_expired` trigger the redirect. A suspended agency keeps full navigation (writes fail at RLS, surfaced only as raw errors).
- Super admins bypass **both** gates, so a super admin without an agency browses agency-scoped pages with `agency = null` — several hooks are enabled: `!!agencyId` and will render empty rather than explaining why.
- The three parallel loading dependencies mean four sequential round-trips (`auth → agency → super-admin → invites`) before any protected page paints.
- No role prop (`requireSuperAdmin / requireAgencyAdmin`) exists, which is why role gating is ad hoc per page.

Agency membership resolution — `useAgency (src/hooks/useAgency.tsx)`

- Query key `["agency", user.id]`; selects `agency_members.role, agencies:agency_id(*)` for the user; if zero rows → `{ agency: null, role: null }`.
- Multiple memberships are sorted by `ROLE_PRIORITY` (owner 0 > admin 1 > member 2 > viewer 3) and the **highest-priority membership wins** — the app is effectively single-agency per session.
- Normalizes the embedded relation (array or object) into `Tables<"agencies">`.
- `deriveAccessState()` computes the full `AgencyAccessState` (see §6).
- `useUpdateAgency()` — mutation patching the primary agency; invalidates `["agency"]`.

Gaps: no agency switcher UI anywhere despite multi-membership being modeled; `refresh()` invalidates only the user-scoped key while `useUpdateAgency` invalidates the whole `["agency"]` prefix (inconsistent); the hook returns `query.isLoading` (not `isFetching`), so background refetches after suspension changes don't show a loading state.

4. Layout Shell

`AppLayout (src/components/layout/AppLayout.tsx)`

Wraps protected pages: `SidebarProvider` → flex row on `bg-cloud` → `<AppSidebar />` + column of `<SuspensionBanner />`, `<ProjectHeader />`, scrollable `<main>`. Prop `surface?: "dark" | "light"` is **legacy and ignored** — both values render the same Flow C light shell (kept only so existing `surface="light"` call sites compile). Used by 27 pages; not used by `Auth`, `Index`, `IndustryDetail`, `IntelligenceArchitecture`, `AcceptInvite`, `AccessSuspended`, `OnboardingCreateAgency`, `NotFound`.

`AppSidebar (src/components/layout/AppSidebar.tsx)`

Collapsible (`collapsible="icon"`) sidebar with tooltip labels when collapsed.

Nav set A — Agency (`agencyNavItems`): `All Projects /projects` · `Clients /clients` · `Activation Types /agency/activation-types` · `Agency Knowledge /agency/knowledge` · `Pricing /agency/pricing` · `Company Profile /company` · `Team /agency/team`.

Nav set B — Platform Owner (`platformOwnerNavItems`): `Accounts /admin` · `Agencies /admin/agencies` · `Industries /admin/industries` · `Super Admins /admin/super-admins` · `Invites /platform-invites` (*no route* → 404).

Selection logic: `showPlatformNav = isSuperAdmin && !previewMode`. Conditional extra item: “Admin Settings” (`/admin`, Shield) shown when `(isAdmin && !isSuperAdmin) || (isSuperAdmin && previewMode)`.

Active-state rules (`isActive`) keep the parent lit on detail routes: `/clients/*`, `/agency/activation-types/*`, `/admin/industries/*`, and `/pricing` keeps `/agency/pricing` active.

Header: logo link targets `/admin` for platform nav else `/projects`; super admins in platform mode get an amber Crown tile + “Platform Admin” caps label instead of the CanopyMark + “CANOPY” wordmark. Collapse/expand chevron buttons.

Preview banner (expanded): amber card with Eye icon, “Preview Mode”, “Exit” (EyeOff) → `setPreviewMode(false)` + `navigate("/admin")`, sub-line “**Read-only agency view**”.

Collapsed: amber Eye pill with tooltip “Exit Preview Mode”.

Enter-preview control: super admin in platform mode gets a dashed “Preview as Agency Admin” button (or icon button when collapsed) → `setPreviewMode(true)` + `navigate("/projects")`.

Workspace label: caps-mono line above the nav reading "Platform" for platform nav, else `agency?.name ?? "Workspace"`.

Footer: avatar (Crown on amber for super admins, else email initial on primary/10) + truncated `user.email` + Logout icon button → `signOut()` then `navigate("/")`; when collapsed, just the tooltip'd sign-out button.

Gaps: dead `/platform-invites` item; "Read-only agency view" is a false claim; no link to `/rhino`, `/knowledge-base`, `/team`, or `/suite`; agency admins have no way to reach the agency-scoped Team page other than `/agency/team` while the orphan `/team` page still exists; no agency switcher for multi-membership users.

ProjectHeader (src/components/layout/ProjectHeader.tsx)

Route-aware header rendered by `AppLayout`.

- **Non-project routes:** a 12px-tall bar containing only the `SidebarTrigger`.
- **Project routes** (`PROJECT_STEPS` paths): three stacked rows —
 1. `ProjectBar` (sidebar trigger as leading, breadcrumb "Projects / {project.name || 'Untitled Project'}") + a mono **spec pill** derived from `parsed_brief.booth_size ?? boothSize` and `parsed_brief.budget` joined with `·` + a **measurement-system dropdown** (Ruler icon, shows m/ft, faint dot when auto-detected; options Imperial (ft, sqft) / Metric (m, sqm); "Reset to auto-detect" removes `localStorage["canopy:project-measurement-system:{projectId}"]` and calls `window.location.reload()`).
 2. `<SuiteContextBar />`.
 3. `StepPillNav` over `PROJECT_STEPS`: Brief /upload · Review /review · Generate /generate · Spatial /spatial · Prompts /prompts · Files /files · Export /export. Step status is purely positional — `index < current = complete`, `current = active`, else pending. Targets carry `?project={id}` when present.
- **Data:** `useProjectSync()` → `dbProject`; `useSearchParams().get("project")`; `useMeasurementSystem(projectId, parsed_brief)`.

Gaps: step completion is derived from position in the array, not real project state — steps you've never visited show as "complete" once you're past them, and `StepPillNav`'s blocked status is never produced by this caller. Measurement preference persists to **localStorage only** (comment: "DB column when Lovable's pipeline cooperates"), so it doesn't follow the user across devices or teammates. "Reset to auto-detect" does a full page reload. Commented-out Calculator (pricing) and Compass (360°) steps remain.

SuiteContextBar (src/components/layout/SuiteContextBar.tsx)

Renders only when the current project has a parent or children. Child view: parent-name breadcrumb → `/suite?project={parentId}`, current name, and a sibling `Select` that navigates to `/review?project={siblingId}`. Parent view: name + "+ N activations". Always right-aligns a "Suite Overview" button → `/suite?project={parentId} ??`

projectId}. Data from useProjectStore.currentProject.hierarchy.parentId + useProjectSuite.

Gaps: sibling switch always lands on /review regardless of the step you're on; reads project identity from the Zustand store while ProjectHeader reads it from useProjectSync/query params (two sources of truth); styled with generic muted tokens rather than the Flow C grammar.

Shell component kit (src/components/shell/)

Component	Purpose / API
CanopyMark	Inline SVG brand mark — 8 gradient canopy segments + 8 tip dots, useId-scoped gradient. { size = 28, className }.
PageHeader	Site-level page header: caps-mono eyebrow, 28/34 navy title, inline titleAside chips, one-line subtitle, right actions, optional leading visual. Presentation only.
EmptyState	Flow C empty-state grammar — cloud icon well, navy title, ≤2 slate lines, primary action. { icon, title, body?, action?, className? }.
IconWell (same file)	Cloud r8 icon container with navy 1.3px glyph; generative flips to the action-gradient square with a white glyph (AI contexts). { icon, size = 36, generative, className }.
SectionLabel	Caps-mono in-page section heading with an 8px gradient swatch; accent ∈ sky/blue/violet/purple/pink/slate maps to a bright swatch + contrast-safe text hex.
StatusChip	Small caps pill encoding the status color grammar: blocking (solid red), warning, attention, pass, generating, neutral.
StatusSquare	Same grammar in r8 square form for icon/counter tiles. { variant, size = 28 }.
SpecMono	IBM Plex Mono wrapper — the rule that every spec-like string (dimensions, costs, counts, IDs) renders in mono.
WorkSheet	The white r20 work sheet: optional cloud header band (eyebrow/title/subtitle/headerRight) + hairline + padded body.
InkRail + RailTitle, RailSection, RailRow	Navy reference drawer pinned right; collapses to a vertical “REFERENCE” tab, open/closed

Component	Purpose / API
	persisted in <code>localStorage["canopy.inkrail.open"]</code> (default open). <code>RailSection</code> accents by gradient stop; <code>RailRow</code> is a label/value row with mono and tone (pass/warn/attention) options and a graceful “—” fallback.
<code>StepPillNav + StepPill</code>	White pill rail on cloud: active (navy, sky step number), complete (green check), blocked (6px red dot), pending . Steps and targets supplied by the caller; <code>aria-current="step"</code> on active.
<code>ProjectBar</code>	Project chrome row: “Projects /” breadcrumb button (navigates <code>/projects</code>), name, optional mono spec pill, status text, leading and right slots.

`index.ts` re-exports all of the above. **Gaps:** `StatusChip` and `StatusSquare` duplicate their `VARIANT_CLASSES` map; `IntelligenceArchitecture` re-implements `SectionLabel` and status pills locally instead of importing; `StepPillNav` supports a **blocked** status no caller produces; several older pages (`Suite`, `AcceptInvite`, `NotFound`) still use raw `shadcn` cards, so the kit is only partially adopted.

Brand components (`src/components/canopy/`): `CanopyLogo` (PNG lockup, variant icon/horizontal/stacked kept for back-compat — all render the full lockup; `showByline` is a no-op since the wordmark is baked into the image), `CanopyAmbientGlow` (blurred gradient orb, position/size/tone/opacity/animate), `CanopyPanel` (glass surface wrapper over `.canopy-panel`, bordered/interactive/padded), `CanopyNodeField`. `Reveal` (`src/components/landing/Reveal.tsx`) wraps content in an `IntersectionObserver`-driven opacity/translate reveal (from: up/down/left/right/scale, delay, distance) built on `useScrollReveal`; `useScrollly` drives landing parallax.

5. Access-Suspension Mechanics — End to End

1. Data model

(`supabase/migrations/20260427120000_agency_access_control.sql`) agencies gains: `access_status` (active | trial | suspended | disabled, default active, CHECK-constrained, indexed), `trial_ends_at`, `suspension_reason`, `suspended_at`, `suspended_by`, `feature_flags` jsonb, `quotas` jsonb, `admin_notes`. New append-only `agency_access_log` (`agency_id`, `action`, `performed_by`, `reason`, `metadata`, `before_state`/`after_state` snapshots) with RLS: super admins `SELECT` all, agency owners `SELECT` their own, `INSERT WITH CHECK (false)` so only `SECURITY DEFINER` RPCs write.

2. Server enforcement `agency_has_access(_agency_id) → true when access_status IN ('active','trial') AND (trial_ends_at IS NULL OR trial_ends_at > now())`. It is wired into the **write** RLS policies (FOR ALL, both USING and WITH CHECK) of clients, projects, knowledge_documents, and the pending_invites agency-invite INSERT policy. Reads remain permitted so the app can render the banner/reason. `agency_effective_status()` mirrors the trial-expiry computation server-side.

3. Super-admin operations — RPCs `suspend_agency`, `reactivate_agency`, `disable_agency`, `set_agency_trial`, `update_agency_feature_flags`, `update_agency_quotas`, `update_agency_admin_notes`, `list_agencies_for_admin`, `get_agency_access_log`, `all_require_super_admin()`-guarded, GRANTED to authenticated, each snapshotting before/after into the audit log.

4. Client hooks — `useAccessControl.tsx` wraps every RPC in a `useMutation` that invalidates `["admin", "agencies"]`, `["agency"]`, and the per-agency access log. `useAdminAgencies()` lists `AgencyAdminRows` (status, effective status, owner email, member/client/project counts, last activity) with a 30s `staleTime`.

5. Admin UI — `/admin/agencies` (`AdminAgencies.tsx`): per-agency detail with **Suspend** (disabled unless `canSuspend`), **Disable** behind an `AlertDialog` confirmation, **Reactivate (set to Active)**, and a **Trial period** date input + “Set trial”, plus status filters (Trial / Suspended / Disabled). Page subtitle: “Suspend, disable, set trial windows, and feature-flag every agency on the platform.”

6. Client state derivation — `deriveAccessState()` in `useAgency`: `effectiveStatus = raw status`, flipped to `trial_expired` when `trial` and `trial_ends_at ≤ now`; `canWrite = active or unexpired trial`; `isLockedOut = disabled || trial_expired`; `isRestricted = anything ≠ active`; plus `suspensionReason`, `trialEndsAt`, `trialDaysRemaining` (ceil of days).

7. Route gate — `ProtectedRoute` step 4 redirects to `/access-suspended` when `enforceAccessGate && !isSuperAdmin && access.isLockedOut`. `/access-suspended` and `/onboarding/create-agency` opt out of the gate.

8. Persistent banner — `SuspensionBanner` (inside `AppLayout`, above `ProjectHeader`, `sticky top-0 z-40, role="alert"`): renders nothing for active agencies or trials with > 14 days left. Tones/icons/headlines by state — disabled (Lock, “Your access has been disabled.”), suspended (`AlertTriangle`, “Your access is currently suspended.”), `trial_expired` (Clock, “Your trial has ended.”), trial warning (`CheckCircle2`, “Trial: N days remaining”). Detail line = `suspensionReason` or a state-specific fallback. A “Details” link → `/access-suspended` appears for the three hard states.

9. Landing page — `/access-suspended` as described in §2, with a bounce back to `/projects` when access is no longer restricted.

Gaps in the suspension chain

- **suspended is a soft state client-side.** `isLockedOut` excludes it, so a suspended agency navigates everywhere and only hits opaque RLS errors on write; the “Details” link is the

only path to the explanation page. `AccessSuspended` itself is written to handle suspended — the gate just never sends them there.

- **canWrite is computed and never used.** No form, button, or mutation anywhere in `src/` reads it (verified by `grep`) — there is no client-side write-disabling for restricted agencies. Suspension is enforced only by RLS, so users get raw Postgres errors instead of disabled UI.
 - **RLS coverage is partial.** `agency_has_access` gates `clients`, `projects`, `knowledge_documents`, and `agency-invite` inserts only. Other agency-scoped tables (project images, prompt artifacts, pricing/BOM, brand intelligence, activation types, etc.) are not gated by this function in this migration — a suspended agency may still be able to write to them.
 - **Edge functions are not gated.** No generation/export function checks `agency_has_access`, so AI spend is not obviously blocked by suspension.
 - `feature_flags` and `quotas` are stored, admin-editable, and audited, but **no client code reads them** — no feature is actually flag-gated and no quota is enforced.
 - `SuspensionBanner` uses dark-theme token classes (`text-red-200`, `text-amber-100`) inside the light Flow C shell — likely low contrast on white.
 - Trial-warning threshold (14 days) and the banner’s own 5.5-day-style logic are hardcoded; `trialDaysRemaining` can render “0 days remaining” in the window before expiry flips.
 - Super admins bypass the gate entirely, so there is no way to see the suspended experience from a platform account (and preview mode doesn’t simulate it).
-

6. Cross-Cutting Gaps (observed, not inferred)

1. **Redirect-intent loss** — `ProtectedRoute (state.from)` and `AcceptInvite (?redirect=)` both pass a destination that `Auth.tsx` ignores; every login lands on `/projects`.
2. **Dead nav target** — `/platform-invites` in `platformOwnerNavItems` has no route.
3. **Orphan routes** — `/rhino`, `/knowledge-base`, `/team` are guarded and code-split but have no link anywhere in the app.
4. **Commented-out feature** — `/explore` (360° Explorer) route, lazy import, and step-nav entry all commented out while `Explore.tsx`, `BoothExplorer.tsx`, `PanoramaViewer.tsx` remain in the bundle-eligible tree.
5. **Preview mode advertises read-only but enforces nothing**, targets no specific agency, and is lost on refresh.
6. **No role guard in the routing layer** — five admin/account routes rely on server enforcement only; only `/admin/super-admins` self-guards.
7. **Two invite systems** (token/project vs. id/agency+super-admin) with only one landing page between them, plus a third legacy `team_members` invite path with an unimplemented accept flow.

8. **Brand inconsistency across the shell** — “BriefEngine” on /invite/:token; hello@exhibitus.com on /access-suspended; canopy.gofightwin.co on /architecture; “Canopy by Exhibitus” in index.html.
9. **RAG scope-count claim conflicts** — “4-scope” (hero + OG meta) vs “5-scope” (pillars, stats, intelligence section, /architecture).
10. **404 page is off-system** — no branding, raw anchor, no signed-in path back.
11. **Legacy/no-op API surface** — AppLayout.surface, CanopyLogo.showByline/variant, useAuth.signUp, useGrantAdminRole/useRevokeAdminRole, void cn in IndustryDetail.
12. **Project step completion is positional, not state-driven**, and StepPillNav’s blocked state is never used.
13. **Measurement-system preference is localStorage-only** with an in-code TODO to move it to a DB column.
14. **Suite budget rollout is an acknowledged stub** that sums only the parent project.

\newpage

I have everything I need. Here is the full report.

Appendix B — The Six-Step Project Flow, Files & Knowledge Base

Scope: /upload, /review, /generate, /spatial, /prompts, /export, plus /files, /knowledge-base, /rhino. All findings read from source.

Cross-cutting facts that apply to every step:

- Every step page is wrapped in AppLayout → ProjectHeader (/Users/bryanscott/Desktop/Brief to Booth/src/components/layout/ProjectHeader.tsx), which renders the project breadcrumb, a derived spec pill (parsed_brief.booth_size · budget), the units dropdown, SuiteContextBar, and StepPillNav.
 - Project identity is carried entirely in the ?project=<uuid> query param. useProjectSync (src/hooks/useProjectSync.tsx) reads it, fetches the projects row via React Query key ["project", id], and hydrates the zustand useProjectStore. Changing project id wipes the store and evicts ["project"] and ["kb-files"] caches.
 - All step-to-step navigation uses useProjectNavigate (src/hooks/useProjectNavigate.tsx), which re-appends ?project=.
 - Persistence helper: saveProjectField(projectId, column, value) — a bare projects UPDATE with no error surfacing (console.error only).
-

01 — Brief · /upload

Route & access /upload?project=<id> inside <ProtectedRoute> (auth + access gate + onboarding gate). src/pages/Upload.tsx + src/components/brief/BriefUpload.tsx. Also accepts ?suite=true to switch copy and redirect target to /suite. Reachable without a project param — BriefUpload.ensureDbProject() creates the row on first parse.

Intent Capture the source brief (uploaded file, pasted text, or a guided Q&A) and convert it into a structured ParsedBrief, while collecting the supporting project context — brand website, brand logo, inspiration imagery, and project knowledge-base documents — that every downstream generation step reads.

UI anatomy

- Worksheet eyebrow “Step 01 / 06”; headerRight StatusChip = “Brief captured” if any of brief_text | brief_file_url | parsed_brief exists, else “Awaiting brief”.
- Mode chooser (only shown when no brief content exists): two cards — *Upload a brief* → BriefUpload; *Build from scratch* → GuidedBriefBuilder. A back-link returns to the chooser.
- BriefUpload has its own internal 3-step bar: **Upload** → **AI Extraction** → **Confirm Details**.
 - *Upload*: tab toggle Upload File / Paste Text. Dropzone accepts PDF, DOCX, TXT only, maxFiles: 1, disabled outside step “upload”. TXT under 20 chars is rejected client-side.
 - *Parsing*: blocking full-panel spinner with a fake six-item checklist.
 - *Confirm*: an “AI extracted N data fields” summary grid (Brand, Objectives, Events, Footprints, Budget, Audiences, Creative Direction, Visual Identity, Deliverables, Winning Criteria); **Project Type** section (AI-inferred, collapsible ProjectTypeSelector incl. custom types); **Client** section (auto-matched pill row, “New client detected” banner with a *Capture brand intelligence* switch, or free-form new-client input); BrandGuidePrompt shown when a matched existing client has thin brand intelligence.
- Below the sheet, only in mode === "upload" with a projectId: BrandWebsiteCard, BrandLogoUpload, InspirationIntake, ProjectKnowledgeBase.
- **Blocking state / gating**: the primary “Continue to Review” CTA is lifted out of BriefUpload (via hideContinueCTA + onContinueStateChange) and rendered by Upload.tsx below the logo/inspiration/KB cards — deliberately, so users capture context before advancing. It only appears once step === "confirm", and is disabled unless selectedType is set or while saving.
- Right rail BrandProfileRail: read-only client name, primary/secondary colors, typeface + tone from useBrandGuidelines, approved/pending brand-intelligence counts, with a footer link to /clients/:id.

Inputs

- User: dropped file or pasted text; project type; client selection/creation; brand website URL; brand logo file; inspiration images and pasted URL list; KB document uploads.
- Data loaded: projects row (useProject), clients (useClients), brand_guidelines + brand_intelligence (useBrandGuidelines, useBrandIntelligence), custom_project_types (useCustomProjectTypes), knowledge_documents scope=project (useKnowledgeDocuments).
- Stores: useProjectStore (setActiveStep, loadFromDb).

Workflows

1. **Parse a brief** — startParsing():
 - ensureDbProject(name) → INSERT projects {user_id, name, status:"draft", project_type:"trade_show_booth"} if no ? project=.
 - If a real file: upload to storage bucket **briefs** at \${user.id}/\${dbProjectId}/original.\${ext} (upsert: true), then createSignedUrl with a **1-year TTL** → briefFileUrl.
 - Raw fetch (not functions.invoke) POST to \${VITE_SUPABASE_URL}/functions/v1/parse-brief with {briefText} or {fileBase64, fileType, briefText?}, 90 s AbortController timeout. Edge fn uses google/gemini-2.5-flash with a forced parse_brief tool call (PDFs escalate to gemini-2.5-pro vision, falling back to text extraction). The tool schema includes hangingElements.
 - Post-parse client-side inference: inferProjectType(parsed) (regex over brand category/objectives/show names → film_premiere / game_release_activation / live_brand_activation / permanent_installation / architectural_brief / trade_show_booth) and fuzzyMatchClient(brand.name, clients) (normalized substring match).
2. **Create client from brief** — handleConfirmNewClient() → useUpsertClient with {name, industry: brand.category, description: brand.pov, primary_color: visualIdentity.colors[0]}.
3. **Confirm & continue** — handleConfirmAndContinue():
 - UPDATE projects set brief_text, brief_file_name, brief_file_url, parsed_brief, status:"reviewed", project_type, client_id, plus brand_website_url when the brief carried a website and the project had none.
 - Primes the React Query cache for ["project", id] **before** navigation (explicit fix for a stale-hydration bug that made Review render “No brief data”), then invalidates.
 - If a client is linked and capture is enabled: extractBrandIntelligence(parsed, clientId, projectId) → useBatchCreateIntelligence → INSERT rows in brand_intelligence.
 - loadFromDb() into the store with all eight elements reset to pending; setActiveStep("review"); navigate to /review?project= (or /suite in suite mode).

4. **Guided builder** (`GuidedBriefBuilder.tsx`) — six sections (project, brand, show & venue, objectives/audience, creative direction, budget/timeline/success). Per-field ✨ suggest calls edge fn **suggest-brief-field** {fieldLabel, fieldHelp, priorAnswers}. Submit calls **synthesize-brief** {answers} → returns {parsed, briefText}; then INSERT/UPDATE projects with status: "reviewed", keyword-maps the free-text project type onto a ALL_PROJECT_TYPES id, matches-or-creates the client, captures brand intelligence, hydrates the store, navigates to /review. Section advance is gated on all non-optional fields being non-empty.
5. **Brand website** (`BrandWebsiteCard`) — persists projects.brand_website_url; when a client is linked it also invokes **deep-dive-brand**.
6. **Brand logo** (`BrandLogoUpload` → `useBrandLogo`) — uploads into the RAG KB (knowledge_documents, scope=project) with user_tags: ["brand-logo"]; consumed later as a signed-URL reference image on every render.
7. **Inspiration intake** (`InspirationIntake`) — images ≤25 MB uploaded one-by-one via `useKnowledgeDocuments.uploadDocument` with user_tags: ["inspiration", "image"]; a pasted URL list is serialized into a generated inspiration-links-<stamp>.md File and uploaded with ["inspiration", "links"].
8. **Project KB** (`ProjectKnowledgeBase` → `KnowledgeBasePanel` scope=project) — see the Knowledge Base section below.

Outputs & side effects

- Tables: projects (insert + update), clients (insert), brand_intelligence (batch insert), custom_project_types, knowledge_documents (+ document_chunks via embed-document).
- Storage: bucket briefs ({userId}/{projectId}/original.ext); bucket knowledge-documents ({agencyId}/project/{projectId}/{ts}_{name}).
- Step status: projects.status moves draft → reviewed; store activeStep = "review".

Current-state gaps

- The brief file's signed URL is baked into brief_file_url with a 1-year expiry — it will silently 404 after that, and there is no refresh path.
 - parse-brief is called with a hand-rolled fetch + anon-key header rather than supabase.functions.invoke, so it diverges from the error-unwrapping used everywhere else.
 - onDrop's useCallback deps ([user, projectId, clients]) omit the fuzzy-match inputs it closes over; a stale clients list can miss an auto-match.
 - Only PDF/DOCX/TXT are accepted; no .doc, .rtf, .pages, or image briefs.
 - Brand-intelligence capture is silently swallowed on failure (console.warn only).
 - mode initial state is computed once from hasBriefContent; if the project loads after mount the chooser can be shown for a project that already has a brief.
-

02 — Review · /review

Route & access /review?project=<id>, ProtectedRoute. src/pages/Review.tsx + src/components/brief/BriefReview.tsx.

Intent Let the user verify and correct every field the parser extracted — and close the specific data gaps the prompt composer needs — before any generation spends tokens.

UI anatomy

- Worksheet eyebrow “Step 02 / 06”; StatusChip “Brief parsed” / “Not parsed”.
- Empty state: “No brief data to review” + an *Upload a Brief* button routing to /upload.
- Top-right primary action: **Confirm & Generate Elements**.
- BriefClarificationContainer — inline gap Q&A cards (blocking gaps sorted first, cap 5 visible with a “show all” toggle, per-field “Saved ✓ — value” state with Edit).
- Six editable Section cards in a 2-col grid, each with a confidence icon (green check = high, amber “!” = medium) and a pencil → Save/X inline editor: **Brand Information, Business Objectives, Events & Shows, Footprints, Target Audiences, Budget**. Only Budget’s confidence is data-driven (medium when no range and no perShow).
- BriefExistingSpace — conditionally rendered only when the **agency’s** primary_industry maps to a builtin industry with inputMode === “existing-space-photo”.
- BriefHangingCard — always rendered; per-element name, physical form, **creative direction** (labelled as EXACT instructions), shape, suspension drop, width/depth, materials/surfaces/lighting/printed tag lists.
- **Creative Direction** card — Embrace / Avoid tag lists that commit immediately on change.
- Read-only: Required Deliverables, Timeline, Contacts (rendered only when present), and OriginalBrief (raw text + the stored file link).
- Right rail FromTheBriefRail: brand name/personality/category, primary objective, secondary count, primary footprint, formatted budget, and **Venue** rendered in the pink MISSING attention tone when absent.

Inputs

- User: inline edits to each section, tag add/remove, hanging-element authoring, existing-space photo + polygon annotations, clarification answers/skips.
- Data loaded: useProjectStore.currentProject.parsedBrief with a defensive fallback to dbProject.parsed_brief (and a hydrating effect if the store is empty); useProject; useAgency (for primary_industry → BUILTIN_INDUSTRIES inputMode).

Workflows

- **Gap detection** — validateParsedBriefForReview(parsedBrief) (src/lib/normalizedBrief.ts) synthesizes a placeholder geometry from spatial.footprints[0].size (defaults 30×30 imperial), runs normalizeBrief +

validateBrief, and returns gaps only — the geometry-dependent constraints collapse to neutral because there are no zones yet. The gap catalog: `brand.colors.hex` (helpful, fires only when the **primary** color lacks a hex), `context.venue.name` (**blocking**), `context.audience` (helpful, options), `hero.physicalForm` (helpful, requires ≥ 10 chars), `hanging.elements` (helpful, Yes/No, suppressed by `_dismissedGaps`), `hero.dimensions` (**blocking**, only when `hero_scale_ok` fails).

- **Answer/skip a gap** — `applyGapAnswer(brief, field, value, writeBack)` maps dot-paths back onto `ParsedBrief`: `venue` → `events.shows[0].location`; `audience` → `audiences[0].name`; `hero form` → `experience.hero.description` (explicitly re-pointed from `creative.designPhilosophy`, which the normalizer never read — the old infinite-save bug); hanging “Yes” seeds a rich default ring element and clears the dismissal, anything else clears `hangingElements` and pushes “`hanging.elements`” into `_dismissedGaps`; hex is encoded into the color string as “orange (#E67E22)” for the normalizer to parse back. Unknown fields log and no-op. Skip applies `gap.fallback`.
- **Section save** — `commitSection({...brief, ...draft})` → `setParsedBrief` + `saveProjectField(projectId, "parsed_brief", ...)`, clearing the draft.
- **Hanging + existing-space edits** — deliberately routed through *sibling* commit paths (`commitHangingSection`, `commitExistingSpaceSection`) that never touch draft, debounced ~400 ms, flushed on unmount, with cross-flush ordering so a “Replace photo” (immediate commit) can’t clobber still-debounced hanging edits.
- **Existing-space photo** — uploads to bucket **project-images** at `${projectId}/existing-space/${ts}.${ext}` (project id must be the first path segment for the RLS policy), commits an optimistic empty block, then async-invokes **analyze-existing-space**, merging the returned analysis over a ref-tracked latest value so in-flight user edits survive.
- **Confirm** — `handleConfirm()` sets `activeStep = "generate"` and navigates to `/generate`. **No validation gate** — blocking gaps do not prevent advancing.

Outputs & side effects

- Tables: `projects.parsed_brief` (repeatedly overwritten in full).
- Storage: `project-images` (existing-space photos).
- Step status: store only (`activeStep`). No DB column is written by this step beyond `parsed_brief`.

Current-state gaps

- Blocking gaps (`context.venue.name`, `hero.dimensions`) are labelled blocking but block nothing — the Confirm button is always enabled.
- `industryInputMode` resolves off the **agency’s primary_industry** because `projects` has no industry column; a comment flags this as temporary. Every project in an agency inherits the same input mode.
- `deriveHangingElements` synthesizes placeholder dimensions `{3,3,1}`, `suspensionDropFt: 3`, `position {0,0}` on every read and never round-trips them; the authoring card lets you edit width/depth/drop but `handleHangingChange` writes

only name/form/shape/materials/surfaces/lighting/printed/creativeDirection back — **dimension and drop edits made here are discarded.**

- Confidence badges are hardcoded "high" for five of six sections.
 - Timeline/Contacts are read-only and accessed via (brief as any) — not in the typed ParsedBrief.
 - Every section save writes the whole parsed_brief blob; concurrent editors last-write-wins.
-

03 — Concept · /generate

Route & access /generate?project=<id>, ProtectedRoute.

src/pages/Generate.tsx + src/components/elements/ElementDashboard.tsx + ElementDetailPanel.tsx + IntelligenceSelector.tsx.

Intent Generate the eight strategic concept elements from the parsed brief plus weighted context (brand intelligence, RAG, company profile, show costs, project KB), and let the user read, inline-edit, and regenerate each with feedback.

UI anatomy

- WorkSheet eyebrow “Step 03 / 06”; header right shows a pass chip when all 8 are complete, else a mono N/8 complete.
- Empty state when parsedBrief is missing: “No brief data available” + *Upload a Brief*.
- Toolbar: progress line, **Generate All with AI / Regenerate All** (variant generative), and — only when all 8 are complete — **Continue to Spatial**.
- IntelligenceSelector (shown when the client has ≥ 1 approved intelligence entry): per-entry checkboxes + select-all; only checked entries are sent.
- Linear progress bar, then a 4-column grid of 8 ElementCards. Card status badge: pending | generating | complete | error; complete cards are clickable and show a 3-line preview (getPreviewText per element type); non-complete cards show a per-card Generate/Retry button.
- Clicking a complete card swaps the whole panel for ElementDetailPanel — a Back button, a Regenerate button, a feedback textarea toggle, and per-element rich editable views (EditableText bound to dot-paths like hero.name, designPrinciples.0.description).
- Right rail WeightedContextRail: approved brand-intelligence count, past-learning count, show-cost count, and count of project KB docs with extracted text.

Inputs

- User: Generate All / per-element Generate / Retry; intelligence entry selection; regeneration feedback; inline field edits.
- Data loaded: useProjectStore.currentProject (brief, elements, clientId, hierarchy.parentId, projectType); useKnowledgeBase(projectId) (legacy knowledge_base_files, filtered to rows with extracted_text); useCompanyProfile; useShowCosts; useBrandIntelligence(clientId);

useClient(clientId);useBrandRAG({clientId, projectId, parentId, showName}) → brandContext + suiteContext strings.

Workflows

- **Auto-generate on arrival** — an effect fires once per project (hasAutoGenerated ref) when parsedBrief exists and all 8 elements are pending and no module-level job is active, with a 300 ms delay.
- **Batch generation** — runGenerationJob() creates a *module-level* activeJob that survives unmount so generation continues in the background; any prior job is marked aborted. It iterates ELEMENT_ORDER **sequentially**, setting status generating, invoking **generate-element**, then setElementData + saveProjectField(projectId, ELEMENT_DB_KEYS[type], data) immediately per element.
- **Payload to generate-element**: {elementType, briefData (=parsedBrief), existingData, feedback?, knowledgeBaseContent: [{fileName, content}], companyProfile: {companyName, industry, defaultBoothSizes, notes}, showCosts: [{showName, city, venue, boothCostPerSqft, drayagePerCwt, laborRatePerHr, electricalPerOutlet, internetCost, unionRequired}], brandIntelligence: [{category,title,content,tags}], clientData: {name, industry, description, primaryColor, secondaryColor, website}, projectType, brandContext, suiteContext}.
- **Edge behavior** (supabase/functions/generate-element/index.ts): google/gemini-2.5-pro with a forced generate_<elementType> tool call; temperature 0.4 for the “structured-heavy” pair (budgetLogic, spatialStrategy), 1.2 on regeneration, else 0.9. For spatialStrategy it injects explicit per-footprint sq-ft arithmetic and hard rules (positions are 0–100 percentages, x=0 left, y=0 aisle-front, no overlap, one config per footprint, hex color codes), then post-processes zones: 0–1 ratios are scaled ×100, width/height clamped to 5–100, x/y clamped to keep zones in bounds. Optional RAG retrieval when agency_id is present.
- **Reliability wrapper** — invokeGenerateElementWithRetry: hard timeout (180 s for spatialStrategy/budgetLogic, 90 s otherwise) plus one retry with a 2 s backoff; a 200 with an empty data.data is treated as retryable. Final failure sets status error.
- **Regenerate with feedback** — handleRegenerateFromDetail calls generateElement(type, feedback), then auto-captures the feedback as a brand_intelligence row (category: "past_learning", source: "feedback", is_approved: true, tags [elementType, "regen_feedback"], source_project_id).
- **Continue** — setActiveStep("spatial") + navigate /spatial.

Outputs & side effects

- Tables: projects columns big_idea, experience_framework, interactive_mechanics, digital_storytelling, human_connection,

adjacent_activations, spatial_strategy, budget_logic (per ELEMENT_DB_KEYS); brand_intelligence (feedback capture).

- No storage writes.
- Step status: element status is derived at hydration purely from column non-nullness (useProjectSync); "generating"/"error" are runtime-only and lost on reload.

Current-state gaps

- **Inline edits in ElementDetailPanel are never persisted.** handleUpdateField calls only setElementData (store); there is no saveProjectField. Any manual edit to an element is lost on reload.
 - A background job survives unmount but is tracked in a module-level singleton keyed only by projectId; two tabs on the same project will fight, and a page reload orphans the in-flight job (statuses stuck at generating are re-derived as pending/complete from the DB).
 - The auto-generate effect depends on currentProject?.id only, with generateAllElements referenced before definition inside the effect (works via hoisting, but the eslint deps are suppressed).
 - useKnowledgeBase here reads the **legacy** knowledge_base_files table, while the Upload step writes the **new** knowledge_documents RAG table — so documents added on the Upload page do **not** appear in knowledgeBaseContent for element generation (they reach the model only through the RAG path when agency_id is present).
 - getPreviewText uses data.x?.slice(0,120) + "... " || "" — the || binds after concatenation, so the fallback never fires.
 - Timeouts don't actually cancel the underlying fetch (documented in-code); a “timed out” element can still complete and write later.
-

04 — Spatial · /spatial

Route & access /spatial?project=<id>, ProtectedRoute. src/pages/Spatial.tsx + src/components/spatial/SpatialPlanner.tsx (+ SpatialCanvas, SpatialCanvasTopDown, SpatialCanvasIso, ZoneDetailPanel, ConstraintPanel, CostEstimator, LayoutMetrics, LayoutVariations, PromptIngredientsEditor, FlowOverlay).

Intent Turn the AI's percentage-based zone allocation into real-unit, drag-editable booth geometry — the ground truth every render is composed against — and enrich each zone with structural form, intent, materials, and sculptural features.

UI anatomy

- WorkSheet eyebrow “Step 04 / 06”; header right = active config label.
- **Blocking empty state:** if spatial_strategy.configs / the active config / the derived layout / metrics are missing, the whole planner is replaced by “No spatial data available” + *Generate Elements First* → /generate.

- Header row: mono spec line (footprintLabel · scaleDescription · N% zone coverage), an **Export SVG** button, and **Generate Prompts** (primary).
- ValidationPanel shown whenever the layout is invalid or has warnings — lists errors (destructive) and warnings (amber).
- ConfigSizeChips — booth-size selector, hidden for single-config projects.
- **Materials & Mood** card, deliberately placed above the canvas.
- Hanging toolbar: + **Hanging element** (inserts a ring at booth center sized $\frac{1}{3} \times \frac{1}{3}$ of the footprint, 3 ft drop) and a local **Show/hide hanging** visibility toggle that does not mutate persisted geometry.
- SpatialCanvas — top-down drag/resize grid plus an optional 3-D isometric preview, with a toolbar: **Suggest layout** (AI enrichment), **Auto-arrange**, **Add feature** menu, **Flow** overlay, **Heatmap** overlay, **Show AI input** (renders the exact floor-plan PNG that will be sent to the image model), **3D** toggle, **Expand** (fullscreen dialog). Per-zone selection exposes shape (rect | L | circle | diamond), height, structural form, materials, and a **per-zone prompt override** editor (customPromptOverride).
- Tab strip below the canvas: **Layout** (BriefReadinessPanel + LayoutVariations + LayoutReasoning + Zone Allocation legend), **Metrics** (LayoutMetrics), **Validate** (ConstraintPanel with ADA/sizing/sightline/utility checks and a one-click **Auto-fix**), **Costs** (CostEstimator with quality tier + budget gauge).
- Dialogs: ZoneDetailPanel, PromptIngredientsEditor, and a “Layout Changed — render this layout?” confirmation.
- Right rail ZoneProgramRail — recomputes zones and validation with the *same* pure utilities (calculateBoothDimensions, normalizeZones, validateSpatialLayout) so it always agrees with the canvas; shows per-zone sqft/%, error/warning counts, allocated %.

Inputs

- User: zone drag/resize/shape/height/form/material edits, feature add/edit, hanging-element add/move, layout-variation selection, config chip selection, constraint auto-fix, quality-tier change, per-zone prompt override.
- Data loaded: currentProject.elements.spatialStrategy.data (configs, zones, features, hangingElements, materialsAndMood, ceilingHeightFt, activeConfigKey), parsedBrief, bigIdea, interactiveMechanics.data.hero, useMeasurementSystem, useBrandIntelligence, useProjectImages, useActiveSpatialConfig.

Workflows

- **Active-config selection** — useActiveSpatialConfig (src/hooks/useActiveSpatialConfig.tsx) resolves: local click → persisted spatial_strategy.activeConfigKey → configs[0]. Selecting persists activeConfigKey = sanitizeConfigKey(footprintSize) onto the spatial_strategy blob root (no new column) and is **shared with the Prompts step**.
- **Legacy ⇌ absolute round-trip** — boothGeometryFromLegacy({...boothDimensions, measurementSystem},

normalizedZones, 12, {features, materialsCatalog, hangingElements}) produces a BoothGeometry in real units (src/lib/geometryModel.ts). Canvas edits come back through handleCanvasGeometryChange, which maps each AbsoluteZone via normalizedFromAbsoluteZone back to the legacy percentage shape, spreads the original zone to preserve untracked fields (notes, requirements, adjacencies), and re-attaches canvas-owned fields (customPromptOverride, heightFt, shape, shapeParams, structuralForm, featureDescription, intent, materialIds).

- **Persistence** — one write: setElementData("spatialStrategy", updatedSpatial) (optimistic) + saveProjectField(projectId, "spatial_strategy", updatedSpatial). configs[activeIndex].zones gets the zones; ceilingHeightFt, features[], and hangingElements[] live at the **root** of spatial_strategy so they survive footprint switching.
- **Layout variations** — generateLayoutVariations(...) produces alternates, each pre-run through fixNormalizedLayoutAutomatically so they satisfy size/percentage minimums before the user clicks. handleVariationSelect is idempotent (clicking the active variation is a no-op, preventing compounding multiplicative transforms), treats "balanced" as identity, and merges only position/percentage/sqft from the variant while keeping all zone metadata.
- **AI enrichment (“Suggest layout”)** — handleEnrichSpatial() invokes **enrich-spatial** with {parsedBrief, bigIdea, heroInstallation, spatialStrategy, boothDimensions}. The edge fn (gemini-2.5-pro, forced spatial_enrichment tool) returns {zones:[{id, structuralForm, featureDescription, intent, materialIds}], features:[...]}. The client merges zone metadata by id (preserving user shape/height edits), **replaces features wholesale**, then runs fixLayoutAutomatically and reports whether positions actually changed. Writes through the same handleCanvasGeometryChange path.
- **2-D floor-plan render** — handleGenerateFloorPlan builds a long text prompt (scale context, variation strategy, per-zone position descriptions, brand-color/material/style/layout blocks from PromptIngredientsEditor, annotation feedback, regeneration feedback) and invokes **generate-view** with {viewPrompt, viewName:"Floor Plan 2D", aspectRatio, angleId:"floor_plan_2d", projectId, boothSize}, saving via useSaveRenderImage.
- **Per-zone prompt preview** — getZoneDefaultPrompt(zoneId) calls generatePrompt("zone_interior_<id>", promptParams) so the Spatial step shows the same base prompt the Prompts step will use; edits save as customPromptOverride and **both** steps respect them.
- **Continue** — setActiveStep("prompts") + navigate("/prompts").

Outputs & side effects

- Tables: projects.spatial_strategy (whole blob); project_images (only via the floor-plan path, angle_id: "floor_plan_2d").
- Storage: project-images (floor-plan render, via save-render-image).
- Step status: none written; progression is store-only.

Current-state gaps

- **Export SVG has no onClick** — a dead button.
 - **enrich-spatial always reads spatialStrategy.configs[0]** (hard-coded server-side), while the client sends boothDimensions for the **active** config. On a multi-config project with a non-first config selected, the AI enriches the wrong zone set and the merge is by id, so mismatched ids silently produce a no-op.
 - The floor-plan render UI was removed but its state (floorPlanView, floorPlanImage, floorPlanAnnotations, PromptIngredientsEditor, the layout-change confirmation dialog, handleGenerateFloorPlan) is all still mounted and reachable only through the pending-variation dialog — a half-wired feature.
 - useMemo is used as a side-effecting hook in two places (hydrating floorPlanImage and floorPlanAnnotations) — calls setState during render.
 - Hanging elements added here land on spatial_strategy.hangingElements and are drawn on the canvas, but **never reach the render prompt** — the composer reads only parsed_brief.hangingElements. This is explicitly flagged as a KNOWN GAP in PromptGenerator.tsx (lines ~369–388).
 - Layout validation errors never block navigation to Prompts.
-

05 — Render · /prompts

Route & access /prompts?project=<id>, ProtectedRoute. src/pages/Prompts.tsx + src/components/prompts/PromptGenerator.tsx (2,889 lines) + HangingElementCheck, PromptInspector, PromptVersionTabs, PreflightChecklist, PromptDebugPanel, ModelBadge, AttachReference, BriefClarification, RenderPromptDialog, ConfigSizeChips.

Intent Compose the exact prompt from the normalized brief + canvas geometry, render a hero 3/4 view first, let the user iterate on it conversationally (and approve its hanging element), then fan out to every other view and zone interior — with the exact prompt for every image preserved and reviewable.

UI anatomy — three phases driven by renderStore.phase.

Persistent header (versionsHeader, all phases): ConfigSizeChips (disabled during any generation), the “Generate all sizes” confirm dialog, the per-config batch-progress dialog, PromptVersionTabs (select/create/rename/delete versions), and an amber **orphaned render sets** recovery banner.

Phase prompt / hero-generation:

- Booth info card (footprint label, area in native units, zone count).
- PreflightChecklist — collapsible, six rows with ok/warn/info status and deep-link edit buttons: Brand identity (name+category+colors, colors falling back to the client brand book, deep-linking to /clients/:id?tab=brand), Brief & objectives, Dimensions & units, Spatial strategy, Brand logo, Visual references.
- AttachReference for hero_34.

- BriefClarification (safety-net mount of the same gap cards as Review).
- **Two-step generation:** Generate Hero Prompt composes the text without spending a render; then PromptInspector (review / copy / edit / reset, with an “Edited” badge) + Generate Hero Image + a Re-compose link.
- Generate all sizes (N) when >1 config; PromptDebugPanel showing the composer’s five output stages.

Phase hero-review:

- Hero image (click-to-lightbox), ModelBadge (Canopy 2.0 / Canopy Lite with the gpt-image-2 failure reason on hover), PromptInspector for the prompt that produced it.
- **Conversation thread** — every render appends a turn; each turn’s thumbnail is clickable to set that image as current (branching).
- Chat-style refinement textarea (⌘/Ctrl+Enter to send), **Send refinement**, a from-scratch refresh button, **Download**, and **Approve & Generate All Views**.
- HangingElementCheck (only when the brief carries hanging elements) — canonical spec lines per element, an editable “Creative direction (treated as EXACT instructions)” field committed on blur, an amber “not approved” warning, **Looks right — approve**, and **Refine hanging element** with quick-chips (Smaller / Larger / Hang higher / Hang lower / Material / Lighting / Remove printing) + free text.

Phase all-views:

- BriefReadinessPanel (score + top-3 gaps, each routing to the owning step), header actions (Generate all sizes, Regenerate All, **All Images (N)** gallery dialog with per-size filter chips and per-render Expand/Download/**Prompt**, Export Package).
- Progress card during generation; a “Style Reference (3/4 Hero View)” card with booth specs + zone allocation.
- **Standard Views** grid (top, front, left, right, back, detail_hero, detail_lounge) and **Zone Interior Views** grid — each card has the image or a pending/generating/error/never-generated state, ModelBadge, PromptInspector, Download, Regenerate, and AttachReference.

Right rail PreFlightRail: geometry reference captured?, brand logo present (pink MISSING if not), active config + footprint sqft, and hanging-element approval state (Approved / Awaiting check / Pending hero / None).

Inputs

- User: prompt edits, hero feedback, per-view reference attachments, version selection/creation, config chip selection, hanging approve/refine/creative-direction, clarification answers.
- Data loaded: store currentProject (brief, spatialStrategy, bigIdea, all elements); useRenderStore; useProjectImages + useSaveRenderImage; usePromptVersions; useActiveSpatialConfig; useBrandIntelligence + useClient; useBrandRAG; useMeasurementSystem; useBrandLogo; useAgencyImageModel + useAgency; useRenderReferences;

useProjectVisualReferences; useGeometryReferences;
useKnowledgeDocuments.

Workflows

Prompt composition (client-owned, src/lib/normalizedBrief.ts):

1. `normalizeBrief({project:{id,name,projectType,industrySlug}, parsedBrief, geometry, elements})` → canonical `NormalizedBrief`. `safeBrief()` back-fills every required field. Colors are parsed out of "name (#HEX)" strings and role-tagged primary/secondary/accent. Zones map to functional purposes via `zoneNameToPurpose` keyword matching. Hero `physicalForm` precedence: `interactiveMechanics.data.hero.physicalForm.structure` → `parsedBrief.experience.hero.description`. Signage is derived (wordmark from brand name; descriptor from tagline). Budget tier + \$/sqft is inferred. `existingSpace` is left **undefined** when absent — its presence is the discriminator for the interior-design path. `hangingElements` normalize from `parsedBrief.hangingElements` only.
2. `validateBrief(normalized)` → {failures, gaps}. Hard constraints: `footprint_match`, `open_sides_clear`, `signage_present`, `descriptor_present`, `hero_scale_ok` (≤30 % of footprint), `forbidden_items_absent`, plus normalize-time `hanging_elements_aloft` (status unknown — no automated verifier exists).
3. `composePrompt(normalized)` → {`briefJson`, `geometrySummary`, `renderer`, `negative`, `compliance`}. It dispatches on the industry `inputMode`: `existing-space-photo` (or hybrid with a photo) → `composeExistingSpacePrompt`, else `rendererPrompt`. `_dismissedGaps` is stripped before emitting `briefJson`.
 - **rendererPrompt sections, in order:** # SCENE · # SPACE (rectangular carpet allocation, max height, open sides, human scale, min circulation, plus an explicit “design organically *above the floor*, the carpet stays rectangular” instruction) · # HANGING ELEMENTS (only when present; per-element form, geometry with the position phrase and drop, materials/surfaces/lighting/printed, and Creative direction (EXACT – follow precisely) treated as a lock, not inspiration) · # STRUCTURAL APPROACH · # ZONE PROGRAM (deduped functional purposes only — the old explicit coordinate grid was deliberately removed because it produced formulaic rectangular pavilions) · # BRAND · # BUDGET REALITY · # CONTEXT · # ENVIRONMENT (always emitted) · # DESIGN INTENT · # HARD CONSTRAINTS · # NEGATIVE (appended inline because gpt-image-2 has no separate negative input).
 - **composeExistingSpacePrompt sections:** # SCENE · # EXISTING SPACE · # REGIONS TO PRESERVE · # REGIONS TO REDESIGN · # REDESIGN INTENT · # HARD CONSTRAINTS · # NEGATIVE. # SPACE and # ZONE PROGRAM are deliberately omitted.
 - Negative list: forbidden items + no overlaid annotations + no zone/room labels on fascia + no dimension callouts + no flat rectangular fascia/generic truss + no

cartoon/over-saturation/AI artifacts + the void-ban (blank background / white void / studio backdrop / isolated product shot / floating booth with no floor).

4. `composeViewPrompt(heroSnapshot, angle, {zoneId})` — deliberately **terse** (~80–150 tokens): one camera instruction (`front | side_left | side_right | back | top | interior | detail`), a consistency guard (“treat the reference image as canonical... only the camera angle changes”), and a restriction line. The rationale is documented: long structured blocks made gpt-image-2 treat the call as a fresh generation instead of an edit.

Hero generation — `handleGenerateHeroImage()`:

- Merges hero attachments + project-wide visual references (deduped).
- `captureGeometryRefs()` → `renderFloorPlanForExport(geometry)` rasterized to a data URL, uploaded via `useGeometryReferences` into `knowledge_documents` tagged `geometry-reference`, resolved to a signed URL; cached by geometry hash. The isometric capture is now explicitly `null`.
- `buildExistingSpaceParams()` → for interior-design projects, `rasterizePolygonMask(photoUrl, annotations.change)` (failure is non-fatal — renders without the mask).
- `renderStore.generateHeroImage({composedPrompt: {renderer,negative,artifacts}, prompt, feedback, previousImageUrl, projectId, boothSize, boothDimensions, geometryReferences, projectType, brandIntelligence, brandContext, suiteContext, brandLogoUrl, imageModel, extraReferenceUrls, existingSpacePhotoUrl, maskDataUrl, onSave})` → invokes **generate-hero** with `project_id` (which gates the server-side storage upload; without it the function returns multi-MB base64 and the save silently drops).
- **Edge branch priority** (`supabase/functions/generate-hero/index.ts`): (1) `composedPrompt.renderer` used **verbatim**; (2) EDIT MODE (`previousImageUrl` + `feedback`, no `composedPrompt`) → the “IMAGE EDIT TASK — NOT A REGENERATION” template; (3) legacy edge-side structured builder. Reference images: existing-space photo is exclusive; otherwise [`previousImageUrl?`, `brandLogoUrl?`, ...`extras`] with SVGs stripped (they 400 the image models). Floor-plan/iso PNGs are deliberately **not** attached (label-bleed bug). Data-URL results are uploaded server-side to bucket `project-images` and returned as a short URL. Response echoes `promptUsed`, `modelUsed`, and `primaryError`.
- On success the store appends a `HeroTurn`, sets phase: “hero-review”, builds `promptArtifacts` via `buildRenderPromptArtifacts` (prompt capped at 20 KB, `data`: URLs never persisted, references deduped and labelled), and calls `onSave("hero_34", "3/4 Hero View", url, meta)`.

Hanging-element hero-first approval flow:

- Gate key = `hangingApprovalKey(activeConfigKey, heroImage)` — scoped to *this hero image within this config*, so any refine/regenerate produces a new URL and approval automatically resets; switching size chips never bleeds approval.

- **Approve** → `renderStore.setHangingApproval(key, true)` + a direct `project_images UPDATE` writing `prompt_artifacts.hangingApproved = true` on the row whose `public_url` matches the hero; re-hydrated on reload by an effect that scans `savedImages`.
- **Refine** → `buildHangingEditInstruction(elements, feedback, units)` (`src/lib/hangingRefinement.ts`) produces an instruction that locks `booth/floor/furnishings/people/environment/lighting/camera`, restates the canonical spec, appends the refinement request, and reasserts suspension + the EXACT creative-direction contract. It is sent through `generateHeroImage` with `previousImageUrl` + `feedback` and **deliberately no composedPrompt** so the edge function takes the EDIT-MODE branch. `threadMessage` carries the user's own words so the conversation stays readable. Result saves as a new hero version in the same config stack.
- **Creative direction edit** → writes `parsed_brief.hangingElements[i].creativeDirection` via `setParsedBrief` + `saveProjectField`.
- **Gate is advisory**: `handleGenerateAllViews` fires a toast when unapproved but does not block.

All-views generation — `renderStore.generateAllViews`: splits exteriors then interiors (interiors run only after all exteriors finish so they can anchor to a finished exterior — preference order front, left, right, hero_34), runs `Promise.allSettled` in batches of 3 (documented as the ceiling before gpt-image-2 rate-limit symptoms), invokes **generate-view** per angle with `{project_id, referenceImageUrl, viewPrompt, viewName, aspectRatio, boothSize, boothDimensions, geometryReferences, brandIntelligence, brandContext, suiteContext, brandLogoUrl, extraReferenceUrls, imageModel, heroPromptText, existingSpacePhotoUrl, maskDataUrl, consistencyTokens?, designContext?, composedPrompt?,}` with one retry on transient `BOOT_ERROR | WORKER_RESOURCE_LIMIT | 503` after 1.5–3 s jitter. One failed view does not abort the batch.

Per-config versioning scheme (`src/lib/promptVersions.ts`):

- Angle id format: `hero_34` (legacy) → `hero_34__v__<versionId>` → `hero_34__v__<versionId>__cfg__<configKey>`. The config suffix is outermost and stripped first by `parseVersionedAngleId`.
- `sanitizeConfigKey(label)` lowercases and slugs to `[a-z0-9.]`, defaulting to `"size"`.
- Version metadata (`PromptVersionMeta`: `id`, `preset`, `label`, `customEmphasis`, `createdAt`, `updatedAt`, `notes`, `claimsUnversioned`, `imageModel`) persists to `projects.prompt_versions` (JSONB) with a **localStorage mirror** (`canopy:prompt-versions:<projectId>`), because that column may not exist — `isMissingColumnError` swallows the failure. It is not in the generated Supabase types.
- Hydration filters saved images **config-first, then version**: an image belongs to the active size when its config key matches, or when it's untagged and the active size is `configs[0]`. If the active version has no images, it falls back to all images *for this booth size* — never across sizes.

- `findOrphanedVersions` detects version suffixes with no metadata (plus a synthetic `LEGACY_VERSION_ID` bucket for unsuffixed renders) and drives the one-click recovery banner.
- Save handler is a **factory** (`makeSaveHandler(configKey, configLabel)`) so the “Generate all sizes” batch binds each render to the config it was generated for, regardless of which chip is active when the render lands.

“Generate all sizes” — `runBatchAllSizes()`: strictly sequential, one hero per config. For each config it re-derives dimensions → zones → geometry → a fresh `normalizeBrief/composePrompt` → the config’s own floor-plan reference → its own save handler. Afterwards it resets the render store so the page re-hydrates the active size.

Per-image prompt transparency — every save carries `promptArtifacts` (exact prompt, negative, geometry summary, compliance list, labelled reference URLs, model, timestamp). `RenderPromptDialog` (`src/components/common/RenderPromptDialog.tsx`) renders these in copyable sections (Prompt / Negative prompt / Geometry / Reference images / Hard constraints + meta chips + Copy all), with an explicit “Prompt not recorded (generated before prompt tracking)” state for legacy rows. Mounted from both the Prompts gallery and the Files lightbox.

Outputs & side effects

- Tables: `project_images` (one row per render, previous rows for the same `angle_id` flipped `is_current:false` by `save-render-image`); `projects.prompt_versions`; `projects.parsed_brief` (creative-direction edits + clarification answers); `project_images.prompt_artifacts` (hangingApproved patch); `knowledge_documents` (geometry references, render references).
- Storage: `project-images` at `{projectId}/{configKey}_{angleId}_{ts}.{ext}`; `knowledge-documents` for geometry/render references.
- Step status: none written to `projects`. phase and all render state live in `useRenderStore` (memory-only, reset on project switch, version switch, and config switch).

Current-state gaps

- **Documented KNOWN GAP** (PromptGenerator ~L369): hanging elements authored on the spatial canvas (`spatial_strategy.hangingElements`) never reach the prompt — only `parsed_brief.hangingElements` does. The two stores are never merged.
- **Style presets are effectively dead on the composed path.** `applyStylePresetToPrompt` is only applied to `buildPrompt(...)` output, but `handleComposeHeroPrompt` prefers `composerOutput.renderer`, and both edge functions prefer `composedPrompt.renderer` verbatim. So a version’s chosen preset changes nothing for hero or view renders whenever composition succeeds.
- `PreflightChecklist` reads `spatialData?.configs?.[0]?.zones` — the **first** config, not the active one. On multi-config projects it can report the wrong zone count.

- `projectIndustrySlug` falls back to the **agency's** `primary_industry` (same limitation as Review) — every project in an agency shares an input mode, which decides whether the existing-space renderer path fires at all.
 - `renderStore` is a whole-store subscription (`const renderStore = useRenderStore()`), forcing several `eslint-disable` deps to avoid React #185 update loops — fragile.
 - `designContext` is built and pushed to the store on every change and sent to both edge functions, but the composed-prompt branch ignores it — redundant payload.
 - `heroThread` / `heroIterations` are memory-only; the conversation history is lost on reload (only the images survive, re-seeded as thread-less iterations).
 - Hero prompt overrides (`renderStore.heroPrompt`) are not persisted; navigating away drops the edit.
 - Interior views pick their reference from `get().generatedImages` mid-batch — with `BATCH_SIZE = 3` the chosen exterior depends on completion timing.
 - The hanging gate warns but never blocks; `hanging_elements_aloft` compliance status is permanently "unknown" (no post-render CV check exists).
 - `save-render-image` carries a PGRST204 fallback that drops `prompt_artifacts` entirely if the column is missing — prompt transparency silently degrades in drifted environments.
-

06 — Export · /export

Route & access `/export?project=<id>`, `ProtectedRoute`. `src/pages/Export.tsx` + `src/components/export/ExportPackage.tsx` (+ `DesignedDeck`, `ProposalExport`, `DeckEditor`, `DeckSlideEditor`, `DeckPreview`, `DeckPreflightChecklist`, `FigmaExportPanel`, `SaveLearningsButton`).

Intent Package the brief, the eight strategy elements, and every current render into client-facing deliverables — an AI-designed deck, a classic PPTX/PDF proposal, a Figma spec, a materials/cost list, a 3-D modeling brief, and a full ZIP — and feed the project's learnings back into client intelligence.

UI anatomy

- `WorkSheet` eyebrow "Step 06 / 06"; header right = mono render count.
- Blocking empty state when `parsedBrief` or `elements` is missing: "No project data to export" + *Start a New Project* → `/upload`.
- `DesignedDeck` — the primary path (Claude designs each slide as HTML+CSS; slide list, preview, per-slide regenerate, inline HTML editing, Export PDF / Export PPTX via offscreen `html2canvas`, plus a `ping` diagnostic that probes the Anthropic key and reports `valid` | `invalid` | `configured` | `missing`).
- `ProposalExport` — the classic `pptxgenjs`/PDF path with an inline `DeckEditor` and section toggles.
- **Download All Assets** ZIP card.
- **Package Summary** (project name, N/8 elements complete, current render count).

- **Materials & Cost Estimate** — Generate/Regenerate + CSV download; renders categories, per-item qty/unit/total, grand total, notes.
- **3D Modeling Brief & Meshy.ai Prompts** — Generate/Regenerate + .md download; renders Meshy prompts (with style tokens and material hints) and the modeling brief (dimensions, scale, layers, materials, construction notes).
- **FigmaExportPanel** — generates a Figma-importable JSON spec (frames, text nodes, image refs, color tokens, typography scale) and downloads it.
- **3D Design Renders** card (polished Rhino renders, click-to-lightbox) — shown only when at least one has `polish_status === "complete"`.
- **SaveLearningsButton**.
- Right rail **PackageRail**: agency logo on file (link to /company if MISSING), client logo (link to /clients/:id), concept element completeness, render count.

Inputs

- User: generate/regenerate deck, per-slide regenerate, slide HTML edits, section toggles, export format choice, materials/3D/Figma generation, learning selection.
- Data loaded: store `currentProject` (brief + elements + clientId + renderPrompts), `useProjectImages`, `useRhinoRenders`, `useBrandIntelligence` (approved only), `useCompanyProfile`, `useAgency`, `useDesignedDeck`.

Workflows

- **Designed deck** — `useDesignedDeck.generate/regenerateSlides` invoke **generate-presentation** with `{mode: "designed-deck", parsedBrief, elements, projectName, imageUrls, brandColor, secondaryColor, agencyName, stylePreset, deckOverrides, agency_id, project_id}` (+ `regenerateSlideIds` and `existingSlides` on regeneration, spliced back by slide id). Errors are unwrapped via `unwrapInvokeError` to surface the real cause (e.g. “invalid x-api-key”).
- **Materials** — **generate-materials** with `{parsedBrief, spatialStrategy, budgetLogic, boothSize: footprints[0].size ?? "30x30", agency_id, client_id, activation_type_id, project_id}` → `data.materials` held in component state; CSV built client-side.
- **3D brief** — **generate-3d-brief** with `{parsedBrief, spatialStrategy, renderPrompts, imageUrls: current renders, boothSize, agency_id, client_id, activation_type_id, project_id}` → `data.brief`; markdown built client-side.
- **ZIP** — `dynamic import("jszip")`; fetches every `is_current` render into `renders/`, every polished Rhino pair into `rhino-renders/`, writes `content.md` (brand overview, objectives, audiences, events, spatial, budget, all eight elements as fenced JSON, render manifest) and `content.json` (structured mirror), then triggers a blob download named `{brand}_Export_Package.zip`.
- **Learnings** — `SaveLearningsButton` builds element summaries + a brief summary and invokes **extract-learnings** `{clientName, projectName, projectType,`

boothSize, briefSummary, elements, feedbackLog}; the returned entries are shown with all selected by default and saved into brand_intelligence.

Outputs & side effects

- Files created locally: .zip, .pptx, .pdf, _materials.csv, _3d_brief.md, Figma.json.
- Tables: brand_intelligence (learnings). Deck state is persisted **only to localStorage** (useDesignedDeck LS_PREFIX cache keyed by projectId).
- Step status: **nothing** is written — projects.status never becomes "completed".

Current-state gaps

- **No completion write.** Projects.tsx pipeline step “Exported” checks p.status === "completed", but no code path ever sets that value. The step can never turn green.
 - The designed deck lives only in localStorage — not shared across devices or teammates, and lost on cache clear.
 - Materials, 3-D brief, and Figma spec are held in React state only; navigating away discards them with no persistence and no regeneration memory.
 - Every export uses footprints[0].size / configs[0] — the active footprint config chosen on Spatial/Prompts is ignored, so a multi-size project always exports the first size’s spec.
 - generate-3d-brief is sent currentProject.renderPrompts, which is always null (see below).
 - ZIP image fetches are sequential and failures are only console.warned — a partially-empty package downloads silently.
 - FigmaExportPanel receives clientLogo: null hardcoded despite the rail checking for a client logo.
 - activationTypeId is read via two speculative (currentProject as any) keys that the Project type does not define — effectively always null.
-

Files · /files

Route & access /files?project=<id>, ProtectedRoute. src/pages/Files.tsx. Reachable from StepPillNav (rendered as pill #6, between Prompts and Export).

Intent The project asset library: every saved render (all versions, all booth sizes), 3-D Rhino uploads with AI polish, and video generation from selected images.

UI anatomy

- PageHeader with an asset count eyebrow (no step eyebrow). Empty state when no ? project=.
- Three tabs: **Images** (count badge), **3D Renders** (count badge), **Video**.
- **Images**: angle filter chip row (All angles + one chip per distinct angle_name, each with counts), a **Current only** toggle, and a **Select** toggle. Select mode adds a bar with select-

all/deselect-all, a selection count, and **Make Video (N)** which programmatically clicks the video tab.

- Grid cards show the render, a hover gradient with angle name + date, a **Current** badge, and a booth-size badge (replaced by a checkbox in select mode). When renders span more than one booth size, the grid is split into per-size sections (project-config order first, then orphaned config keys, then “Earlier renders” for untagged/legacy) — otherwise a flat grid.
- Custom fullscreen lightbox with prev/next walking the flattened display order, angle name, Current badge, timestamp, **Download**, and **View prompt** (→ shared `RenderPromptDialog`).
- *3D Renders*: `RhinoUploadPanel` + `RhinoGallery`.
- *Video*: `FilesVideoPanel`, pre-seeded with the images selected on the Images tab.

Inputs `useProjectImages(projectId)` (`project_images`), `useRhinoRenders(projectId)` (`rhino_renders`), `useProjectStore` for `spatialStrategy.data.configs` (to map config keys → human labels) and `clientId`. Size tagging reads `parseVersionedAngleId(angle_id).configKey` first, then `prompt_artifacts.configKey/configLabel`.

Workflows

- Filter (angle + current-only) → group by booth size → flatten for lightbox navigation.
- Download builds an `<a download>` with `{angle_name}_{created_at_ms}.png`.
- View prompt opens `RenderPromptDialog` against the row’s `prompt_artifacts`.
- Rhino polish (`useRhinoRenders`) sets `polish_status`: “processing”, invokes **polish-rhino-render**, and writes “error” + `polish_feedback` on failure.
- Video generation runs through `useVideoStore` → **generate-video**.

Outputs & side effects Read-mostly. Writes come from the Rhino panel (`rhino_renders` + storage) and the video panel. No projects writes, no step status.

Current-state gaps

- `/files` occupies step slot 6 in `StepPillNav` while Export is slot 7, contradicting every `WorkSheet` eyebrow (“Step 06 / 06” on Export). The step model in the header and the step model on the sheets disagree.
- The download always names the file `.png` regardless of the actual stored extension.
- The “Make Video” handoff uses `document.querySelector('[data-value="video"]')?.click()` — a DOM escape hatch that will break if the Tabs implementation changes its attributes.
- Angle chips are computed from all saved images while the size grouping applies only to filtered, so a chip can show a count that no visible group contains.
- Files has its own bespoke lightbox rather than the shared `ImageLightbox` used elsewhere.
- No delete, rename, tag, or “set as current” action on renders — the library is read-only aside from Rhino/video.

Knowledge Base · /knowledge-base (project-scoped)

Route & access /knowledge-base?project=<id>, ProtectedRoute.
src/pages/KnowledgeBase.tsx. **No navigation link anywhere** — not in AppSidebar, not in StepPillNav. Reachable only by typing the URL.

Intent Attach project reference material (previous projects, inspiration, pricing docs, specs) with editable extracted text that gets injected into element generation.

UI anatomy PageHeader with a document count; empty state when no ?project=. Dropzone accepting **any** file type up to 20 MB. Per-file card: type icon, filename, size, date, a “Has content” badge, an expand chevron revealing a “Content / Notes (included in AI generation)” textarea + **Save Notes**, and a delete AlertDialog.

Inputs / Workflows useKnowledgeBase(projectId)
(src/hooks/useKnowledgeBase.tsx) against the **legacy** knowledge_base_files table.
Upload: extractTextFromFile(file) (client-side text read for txt/csv/md/html/json/xml/yaml/log only; everything else returns null) → supabase.storage.from("knowledge-base").upload("{userId}/{projectId}/{ts}_{name}") → getPublicUrl → INSERT knowledge_base_files {project_id, user_id, file_name, file_type, storage_path, public_url, extracted_text, file_size_bytes}. Delete removes the storage object then the row. updateExtractedText patches the row.

Outputs & side effects Table knowledge_base_files; storage bucket knowledge-base. Consumed by ElementDashboard.getContextPayloads() as knowledgeBaseContent: [{fileName, content}] (rows without extracted_text are dropped) and by Generate.tsx’s Weighted Context rail.

Current-state gaps

- **Two parallel knowledge bases exist.** This page writes knowledge_base_files / bucket knowledge-base; the Upload step’s ProjectKnowledgeBase → KnowledgeBasePanel writes knowledge_documents / bucket knowledge-documents with embeddings + auto-tagging. They do not see each other. migrate-legacy-kb exists as an edge function but nothing in this UI calls it.
- The page is **orphaned** — no link reaches it.
- Non-text files land with extracted_text: null and are therefore invisible to element generation until a human manually pastes the content.
- Files are stored with getPublicUrl on the knowledge-base bucket (public reads) while the newer knowledge-documents bucket is private and signed — inconsistent access posture.

The RAG-backed panel (src/components/knowledge/KnowledgeBasePanel.tsx, used at project scope from /upload): dropzone restricted to PDF/DOC/DOCX/XLS/XLSX/PPT/PPTX/TXT/MD/CSV/PNG/JPG/WEBP up to 50 MB; per-doc rows with embedding status, auto-tags + user tags, pin/unpin, re-embed, preview (signed

URL, 1 h), download, delete; search by filename/title/tag once >3 docs. Upload path: storage {agencyId}/{scope}/{scopeId}/{ts}_{safeName} → INSERT knowledge_documents with status: "pending" → invoke **embed-document** → **auto-tag-document** on a 3 s setTimeout. Tag conventions drive downstream consumers: brand-logo, inspiration+image, inspiration+links, render-reference, geometry-reference.

Gap: the auto-tag call is a fire-and-forget setTimeout racing text extraction; embed failure is swallowed with only a console error and the row left at pending/failed.

Rhino · /rhino

Route & access /rhino?project=<id>, ProtectedRoute. src/pages/Rhino.tsx. **No navigation link anywhere** — orphaned like /knowledge-base.

Intent / anatomy Upload Rhino/SketchUp/3-D screenshots and have AI polish them into photorealistic renders. The page is a thin shell: header, RhinoUploadPanel, RhinoGallery — the exact same two components the Files page mounts in its “3D Renders” tab.

Workflows / outputs useRhinoRenders(projectId) → table rhino_renders. Polish sets polish_status: "processing" + polish_prompt, invokes **polish-rhino-render**, and writes polish_status: "error" + polish_feedback on failure or when no image is returned. Polished renders (polish_status === "complete" with polished_public_url) flow into the Export step’s ZIP, proposal, and Figma spec.

Current-state gaps Fully duplicated by the Files → 3D Renders tab, unreachable via UI, and unlike Files it lacks the video tab and the surrounding library context. It is dead surface area kept alive by the router.

Step gating & status model

There are four independent, partly-contradictory notions of “where am I”:

1. Header pill nav — purely positional, not data-driven. ProjectHeader.tsx builds stepPills from a hardcoded PROJECT_STEPS array of **seven** entries (Brief, Review, Generate, Spatial, Prompts, **Files**, Export). Status is computed as: location.pathname === step.path ? "active" : idx < currentStepIndex ? "complete" : "pending". It never reads project data. Consequences: everything left of the current URL renders with a green check regardless of whether it was done; deep-linking to /export marks all six prior steps complete; StepPillNav’s “blocked” status (the red 6 px dot) is **never produced by any caller** — the visual state exists but is dead code. The sheets say “Step NN / 06” while the nav shows seven pills, so Files is #6 in the nav and Export is #7 but titled “Step 06 / 06”.

2. useProjectStore.activeStep — advisory, never read for gating. A union of "upload" | "review" | "generate" | "spatial" | "prompts" | "export". Set on hydration by loadFromDb using a cascading heuristic (parsedBrief → review; any

element complete → generate; renderPrompts → prompts) and imperatively by each step's Continue handler. Nothing reads it to decide what to render or whether a route is permitted.

3. Per-screen data guards — the *only* real gating, and each one is a soft redirect.

- /review: !brief → “No brief data to review” + button to /upload.
- /generate: !currentProject?.parsedBrief → “No brief data available” + button to /upload.
- /spatial: missing spatial_strategy.configs / active config / layout / metrics → “No spatial data available” + button to /generate.
- /prompts: !brief || !spatialData || !bigIdea → “Generate all elements first” + button to /generate.
- /export: !brief || !elements → “No project data to export” + button to /upload. All are advisory dead-ends, not redirects; the user can always navigate directly via the pill rail and simply lands on the empty state. Nothing prevents forward navigation from a step with unresolved blocking gaps.

4. Persisted status — thin and incomplete.

- projects.status is a free-text string. Only two values are ever written: "draft" on project creation (BriefUpload.ensureDbProject, useProjects) and "reviewed" on brief confirmation (BriefUpload.handleConfirmAndContinue, GuidedBriefBuilder.handleSubmit). "parsing", "generating", and "completed" are read (by Projects.handleOpenProject's route map and the pipeline checks) but **never written**.
- The real progress signal is **column presence**, evaluated by PIPELINE_STEPS in src/pages/Projects.tsx: brief (brief_text | brief_file_url | brief_file_name), review (parsed_brief.brand.name **and** an objective or a show), elements (big_idea **and** one of experience/interactive/digital), spatial (spatial_strategy.configs.length | zones.length | scalingStrategy), prompts (render_prompts non-null), export (status === "completed").
- **Two of those six can never be satisfied.** setRenderPrompts exists on the store but is called from nowhere, and projects.render_prompts is only ever *read* — so “Render Prompts Ready” is permanently false even after a full render sweep. And nothing writes status: "completed", so “Exported” is permanently false. The hero_prompt and hero_style_confirmed columns are likewise read but never written.
- Element status is not stored as status at all — useProjectSync derives "complete" vs "pending" purely from whether the element's JSON column is non-null. Runtime "generating" and "error" states are memory-only and vanish on reload.
- Render-phase state (renderStore.phase, hero thread, prompts, progress) is entirely in memory, reset on project switch, prompt-version switch, and booth-size switch; the only durable render state is the project_images rows, re-hydrated by scanning for a current hero_34 under the active version+config.

- The one genuine per-step approval gate in the product is the **hanging-element check**, keyed `hangingApprovalKey(configKey, heroImageUrl)`, held in `renderStore.hangingApprovals` and persisted to `project_images.prompt_artifacts.hangingApproved` — and even that only warns rather than blocking view generation.
- Two per-project selections persist outside the schema: `spatial_strategy.activeConfigKey` (rides the existing JSONB blob) and the measurement system (`localStorage.canopy:project-measurement-system:<projectId>`, resolution order: explicit user choice → unit detected from the brief → imperial).

\newpage

I have everything I need. Here is the full audit.

Appendix C — Agency Workspace (Site-Level Pages)

Scope: all site-level (non-project-step) agency pages. All routes are wrapped in `ProtectedRoute (/Users/bryanscott/Desktop/Brief to Booth/src/components/layout/ProtectedRoute.tsx)`, which enforces: auth → onboarding gate (must belong to an agency, else `/onboarding/create-agency`) → access gate (disabled/trial-expired → `/access-suspended`). Super admins bypass both gates. Sidebar nav for this whole surface is defined in `/Users/bryanscott/Desktop/Brief to Booth/src/components/layout/AppSidebar.tsx` (`agencyNavItems`).

1. Projects — `/projects`

File: `/Users/bryanscott/Desktop/Brief to Booth/src/pages/Projects.tsx` (853 lines) Components: `/Users/bryanscott/Desktop/Brief to Booth/src/components/projects/ProjectCard.tsx`, `/Users/bryanscott/Desktop/Brief to Booth/src/components/projects/ProjectVisualThumb.tsx`, `/Users/bryanscott/Desktop/Brief to Booth/src/lib/projectDisplay.ts`

Route & access — `/projects`, `ProtectedRoute` (all gates on). Sidebar label “All Projects”. Additional admin affordance: an owner-filter toggle (“My Projects” / “All Users”) appears only when `useIsAdmin()` is true.

Intent — The workspace home: every project the user can see, as a visual grid or a client-grouped table, with pipeline-completion at a glance and a single entry point for creating new projects or suites.

UI anatomy

- **PageHeader**: eyebrow `{agency.name} · {n} active projects` (or “projects · all users” in admin mode), title “Your Projects” / “All Projects”, action = **New Project** dialog.

- Filter bar: admin owner toggle (My Projects / All Users) · search input · client Select (only rendered when `clients.length > 0`, with an X to clear) · grid/table view toggle (right-aligned, persisted to `localStorage["projects:viewMode"]`) · result count.
- Admin notice banner when in “All Users” mode.
- **Grid view:** ProjectCard per top-level project. Card anatomy: 16:9 banner (hero render; hover scrub across cursor X indexes into all current renders, with dot indicators; falls back to client-logo-on-brand-gradient, then to project initial), status pill top-right, 3-dot menu top-left, “Suite · N” badge bottom-left; body = client name (title) / activation name (subtitle) / updated-time + Mine|Shared chip / 6-segment pipeline progress bar with per-step tooltips. Suite parents span full width and can expand to a nested child grid.
- **Table view:** projects grouped by client (SectionLabel + count chip), rows show ProjectVisualThumb (hero → client logo → brand-color gradient with initial → folder icon), activation name, updated-relative time, % complete, Shared marker, status pill, 3-dot menu.
- Empty states: “No results for ...” (when searching) vs “No projects yet” with a Create First Project CTA.
- 3-dot menu items: Open suite overview (suite parents only) · Share... · Archive · Delete (own projects only, behind an AlertDialog).

Inputs

- User: project name, optional brand website URL, “Create Suite instead” toggle, search text, client filter, owner filter, view mode, suite expand/collapse.
- Data: `useProjects(adminMode)` → `projects` table `select * ordered by updated_at desc`; non-admin mode additionally `.eq("user_id", user.id)`. `useClients()` → `clients` filtered by `agency_id`. `useProjectThumbnails(projectIds)` → single bulk query on `project_images` (`is_current = true`), bucketed per project and sorted hero-first (`hero_34` → `hero*` → `rest`), 5-min `staleTime`. `useIsAdmin()` → `user_roles`. `useAgency()` for the header.

Workflows

- **Create project** → `createProject.mutateAsync({name, brand_website_url})` inserts into `projects` (`user_id, name, status: "draft", brand_website_url`) → if “Create Suite”, an immediate second `supabase.from("projects").update({is_suite:true})` → `setActiveStep("upload")` → navigate `/upload?project=<id>(&suite=true` for suites).
- **Open project** → suite parents (has children OR `is_suite`) → `/suite?project=<id>`; otherwise status-routed: `draft/parsing` → `/upload`, `reviewed` → `/review`, `generating` → `/generate`, `completed` → `/export`, default `/upload`.
- **Open suite child** (expanded grid) → `/review?project=<childId>`.
- **Delete** → `deleteProject.mutate(id)` → hard DELETE on `projects`, invalidates `["projects", user.id]`, toast.

- **Share... / Archive** → toast only (“coming soon”). No mutation.

Outputs & side effects — Insert/update/delete on `projects`; `localStorage` write for view mode; navigation into the project pipeline. No edge functions invoked from this page.

Current-state gaps

- **Share and Archive are pure stubs.** Both the card menu (Projects.tsx L793–800) and the table-row menu (L690–713) fire toasts titled “Sharing — coming soon” / “Archive — coming soon”. There is no `archived` column and no per-project sharing model.
 - **Create-dialog copy over-promises.** The helper text says “We’ll scrape colors, logo, fonts, and voice from this URL and pre-load it into brand intelligence for this project” (L414–417), but `createProject` only persists `brand_website_url`. The actual scrape (deep-dive-brand) is only triggered later from `BrandWebsiteCard` on the Upload step.
 - **Grouping computes but discards parentIds.** The `useMemo` at L211–237 builds a `parentIds` set that is never returned or used.
 - Search matches on raw `p.user_id` UUID (L250) — placeholder for “search by user”; no email/display-name join exists.
 - The client filter is hidden entirely when the agency has zero clients, and grouping in table view buckets unlinked projects under a synthetic `__brand__: <name>` key derived from `parsed_brief.brand.name`, or “Unassigned”.
 - `deleteProject` is only offered for `isOwn` projects even in admin mode; admins can open but not delete others’ projects from this page.
 - Non-admin `useProjects` re-applies `.eq("user_id", user.id)` on top of RLS, so agency teammates’ projects are invisible on `/projects` even though the `ProjectCard` has a “Shared” affordance and RLS would permit them.
-

2a. Clients — `/clients`

File: `/Users/bryanscott/Desktop/Brief to Booth/src/pages/Clients.tsx` (290 lines) Hook: `/Users/bryanscott/Desktop/Brief to Booth/src/hooks/useClients.tsx`

Route & access — `/clients`, `ProtectedRoute`. Sidebar “Clients”. Sidebar active-state also matches `/clients/:id`.

Intent — Roster of the brands the agency designs for; each card is the doorway into that client’s brand intelligence, knowledge, and projects.

UI anatomy — `PageHeader` (eyebrow `{agency.name}` · `N clients`, title “Clients”, action = **Add client** dialog). Body = 3-column card grid sorted by name; each `ClientCard` shows logo (or 2-letter initials fallback), name, industry, a `{n} projects` mono pill, and an “Open →” link to `/clients/:id`. Empty state = “No clients yet” with an Add-your-first-client CTA.

Inputs

- User (Add-client dialog): Name (required), Industry, Website, Logo URL (plain text URL — no upload), Notes.
- Data: `useClients()` → `clients` where `agency_id = agency.id`, ordered by name, enabled only when both user and `agency.id` exist.
`useClientProjectCounts(agency.id)` → `projects`
`select("client_id").not("client_id", "is", null)`, tallied client-side.

Workflows — Create client → `useUpsertClient` inserts into `clients` with `user_id = auth user` and `agency_id` stamped from `useAgency()`; invalidates `["clients"]`, toast “Client saved”, closes dialog, resets form.

Outputs & side effects — Insert into `clients`. Navigation to the client dashboard.

The project-count fix (as observed in code) — `useClientProjectCounts` (L35–59) carries an explicit comment: the count query is *deliberately not* filtered by `projects.agency_id`, “because that column is only backfilled when an agency is created and is never stamped on new project inserts, so filtering on it returned 0 for every client.” It now relies on RLS to scope the rows and tallies by `client_id`. This is confirmed by `useProjects.createProject`, which inserts no `agency_id`, and by the `agency_members_can_write_projects` policy in `/Users/bryanscott/Desktop/Brief to Booth/supabase/migrations/20260427120000_agency_access_control.sql` (L520+), which keys off `projects.agency_id`. Same pattern is called out as the precedent used by `AgencyPricing`’s plan-item rollout.

Current-state gaps

- The query is still gated on `enabled: !!agencyId` even though it no longer filters by agency — counts silently render 0 for a user with no agency.
 - Underlying data issue remains: `projects.agency_id` is not stamped on insert, so any RLS/analytics that keys off it is unreliable for new projects.
 - No edit or delete affordance on this page (`useDeleteClient` exists in the hook but is only wired in `ClientsManager`); no logo upload (URL string only); `primary_color` / `secondary_color` are on the `Client` model and consumed by `ProjectCard`/`ProjectVisualThumb` but have **no editor on this page or on the client dashboard** — they are only ever populated by the deep-dive.
-

2b. Client Dashboard — `/clients/:clientId`

File: `/Users/bryanscott/Desktop/Brief to Booth/src/pages/ClientDashboard.tsx` (496 lines) Components:
`/Users/bryanscott/Desktop/Brief to Booth/src/components/admin/BrandIntelligencePanel.tsx` (781 lines),
`/Users/bryanscott/Desktop/Brief to Booth/src/components/knowledge/KnowledgeBasePanel.tsx`

Route & access — `/clients/:clientId`, `ProtectedRoute`. Tab is controlled by `?tab=` (`brand` | `knowledge` | `projects`, else `overview`) so other surfaces can deep-link (the code cites Preflight’s “Edit brand” → `?tab=brand`); tab changes replace the URL.

Intent — The per-client hub: identity, freeform notes, AI-extracted brand intelligence, scoped RAG documents, and the client’s project list.

UI anatomy

- Back link to /clients; ClientHeader (PageHeader with 64px logo/initials leading, eyebrow “Client · {industry}”, title = name, subtitle = clickable website, action = **Edit client**).
- Tabs: **Overview · Brand · Knowledge · Projects**.
- Overview: two stat cards (Projects count; Last activity = updated_at relative + created date) + a full-width auto-saving Notes textarea.
- Brand → BrandIntelligencePanel: (a) “Brand auto-discovery” card with a Website|Brand-book-PDF mode toggle and Run/Re-run deep dive button; (b) “Brand assets” card showing logo + primary/secondary color swatches; (c) intelligence entries grouped by category card (Strategy & Voice, Visual Identity, Brand Rules, Past Learnings, Vendors & Materials, Cost Benchmarks) with Approved/Pending badges, “From past project” badge, hover Approve/Delete buttons, tag chips, and — for the “Brand Colors” entry — a parsed hex ColorSwatchRow.
- Knowledge → KnowledgeBasePanel scoped client / scopeId = clientId.
- Projects → flat list of linked projects, each linking to /upload?project=<id> with a status badge and relative updated time.
- EditClientDialog: Name, Industry, Website, Logo URL.

Inputs

- Data: useClient(clientId) → single clients row.
useClientProjects(clientId) → projects select id,name,project_title,status,updated_at,created_at where client_id = :id.
useBrandIntelligence(clientId) → brand_intelligence ordered by category then created_at.
useBrandGuidelines (via the panel’s upsert).
- User: client field edits; notes; website URL or a PDF file for the deep dive; approve/delete on individual intelligence entries; KB document drops.

Workflows

- **Edit client** → useUpsertClient update on clients (re-stamps agency_id).
- **Notes** → on blur, direct supabase.from("clients").update({notes}), invalidates ["client", id], toast “Notes saved”.
- **Deep dive (URL)** → persists a changed website first, then supabase.functions.invoke("deep-dive-brand", { url, clientName, industry }).
- **Deep dive (PDF)** → three size-tiered payloads to the same function: ≤2 MB inline {fileBase64, fileType:"pdf", ...}; 2–6 MB uploads to storage bucket knowledge-base at clients/{clientId}/brand-pdf/{ts}_{name} then sends {storageBucket, storagePath, ...}; >6 MB rasterizes client-side via renderPdfToImages (maxWidth 1500, quality 0.72, **maxPages 40**) and sends {pageImages: string[]}. Progress is surfaced (“Rendered page i of N…”).

- **Deep-dive persistence** (`persistDeepDiveResult`) → dedupes incoming entries against existing by `category::title: identical content+tags` → skipped; changed content → row UPDATE (preserving approval state); new → `useBatchCreateIntelligence` insert with `source: "ai_extracted"`, `is_approved: false`, `confidence_score: 0.85`. Then upserts `brand_guidelines` (non-fatal, logged on failure), then patches the client row with `logo_url / primary_color / secondary_color` **only when those fields are currently empty**. Toast reports “N added · N updated · N unchanged”.
- **Approve entry** → `brand_intelligence.update({is_approved:true, approved_at})`. **Delete entry** → row delete.

Outputs & side effects — Writes to `clients`, `brand_intelligence`, `brand_guidelines`, storage bucket knowledge-base (large PDFs), plus everything the `KnowledgeBasePanel` does (see §4). Edge function: `deep-dive-brand`.

Current-state gaps

- `SpecMono` is imported but never used (`ClientDashboard.tsx` L37).
 - Overview “Projects” stat and the Projects tab both count *all* projects linked to the client with no status/archived filter.
 - No delete-client action on the dashboard.
 - Brand colors and logo can only be set by the deep dive or the Logo-URL text field; there is no color editor, so `primary_color/secondary_color` (used for card gradients) can never be corrected in the UI.
 - Deep-dive PDF path silently truncates at 40 pages (a toast warns, but content past page 40 is never analysed).
 - Approval state is captured but **not enforced anywhere** — nothing filters `brand_intelligence` by `is_approved` when building generation context.
 - `brand_guidelines` upsert failure is swallowed as non-fatal, so the structured projection can silently drift from `brand_intelligence`.
-

3a. Activation Types — `/agency/activation-types`

File: `/Users/bryanscott/Desktop/Brief to Booth/src/pages/ActivationTypes.tsx` (392 lines) Hook: `/Users/bryanscott/Desktop/Brief to Booth/src/hooks/useActivationTypes.tsx`

Route & access — `/agency/activation-types`, `ProtectedRoute`. Sidebar “Activation Types”.

Intent — The catalogue of formats the agency builds (booths, pop-ups, installations ...), mixing platform built-ins with agency-created custom types, so that the AI knows the grammar of each format.

UI anatomy — `PageHeader` with vocabulary-driven labels (`useVocabulary()` supplies `project_types / project_type / projects per industry`), eyebrow Agency ·

{visible} of {total} types · {industries}, action = **New {project type}**. Optional industry filter chip row (only when the agency spans 2+ industries; “All industries” + one chip per industry). Body = types grouped by category (11 categories: engagement, hospitality, support, outdoor, digital, residential, commercial, civic, film, live, themed), each group headed by a colored **SectionLabel** + count chip; cards show label, mono slug, a star favorite toggle, description (1-line clamp), and meta chips ~{defaultSqft} sqft, {defaultScale}, Built-in. Create dialog: Name, Category select, Description.

Inputs

- Data: `useActivationTypes()` → `activation_types` where `is_builtin = true` OR `user_id = me`, ordered by category then label, then **merged** with this agency’s rows from `activation_type_overrides` (`agency_id = agency.id`). Industry isolation filter: built-ins pass only if their `industries[]` intersects the agency’s industries; untagged built-ins are treated as universal; custom types always pass. `useIndustries()` for the filter chips, `useAgency()` for industries / `primary_industry`.
- User: label, category, description; industry filter; favorite stars.

Workflows

- **Create type** → slug is derived from the label (lowercase → non-alnum to `_` → trim `_`) → `useCreateActivationType` inserts into `activation_types` with `is_builtin: false`, `user_id`, and `industries` auto-resolved from the agency’s industries (fallback `["experiential"]`). Invalidates `["activation-types"]`.
- **Favorite** → local only: a `Set` in state mirrored to `localStorage["canopy.activationTypes.favorites"]`; sorts favorited types to the front of their category group. No DB write, not shared across users or devices.
- **Open type** → `/agency/activation-types/:typeId`.

Outputs & side effects — Insert into `activation_types`; `localStorage` write for favorites.

Current-state gaps

- Favorites are device-local and invisible to teammates (documented as “a local presentation preference (no schema change)”).
 - The create dialog only captures label/category/description — `default_scale`, `default_sqft`, `parent_type_affinity`, `element_emphasis`, and `render_context_override` are all supported by `useCreateActivationType` but have no field in this UI (they are only editable in the *platform* `ActivationTypeManager`).
 - No delete/rename for custom types on this page (`useDeleteActivationType` and `useUpdateActivationType` exist but the list surfaces neither).
 - `agencyIndustries` falls back to `["experiential"]` when the agency’s `industries[]` is empty — a defensive workaround for an unbackfilled column.
 - Duplicate slugs are not checked client-side; a collision surfaces as a raw DB error toast.
-

3b. Activation Type Dashboard — `/agency/activation-types/:typeId`

File: `/Users/bryanscott/Desktop/Brief to Booth/src/pages/ActivationTypeDashboard.tsx` (434 lines)

Route & access — `/agency/activation-types/:typeId`, `ProtectedRoute`. Renders “Activation type not found” when the id isn’t in the (industry-filtered) list.

Intent — Teach the AI one format: what it must include, what it must never include, its size envelope, free-form notes, plus the reference documents that ground generation for it.

UI anatomy — Back link; `PageHeader` with `Built-in` and/or `Customized` status chips and a **Restore defaults** action (only when built-in *and* overridden, behind an `AlertDialog`). Two tabs:

- **Template:** an explainer card for built-ins (different copy depending on whether an override exists) + five cards — Description (textarea) · Must-have elements (input + Add, removable badges) · Must-avoid elements (same, destructive badges) · Typical size range (Min/Max sqft number inputs) · Template notes (textarea, described as “injected into every prompt for this activation type”). Save button, disabled until dirty.
- **Knowledge:** two stacked `KnowledgeBasePanels` — scope `activation_type_agency` (“Your agency’s knowledge”, agency-private) and, in a dashed card badged “System foundation · Managed by platform admins”, scope `activation_type` (“Foundation”, platform-wide baseline).

Inputs — `useActivationTypes()` (the same merged list; the effective template is `elementEmphasis.template`, which is the override’s template when present, else the built-in default). User inputs: description, `must_have[]`, `must_avoid[]`, `sqft_min`, `sqft_max`, notes, document uploads.

Workflows

- **Save (built-in)** → `useUpsertActivationTypeOverride` upserts `activation_type_overrides` on conflict (`agency_id`, `activation_type_id`) with `{agency_id, activation_type_id, description, template, created_by}`. Toast: “Your customizations are now active for your agency.”
- **Save (custom)** → `useUpdateActivationType` updates the `activation_types` row directly (`.eq("user_id", me)`), writing `element_emphasis = {...existing, template}`.
- **Restore defaults** → `useDeleteActivationTypeOverride` deletes the override row for (`agency_id`, `activation_type_id`). Copy explicitly notes KB entries are unaffected.

Outputs & side effects — Writes to `activation_type_overrides` or `activation_types`; document uploads (see §4).

Current-state gaps

- **The `activation_type_agency` scope is a dead end for retrieval.** `rag-retrieve` (`/Users/bryanscott/Desktop/Brief to Booth/supabase/functions/rag-retrieve/index.ts` L55–60, L264–268) and `_shared/rag-helper.ts` both only

build scope calls for `agency | client | activation_type | project`, and `SCOPE_WEIGHTS/DEFAULT_SCOPE_WEIGHTS` have no `activation_type_agency` entry. Documents uploaded into the “Your agency’s knowledge” panel are stored and embedded but never retrieved. Only the DB check constraint (migration `20260430192731...sql L436`) knows the scope exists.

- Built-in *type-level* fields (`default_scale`, `default_sqft`, `render_context_override`, `element_emphasis` emphasis weights, `parent_type_affinity`) cannot be overridden per agency — the override table only carries `description + template`.
 - The `useEffect` sync depends on `(type as any).updatedAt`, a property that doesn’t exist on `ActivationType` (it’s `updated_at` on the row and isn’t mapped) — the dependency is inert.
 - No confirmation of whether template notes actually reach prompts is verifiable from this page; the copy asserts injection but the injection point is in the generation functions, not here.
 - The platform-managed “Foundation” panel is fully interactive (upload/delete) for agency users; access is enforced only by RLS, not by the UI.
-

4. Agency Knowledge — `/agency/knowledge`

Files: `/Users/bryanscott/Desktop/Brief to Booth/src/pages/AgencyKnowledge.tsx` (60 lines),
`/Users/bryanscott/Desktop/Brief to Booth/src/components/knowledge/KnowledgeBasePanel.tsx` (564 lines),
`/Users/bryanscott/Desktop/Brief to Booth/src/hooks/useKnowledgeDocuments.tsx`

Route & access — `/agency/knowledge`, `ProtectedRoute`. Sidebar “Agency Knowledge”. Renders “You aren’t a member of an agency yet” when `agency` is null.

Intent — The agency-wide document layer: playbooks, rate standards, style references — everything that should influence generation across all clients and projects.

UI anatomy — `PageHeader` (eyebrow `{agency.name}` · Shared context) wrapping a single `KnowledgeBasePanel` with `scope="agency"`. Panel anatomy: dropzone (“PDF, DOCX, XLSX, PPTX, TXT, MD, CSV, or images · up to 50 MB”) · a search input that appears once `>3` documents · document rows. Each row: 48px thumbnail (signed-URL image preview for images, else a type-colored icon), title/filename, AI summary (2-line clamp), status badge (Ready / Processing / Pending / Failed with error tooltip), `doc_type` badge, up-to-5 tag chips (`auto_tags + user_tags`) with “+N more”, chunk count + relative age, and action buttons: Preview (dialog with image/PDF/text iframe, Open-in-new-tab, Download), Pin/Unpin, Retry embedding (failed docs only), Delete.

Scopes supported by the panel — `agency · activation_type · activation_type_agency · client · project · industry`. Industry-scoped docs are global (`agency_id IS NULL`) and super-admin-managed.

Inputs — Files (accept list is MIME-mapped, 50 MB hard cap enforced client-side twice: dropzone maxSize and an explicit per-file check with a toast). Data: knowledge_documents filtered by scope, scope_id, and agency_id (or agency_id IS NULL for industry scope), ordered created_at desc.

Workflows — the upload/processing pipeline

1. Upload file to storage bucket knowledge-documents at {agency_id}/{scope}/{scope_id}/{timestamp}_{sanitizedName} (industry scope: industry/{scopeId}/...).
2. Insert a knowledge_documents row with status: "pending", uploaded_by, mime_type, file_size_bytes, user_tags.
3. Invoke edge function **embed-document** {document_id} (non-blocking; failure is logged, not thrown — the row's status reflects it).
4. After a 3 s setTimeout, invoke **auto-tag-document** {document_id} (fire-and-forget, warn on failure).

embed-document (/Users/bryanscott/Desktop/Brief to Booth/supabase/functions/embed-document/index.ts): sets status: "processing" → extracts text (PDF via unpdf, DOCX via mammoth, images become a *synthetic caption* "Image asset: {filename}. Reference: {stem}." — images are never OCR'd or vision-tagged here, TXT/MD/CSV/unknown decoded as UTF-8) → sanitizes for Postgres (NUL bytes, C0 controls, lone surrogates) → chunks at ~1000 chars target / 1500 max / 100 overlap on paragraph boundaries → deletes prior knowledge_chunks for idempotency → embeds each chunk sequentially with Gemini gemini-embedding-001 at 768 dims, taskType: RETRIEVAL_DOCUMENT, small inter-chunk delay → inserts knowledge_chunks → sets status: "embedded", chunk_count. On error: status: "failed" + processing_error.

auto-tag-document: Gemini 2.5 Flash tool-call classify_document returning doc_type (brief | rate_card | research | past_work | brand_guide | spec_sheet | contract | other), 3–8 lowercase tags, confidence, and a <30-word summary; updates the document row in place.

How docs feed generation (rag-retrieve weighting) —

/Users/bryanscott/Desktop/Brief to Booth/supabase/functions/_shared/rag-helper.ts buildRagContext() is what generation actually calls (consumers: parse-brief, generate-element, generate-hero, generate-view, generate-materials, generate-3d-brief, generate-presentation). Mechanics, mirrored in the standalone rag-retrieve function:

1. Embed the query once (RETRIEVAL_QUERY, 768 dims).
2. Fan out one match_knowledge_chunks RPC per active scope (agency always unless disabled; client, activation_type, project when ids are supplied), each with a candidate pool of max(top_k*3, 12) split across scopes, _vector_weight default **0.7** (hybrid pgvector + BM25).
3. Multiply each row's hybrid_score by its **scope weight**: project 1.0 · client 0.85 · activation_type 0.75 · agency 0.6. Per-document priority_weight is already folded into hybrid_score inside SQL.

4. De-dupe by `chunk_id` keeping the highest weighted score.
5. Optional LLM reranker (`rerank: true`, default off) over the top ≤ 20 candidates via Lovable gateway → Gemini 2.5 Flash fallback, scoring 0–1.
6. **Pinned documents are force-injected** — one chunk per pinned doc is pushed to the front before the top-K fill.
7. Enrich with `filename/title/doc_type`, `group by_scope`, and fire-and-forget an analytics row into `rag_query_log` (`query`, `scopes`, `scope_ids`, `top_k`, `result_chunk_ids`, `result_doc_ids`, `reranked`, `pinned_doc_ids`, `duration_ms`).

Default `top_k` is 8 in `rag-retrieve`, 6 in `buildRagContext`.

Outputs & side effects — Storage object write/delete in `knowledge-documents`; rows in `knowledge_documents` and `knowledge_chunks` (cascade-deleted with the doc); `rag_query_log` rows at retrieval time; two edge-function invocations per upload.

Current-state gaps

- **The 3-second `setTimeout` before `auto-tag-document` is a race**, acknowledged in the comment (“Delay briefly so `embed-document` has a chance to extract text first”). Large PDFs will not have finished extraction, so tagging can run against nothing.
 - **`priority_weight` has no UI.** `updateDocument` accepts it and the SQL scores on it, but the panel exposes only Pin/Unpin. Same for `title` and `user_tags` — writable by the hook, no editing control in the row.
 - **`summarize-document` is dead code.** The edge function exists and is registered in `supabase/config.toml`, but nothing in `src/` or in any other function invokes it; summaries in the UI come from `auto-tag-document`.
 - **`extract-pricing` is also uninvoked** from the app (see §5).
 - Images are indexed by a synthetic filename caption only — no vision tagging despite the `image/*` accept list, so visual case studies contribute almost nothing to retrieval.
 - `filterTags` is a supported prop on `KnowledgeBasePanel` but no caller passes it.
 - Deletion of the storage object is best-effort (`await` without error handling) before the row delete — orphaned blobs are possible.
 - Preview signed URLs are minted for 1 hour on every render of an image row (`useSignedUrl` is unconditional for images), so a large list issues one storage call per image.
 - Failed docs offer Retry only; there is no bulk re-embed here (that lives in the KB Health tab).
-

5a. Agency Pricing — `/agency/pricing`

File: `/Users/bryanscott/Desktop/Brief to Booth/src/pages/AgencyPricing.tsx` (259 lines)

Route & access — `/agency/pricing`, `ProtectedRoute`. Sidebar “Pricing”; the sidebar treats `/pricing` as the same nav item.

Intent — Agency-level home base for the pricing engine: explain the model, list every project that already has a bill of materials, and route into the per-project editor.

UI anatomy — PageHeader with a **Beta StatusChip**, eyebrow “Agency · Rate cards & supplier feeds”. “How it works” overview card (notes coverage is strongest for architecture/construction/exhibit builds; “CSI MasterFormat divisions are wired in; Uniformat support is partial”). “Projects with a bill of materials” section — one row per project with an {n} line items pill and “Updated {relative}”, linking to `/pricing?project=<id>`; empty state “No bills of materials yet” → Pick a project. “Coming soon” roadmap grid of four cards: Rate-card import · Estimate with AI · Blueprint → BOM · Live supplier feeds.

Inputs — `useProjects()` (own projects) and `usePlanItemRollups()` → `plan_items select("project_id, updated_at")` ordered by `updated_at desc`, rolled up client-side; **swallows “table does not exist” errors** so the page still renders before the Phase-1A migration is applied. 30 s `staleTime`.

Workflows — Read-only. Navigation to `/pricing?project=<id>` and `/projects`.

Outputs & side effects — None.

Current-state gaps — Everything in the roadmap section is non-functional marketing copy. Rollups are joined against `useProjects()` (own projects only), so a teammate’s BOM disappears from the list even if `plan_items` RLS returns it. No rate-card, supplier-credential, regional-grid, or snapshot UI exists anywhere despite the page’s own comments promising them for “Phase 1B”.

5b. Pricing (BOM editor) — `/pricing?project=:id`

Files: `/Users/bryanscott/Desktop/Brief to Booth/src/pages/Pricing.tsx` (838 lines), `/Users/bryanscott/Desktop/Brief to Booth/src/hooks/usePricing.tsx`, `/Users/bryanscott/Desktop/Brief to Booth/supabase/migrations/20260427210000_pricing_engine_phase1a.sql`

Route & access — `/pricing`, `ProtectedRoute`; requires `?project=` or it renders “No project selected”.

Intent — Per-project bill of materials with live cost roll-up: add line items, pick region + quality tier, and see the best-available unit price per line with its provenance.

UI anatomy — PageHeader (eyebrow “Pricing engine · N line items”) with a quality-tier **Select** (Basic/Standard/Premium/Custom), a free-text Region input (default “US”), and **Add line item**. Four **SummaryTiles**: Grand total (gradient), Line items, Priced, Unpriced (amber when >0). Line-item card grouped by CSI division, each group headed with its subtotal; rows show description + manufacturer/model, category + `mono item_key`, inline-editable quantity, unit price, total (with `×N.NN regional` note when the factor $\neq 1$), a color-coded **source badge**, and hover Edit/Delete. Grand-total footer. Optional “Roll-up” card summarizing by CSI division with unpriced counts. Add/Edit dialog: Description* · Quantity · Unit (12 options) · CSI division

(21 divisions) · Category (13) · Manufacturer · Model number · Quality tier · Override unit price · Item key (auto-slugified from the description when blank).

BOM model — `plan_items` (`project_id`, `agency_id`, `csi_division`, `unifomat_class`, `category`, `item_key`, `description`, `manufacturer`, `model_number`, `quantity`, `unit`, `quality_tier`, `position`, `override_unit_price/currency/reason`, `notes`, `metadata`).

Sources of pricing — the `price_plan(_project_id, _region, _quality_tier)` RPC resolves per item in strict order: **1.** `override_unit_price` on the item (always wins, confidence forced to high) → **2.** agency-owned `pricing_quotes` matching (`item_key`, `quality_tier`, `agency_id`, `region|null|'global'`), most recent `fetch_at` → **3.** global quotes (`agency_id IS NULL`) when no agency quote exists → **4.** unpriced (`source = 'no_quote'`). Non-override prices are multiplied by a `regional_factors` lookup (most-specific of zip/metro/state/country; per-category factor beats the category-null factor; default 1.0). Source badge vocabulary in the UI: override, agency_rate_card, agency_inventory, ai_estimate, commodity_feed, vendor_api, rsmeans, subcontractor_quote, manual, no_quote — with the color grammar green = your own data, violet = AI, amber = external feed, red = unpriced.

Totals & tiers — `total_price = quantity × unit_price × regional_factor`; grand total is summed client-side from the priced rows; `project_pricing_summary` RPC provides the by-division/by-category subtotals and `unpriced_count`. The page-level tier/region selectors are query parameters to both RPCs; each line item also carries its own `quality_tier` that must match a quote for a price to resolve.

Workflows — Create/update/delete `plan_items` via `usePricing` mutations, each invalidating `["plan-items"]`, `["priced-plan"]`, `["pricing-summary"]` for the project. Quantity edits save on blur/Enter. Delete uses a native `confirm()`.

Outputs & side effects — Writes to `plan_items` only. Two RPC reads.

What's real vs roadmap

- **Real:** BOM CRUD, override pricing, CSI grouping, regional factors, quote resolution, division roll-up, unpriced counting.
- **Not real:** there is **no UI anywhere to create pricing_sources or pricing_quotes**, so in practice every line resolves to `override` or `no_quote`. Rate-card CSV import, “Estimate with AI”, snapshot/version history, and PDF export are named as Phase 1B in the file header and do not exist. `extract-pricing` (which parses rate cards out of knowledge documents into `knowledge_documents.metadata.pricing`) exists as an edge function but is never invoked from the app and never writes into `pricing_quotes`.
- Region is a free-text input with no validation against `regional_factors` values; `unifomat_class` exists on the model with no UI (matching the “Unifomat support is partial” note); `position`, `override_reason`, and `override_currency` are likewise unexposed.
- `ItemDialog` uses `useMemo` for a side-effecting form reset (a `useEffect` in disguise), so the edit form can lag when opening a different row.

6. Company Profile — /company

Files: /Users/bryanscott/Desktop/Brief to Booth/src/pages/CompanyProfile.tsx (780 lines), /Users/bryanscott/Desktop/Brief to Booth/src/hooks/useCompanyProfile.tsx

Route & access — /company, ProtectedRoute. Sidebar “Company Profile”. Page title renders as “Company Settings”.

Intent — The agency’s own identity (as opposed to a client’s): branding used on proposals and exports, contact block, and a show/venue cost database.

UI anatomy — PageHeader with a single top-right **Save Changes** button, disabled until profileDirty. Three tabs:

- **Branding** — Company Information card (Company Name, Tagline, Industry, Default Booth Sizes as a comma-separated string) · Logo Assets card (two LogoUploader dropzones, light-background and dark-background variants, showing an “Uploaded” check badge) · Brand Colors card (two ColorPickers = swatch + native color input + hex text field) with a live preview strip (primary button, secondary button, gradient bar).
- **Contact Info** — Primary Contact Name, Email, Phone, Website, Address (textarea), Internal Notes.
- **Show Costs** — “Show & Venue Cost Database” with an **Add Show** button and a table: Show (with a “Preset” badge), City/Venue, Booth \$/sqft, Labor \$/hr, Drayage \$/cwt, Union Y/N, and Edit/Delete actions (Delete hidden for presets). Add/Edit dialog fields: Show Name*, City*, Venue, Industry, Booth \$/sqft, Labor \$/hr, Drayage \$/cwt, Electrical \$/outlet, Internet, Lead Retrieval, a Union Labor Required switch, and Notes.

Fields (model) — company_profiles: company_name, industry, default_booth_sizes (string[]), notes, logo_url, logo_dark_url, brand_color, secondary_color, tagline, contact_name, contact_email, contact_phone, address, website. show_costs: show_name, city, venue, industry, estimated_booth_cost_per_sqft, estimated_drayage_per_cwt, estimated_labor_rate_per_hr, estimated_electrical_per_outlet, estimated_internet_cost, estimated_lead_retrieval_cost, badge_scan_cost, union_labor_required, notes, is_preset.

Inputs — All of the above from the user. Data: useCompanyProfile() → company_profiles .maybeSingle() (no explicit user filter; relies on RLS). useShowCosts() → show_costs ordered by show_name.

Workflows

- **Logo upload** → 2 MB client-side cap → supabase.storage.from("company-assets").upload("{user.id}/logos/{light|dark}_{ts}.{ext}", file, {upsert:true}) → getPublicUrl → sets local state and marks dirty. **The URL is only persisted when the user then presses Save Changes.**

- **Save Changes** → `upsertProfile` updates the existing `company_profiles` row by `id`, or inserts one with `user_id` when none exists.
- **Add/Edit show cost** → numeric strings coerced with `parseFloat` or `null`. Editing a **preset** creates a new custom copy (`addShowCost`) rather than updating it; toast says “Custom copy created from preset”. Delete is a plain row delete behind an `AlertDialog`.

How these are used downstream

- `logo_url / company_name` → `DesignedDeck` (agency logo + resolved agency name on deck slides), `ProposalExport`, `ExportPackage`, `Export` page.
- `company_name`, `industry`, `default_booth_sizes`, `notes` → packaged as `companyProfile` in `ElementDashboard`’s payload to the **generate-element** edge function, which injects it verbatim into the user prompt as a `--- COMPANY PROFILE ---` block.
- `show_costs` → consumed by `Generate.tsx` via `useShowCosts` and passed as `showCosts` into `generate-element`.

Current-state gaps

- **brand_color, secondary_color, tagline, and all contact fields are saved but never read by any generation or export code path** — `generate-element` receives only `name/industry/booth sizes/notes`; the export components read only `logo_url` and `company_name`. The Branding tab’s promise (“Colors used in proposal headers, accents, and styling”) is aspirational.
 - **logo_dark_url is written but never consumed** anywhere in `src/`.
 - **badge_scan_cost is a dead field in the form**: it’s in `costForm` state and in the save payload, but no input renders it, so it can only ever be `null` or a value inherited from an edited row.
 - The profile query has no `.eq("user_id", ...)` and uses `maybeSingle()`; correctness depends entirely on RLS returning exactly one row.
 - This is a **per-user** profile (`company_profiles.user_id`), not per-agency — two members of the same agency maintain separate company profiles and separate `show_costs` lists.
 - Logo storage path is keyed on `user.id` with `upsert: true`, but the filename is timestamped, so old logos are never cleaned up.
 - Dirty state is not tracked for logo removal (there’s no remove/clear affordance at all) and navigating away loses unsaved edits with no guard.
-

7a. Agency Team — `/agency/team`

Files: `/Users/bryanscott/Desktop/Brief to Booth/src/pages/AgencyTeam.tsx` (411 lines), `/Users/bryanscott/Desktop/Brief to Booth/src/hooks/useAgencyTeam.tsx`

Route & access — `/agency/team`, `ProtectedRoute`. Sidebar “Team”. Write actions gated on `canManage = role === "owner" || role === "admin"` (from `useAgency()`).

Intent — The real agency membership surface: who belongs to the agency, at what role, plus outstanding email invitations.

UI anatomy — PageHeader (eyebrow {agency.name} · N members, action **Invite member** for managers). Members card: per-row avatar (Crown for the primary owner, else initial), email, “Joined {relative}” + “· Primary owner”, and either a role **Select** (Owner/Admin/Member/Viewer) or a read-only role badge, plus a remove (trash) button. Pending invites card (managers only): dashed rows with a mail icon, email, “Sent {relative} · expires {relative}”, role badge, and cancel. Role-reference card explaining Owner/Admin/Member/Viewer and noting only the primary owner can transfer ownership. Non-managers with visible members get a “You don’t have permission...” card. Invite dialog: Email + Role select (Admin/Member/Viewer, each with a hint line).

Inputs

- useAgencyTeam(agencyId) → **SECURITY DEFINER RPC**
list_agency_members(_agency_id) returning {id, user_id, email, role, joined_at, is_primary_owner} (the RPC is what exposes emails).
- useAgencyInvites(agencyId) → pending_invites where invite_type = 'agency_member', agency_id = ..., status = 'pending'.

Workflows

- **Invite** → email lowercased/trimmed and validated for @ → insert into pending_invites {email, invite_type: "agency_member", agency_id, role, invited_by}. Toast “Invite sent”. **No email is actually dispatched** — the model is “invitations apply automatically when the invitee signs up”, and acceptance goes through the accept_pending_invite(_invite_id) RPC (surfaced on /invite/:token / AcceptInvite, listed via the my_pending_invites RPC).
- **Change role** → agency_members.update({role}).eq("id", memberId); blocked client-side for the primary owner with a destructive toast.
- **Remove member** → agency_members.delete(); blocked for the primary owner; confirmed with a native confirm().
- **Cancel invite** → pending_invites.delete().

Outputs & side effects — Writes to agency_members and pending_invites. One RPC read. No edge functions.

Current-state gaps

- The invite dialog’s copy (“Invite sent”, “They’ll automatically join {agency} when they sign up with this email”) is accurate only in the sense that a row was created — **nothing sends an email**. The dedicated admin-invite-user edge function exists but is only wired to the *platform* invite flow in useAdminRole.useInviteUser, not here.
- The member role **Select** offers **Owner** while the invite dialog does not, so ownership can be granted by promotion but there is no explicit transfer flow despite the role-reference card promising one.

- Primary-owner protection is client-side only (two `if (member.is_primary_owner)` toasts); enforcement depends on RLS/DB triggers not visible from this page.
 - Expiry is displayed but never enforced in the UI — expired invites still render as “pending” with a negative relative time.
 - Removal uses a browser `confirm()` rather than the `AlertDialog` pattern used elsewhere.
-

7b. Team — /team

Files: `/Users/bryanscott/Desktop/Brief to Booth/src/pages/Team.tsx` (21 lines), `/Users/bryanscott/Desktop/Brief to Booth/src/components/admin/TeamManager.tsx` (294 lines), `/Users/bryanscott/Desktop/Brief to Booth/src/hooks/useTeam.tsx`

Route & access — `/team`, `ProtectedRoute`. **Not in the sidebar** — reachable only by direct URL. The same `TeamManager` component is also mounted as the “Team” tab of the agency-admin `/admin` view and as “Invites & Team” in the platform-admin view.

Intent — A legacy, pre-agency team model: an owner adds named collaborators to *their own* workspace with a different role vocabulary.

UI anatomy — `PageHeader` (“Agency · Members & roles”) + `TeamManager`: header row with **Invite Member**; a card list of owned members (icon by role, display name, Pending/Active badge, invited email, “Added {relative}”, a role `Select`, and a delete button behind an `AlertDialog`); an empty state; and a dashed “You (Owner)” card showing the current user’s email. Invite dialog: Display Name, Email, Role.

Model & differences from `/agency/team` — This page reads/writes the `team_members` table, not `agency_members`. Roles are **admin** / **designer** / **viewer** (vs owner/admin/member/viewer). Membership is keyed on `team_owner_id` (a personal workspace), not `agency_id`. The list is filtered client-side to `m.team_owner_id === user.id && m.user_id !== user.id`.

Workflows

- **Invite** → inserts a `team_members` row with `user_id` set to **the owner’s own id as a placeholder** (“until the invite is accepted”), plus `team_owner_id`, `role`, `display_name`, `invited_email`, `invited_by`. Toast claims “Invitation sent successfully.”
- **Change role** → `team_members.update({role})`. **Remove** → row delete.

Outputs & side effects — Writes to `team_members` only.

Current-state gaps

- **This is a parallel, orphaned team system.** It shares no data with `/agency/team`; a person added here gains nothing in the agency RLS model, and vice versa. Two “Team” surfaces exist with incompatible role vocabularies.

- **The invite never reaches anyone:** no email, no token, no accept flow. `accepted_at` stays null forever, so every member renders as “Pending”. The placeholder `user_id = owner.id` means the row is indistinguishable from a self-membership at the DB level.
 - `useCurrentRole()` in `useTeam.tsx` derives an app-wide role from this table and would only ever return “owner” given that `accepted_at` is never set.
 - `useTeam` also carries a fully-built **project-invite** system (`project_invites`, `useCreateInviteLink` with `upload_only/view_comment/full_edit` scopes and 7-day expiry, `useRevokeInvite`, `useAcceptInvite` via the `accept_project_invite` RPC) that this page does not surface — and which is the natural backing for the Projects page’s stubbed “Share...” menu item.
 - The page is unlinked from navigation, so it is effectively hidden but still live.
-

8. Admin Settings (agency-admin view) — /admin

Files: `/Users/bryanscott/Desktop/Brief to Booth/src/pages/AdminSettings.tsx` (155 lines),
`/Users/bryanscott/Desktop/Brief to Booth/src/components/admin/ProjectTypeManager.tsx` (388 lines),
`/Users/bryanscott/Desktop/Brief to Booth/src/lib/projectTypes.ts` (253 lines)

Route & access — `/admin`, `ProtectedRoute`. The sidebar shows “Admin Settings” when `(isAdmin && !isSuperAdmin) || (isSuperAdmin && previewMode)`. The page itself branches on `isPlatformView = isSuperAdmin && !previewMode`: platform admins get a completely different tab set (All Accounts, Activation Types, Venues & Shows, Image Models, AI Usage, Invites & Team); everyone else gets the agency view. Header changes accordingly (“Platform Admin” + crown vs “Agency Settings” + `Settings2` icon).

Intent (agency view) — One place for agency-level configuration: how the AI should think about each project type, client/brand intelligence, agency knowledge, KB telemetry, and team.

UI anatomy (agency view tabs)

1. **Project Types** → `ProjectTypeManager`
2. **Clients & Brand Intelligence** → `ClientsManager` (926 lines; a master/detail client browser with `AddClientWizard`, `BrandGuidelinesEditor`, `BrandAssetLibrary`, `ClientBrandKnowledgeBase`, manual intelligence-entry CRUD across the six categories, and a `scrape-brand-guidelines` edge-function call)
3. **Agency Knowledge Base** → `AgencyKnowledgeBase`, a thin wrapper over `KnowledgeBasePanel` `scope="agency"` — the *same* surface as `/agency/knowledge`
4. **KB Health** → `KnowledgeHealthDashboard`: totals, per-scope distribution (agency/activation_type/client/project), embedding-status breakdown, recent uploads, failed docs with one-click re-embed, and a `migrate-legacy-kb` edge-function trigger
5. **Team** → `TeamManager` (the legacy `team_members` component from §7b)

ProjectTypeManager anatomy — Two-pane layout. Left: sticky sidebar listing the six registry types with a resolved lucide icon, `config.label ?? type.shortLabel`, a violet **custom** chip when an override row exists, and a “Disabled” hint. Right: editor with an inline-editable label + Active/Disabled `StatusChip` + tagline + mono type id, an enable/disable `Switch`, `Reset` and `Save` buttons that appear only when dirty, a `Description` textarea, a **Render Context** mono code-well (“injected into every AI image generation prompt”), and a **Strategic Elements** list — eight numbered rows per type that expand into `Element Title / Short Description / AI Guidance` (“system prompt for this element’s generation”).

The projectTypes registry (`/Users/bryanscott/Desktop/Brief to Booth/src/lib/projectTypes.ts`) — the single source of truth, six `ProjectTypeIds`: `trade_show_booth`, `live_brand_activation`, `permanent_installation`, `film_premiere`, `game_release_activation`, `architectural_brief`. Each `ProjectTypeDef` carries `label/shortLabel/tagline/description/icon/accentColor`, a **renderContext** string (the physical scene description used to frame renders — e.g. “trade show exhibit booth on a convention hall floor with carpet, neighboring booths, overhead lighting”), **spatialDefaults** (`primaryUnit` `sqft|sqm|linear_ft`, `defaultSize`, `sizeLabel`), **eight elements** (`bigIdea`, `experienceFramework`, `interactiveMechanics`, `digitalStorytelling`, `humanConnection`, `adjacentActivations`, `spatialStrategy`, `budgetLogic` — each with a per-type title, description, icon, color, and `aiGuidance` prompt fragment), and **costCategories** (5 per type with `typicalPercentage` weights). The file header documents the 3-step process for adding a type (union member → registry entry → DB check-constraint migration).

Inputs — `useProjectTypeConfigs()` → `project_type_configs` ordered by `sort_order`. Merged with the registry by `mergeWithConfig()`: config value wins per field, else registry default; `element_overrides` are matched by element key.

Workflows — **Save** → `useUpsertProjectTypeConfig` upserts `project_type_configs` on conflict (`user_id`, `project_type_id`) with `{label, tagline, description, render_context, is_enabled, element_overrides}`. Notably, scalar fields are written as **null when unchanged from the registry default** (so only genuine deltas persist), while `element_overrides` is always written as the full 8-element array. **Reset** is local-only — it re-merges with undefined config and clears dirty; it does **not** delete the config row.

Outputs & side effects — Writes to `project_type_configs`; everything else in the tabs writes through its own component (see §2b, §4).

What render context / strategic elements do downstream — and the gap

- **Registry values are live:** `ALL_PROJECT_TYPES / DEFAULT_PROJECT_TYPE` drive `ProjectTypeSelector`, `GuidedBriefBuilder`, and `BriefUpload`; `projectTypeRules.ts` (which imports `ProjectTypeId`) feeds `promptBuilder.ts` and `briefReadiness.ts`.
- **The agency’s saved overrides are not.** A repo-wide grep shows `useProjectTypeConfigs`, `project_type_configs`, `element_overrides`, and the config-sourced `render_context` are referenced **only inside ProjectTypeManager.tsx itself**. No generation path, prompt builder, or edge

function reads `project_type_configs`. Editing render context or AI guidance here changes nothing about generated output today — it is a write-only configuration surface.

- Separately, `custom_project_types` (`useCustomProjectTypes`, with its own `render_context`) and `activation_types.render_context_override` are *third* and *fourth* parallel render-context stores, none of which reconcile with `project_type_configs`.

Other current-state gaps

- `project_type_configs` is keyed on **`user_id`**, not `agency_id` — “Agency Settings” project-type customization is actually per-user.
 - **Two nav paths to the same knowledge surface:** `/agency/knowledge` and the “Agency Knowledge Base” tab render identical panels with different copy.
 - **The `/admin` “Team” tab uses the legacy `TeamManager`** (`team_members`), while the sidebar “Team” link goes to `/agency/team` (`agency_members`) — an agency admin sees two unrelated team rosters depending on which door they use.
 - `costCategories` and `spatialDefaults` from the registry have no editor in `ProjectTypeManager`, and `cost_category_overrides` exists on the `ProjectTypeConfig` type (`useClients.tsx` L49) with no UI at all.
 - “Reset to defaults” is misleading: it clears the form but leaves the persisted override row intact until the user saves.
 - Disabling a type (`is_enabled: false`) renders a “Disabled” hint here, but the brief-side selectors import `ALL_PROJECT_TYPES` directly from the registry and never consult the configs, so a disabled type still appears to users.
 - Sidebar-level: `platformOwnerNavItems` includes `/platform-invites`, which has **no route** in `App.tsx` — clicking it 404s.
-

Cross-cutting observations

1. **Four parallel “type” systems.** `PROJECT_TYPE_REGISTRY` (code) → `project_type_configs` (per-user overrides, unread) · `custom_project_types` (AI-detected, per-user) · `activation_types` + `activation_type_overrides` (per-agency). Only the last one is fully wired end-to-end into RAG scoping.
2. **Two parallel team systems** (`agency_members` + `pending_invites` vs `team_members`), both surfaced in the product, only one enforced by RLS.
3. **Scope coverage hole in RAG.** `activation_type_agency` and `industry` scopes accept uploads and get embedded but are never retrieved; only `agency`, `client`, `activation_type`, `project` participate in `buildRagContext`.
4. **`projects.agency_id` is never stamped on insert.** It is only backfilled at agency creation, which is the root cause of the client-count bug that was fixed by dropping the agency filter, and which weakens the `agency_members_can_write_projects` RLS policy for all newer projects.
5. **Three “coming soon” surfaces are shipped as UI:** project Share/Archive, the AgencyPricing roadmap grid, and the pricing engine’s quote sources (no way to create a

pricing_source or pricing_quote means everything resolves to override-or-unpriced).

6. **Two registered edge functions are never invoked from the app:** summarize-document and extract-pricing.

\newpage

I have everything needed. Here is the full audit.

Appendix D — Super-Admin / Platform-Owner Surfaces

Functional audit (read-only), repo /Users/bryanscott/Desktop/Brief to Booth

Role model (foundation for everything below)

- Roles live in `public.user_roles` as `admin` (agency admin) and `super_admin` (platform owner). `src/hooks/useAdminRole.tsx` exposes `useIsAdmin()` (either role) and `useIsSuperAdmin()` (`super_admin` only), both cached 5 min.
 - Server-side guard is `public.is_super_admin(_user_id)` (SECURITY DEFINER, `supabase/migrations/20260423155547_*.sql`), used by every platform RPC via `public._require_super_admin()`.
 - There is **no impersonation/act-as mechanism**. “Preview mode” is a client-only boolean (see §7).
-

1. /admin — Platform Admin (super-admin branch of AdminSettings)

`src/pages/AdminSettings.tsx`

Route & access — `/admin`, wrapped in `ProtectedRoute` (auth + onboarding + access gate only). **No role guard on the page itself**. The component branches on `isPlatformView = isSuperAdmin && !previewMode`; any authenticated non-admin who navigates to `/admin` gets the full **Agency Settings** surface (the else-branch), not a redirect.

Intent — Give the platform owner a single console for platform-wide accounts and the shared defaults every agency inherits, explicitly excluding agency-scoped data (the comment at lines 55–59 states agency data is reachable only via Preview Mode).

UI anatomy — `PageHeader` (amber Crown, eyebrow “Platform · Accounts & defaults”) + a 6-tab `Tabs`:

Tab	Component	Function
All Accounts (accounts)	<code>UserAccountsManager</code>	Every user on the platform; role grants; platform invite ledger
Activation Types	<code>ActivationTypeManager</code>	CRUD on activation/project

Tab	Component	Function
Venues & Shows	VenueIntelligenceManager	types (built-ins + custom) Show/venue intelligence records (design tips, traffic, union labor, booth sizes)
Image Models	PlatformImageModelManager	Per-agency image-generation model routing — the only place provider IDs are exposed
AI Usage	AiUsageManager	Fleet cost/usage: date-range chips, 6 fleet stats, agency leaderboard, cross-agency user leaderboard, feature × model breakdown
Invites & Team	TeamManager	Invite/manage team members

The agency branch (same route, non-super-admin or preview mode) shows a different 5-tab set: Project Types, Clients & Brand Intelligence, Agency Knowledge Base, KB Health, Team.

Inputs

- `useIsSuperAdmin()` → `user_roles`.
- `usePlatformOwner()` → `previewMode`.
- **All Accounts:** `useAdminProfiles()` → `RPC get_all_user_profiles()` (returns `user_id`, `email`, `display_name`, `avatar_url`, `is_admin`, `is_super_admin`, `created_at`; the SQL WHERE clause self-gates to admin or super_admin) + a full projects select grouped client-side by `user_id`; `useAiUsageByUser(30d)` for inline spend; `usePlatformInvites()` → `platform_invites` table.
- **Image Models:** `agencies.image_model` column + `src/lib/imageModels.ts` registry.
- **AI Usage:** RPCs from `20260507214400_*.sql (_from/_to, columns calls / total_tokens / cost_usd)`.

Workflows (All Accounts tab)

1. **Search** — client-side filter over email / display name / `user_id` / project titles.
2. **Stat cards** — Total accounts, Total projects, Agency Admins, Platform Owners.
3. **Row click** → `navigate('/account/:userId')` (see §5).
4. **Make/Remove Agency Admin** (hover button, shown only when `currentUserIsSuperAdmin && !isSelf && !profile.is_super_admin`) → `useManageAdminRole()` → `supabase.functions.invoke("admin-manage-role", { target_user_id, action: "grant_admin"|"revoke_admin" })` with bearer token. The edge function (`supabase/functions/admin-manage-role/index.ts`) re-checks caller roles with a service-role client, `upserts/deletes`

user_roles, blocks revoke_super_admin on self, and requires super_admin for the two super-admin actions.

5. **Invite User** dialog → role select (member / admin / super_admin — the last only rendered when isSuperAdmin) → useInviteUser() → edge function admin-invite-user: checks existing auth users, either records an already-accepted platform_invites row + upserts profiles, or calls auth.admin.inviteUserByEmail(email, { data: { invited_role: role }, redirectTo: origin + "/auth" }), inserts a platform_invites row, and pre-creates the profile.
6. **Invitations sub-tab** — lists platform_invites with derived state Accepted / Expired (expires_at < now) / Pending. Read-only: no resend, no revoke.

Outputs & side effects — writes to user_roles, platform_invites, profiles; Supabase Auth invite email; navigation to /account/:userId. Query invalidation on admin-all-profiles, is-admin, is-super-admin, admin-platform-invites.

Current-state gaps

- **Blank first paint for super admins.** useState(isPlatformView ? "accounts" : "project-types") is evaluated on the first render, when useIsSuperAdmin() data is still undefined. The tab state initializes to "project-types", which matches no trigger in the platform TabsList — a super admin landing on /admin sees the platform tab bar with **no tab selected and an empty body** until they click one.
- **No feature-flag tab exists at /admin.** Feature flags are per-agency only, inside /admin/agencies (§2). There is no platform-wide flag surface.
- **admin-invite-user cannot be called by a pure super admin.** It gates on .eq("role", "admin").maybeSingle() — a user holding only super_admin gets 403. Contrast with admin-manage-role, which accepts either role.
- **The super_admin option in the invite dialog does nothing.** admin-invite-user only records role in platform_invites and stuffs invited_role into auth metadata; **no code path reads invited_role** to grant the role. Super-admin granting only actually works via /admin/super-admins (pending_invites + accept_pending_invite) or the grant_super_admin action (which no UI calls).
- **“Venues & Shows” is not platform-scoped.** useVenueIntelligence() filters on user_id — the tab shows only the signed-in super admin’s own venue rows, not a platform corpus.
- **“Invites & Team” is not platform-scoped either.** TeamManager uses useTeamMembers() from src/hooks/useTeam.tsx, which reads the legacy team_members table filtered by team_owner_id.eq.<me> OR user_id.eq.<me> — a personal team list, and it’s the *same* component rendered in the agency branch. Its invite flow also inserts a placeholder user_id = owner_id “until the invite is accepted” (comment at useTeam.tsx:70-73).
- **No revoke/resend on platform_invites**, and no cross-check between platform_invites (legacy) and pending_invites (current) — two parallel invite systems.

2. /admin/agencies — Agency lifecycle console

src/pages/AdminAgencies.tsx (893 lines) · src/hooks/useAccessControl.tsx · supabase/migrations/20260427120000_agency_access_control.sql

Route & access — /admin/agencies. Guarded in-component: `useIsSuperAdmin()` → spinner while loading, `<Navigate to="/projects" replace />` if not super admin. Server side, every RPC calls `_require_super_admin()`.

Intent — The commercial control plane: see every agency’s health at a glance and suspend, disable, trial-window, feature-flag, quota, re-industry, and annotate any of them, with a full audit trail.

UI anatomy

- Header (Crown, “Platform · N agencies”) + search box (name / slug / owner email).
- **5 summary tiles:** Total, Active, Trial, Suspended (counts `suspended + trial_expired`), Disabled.
- **Agency list** — one row per agency: building icon, name, `StatusPill`, slug, owner email, “N members · N projects”, inline 30-day AI spend (hidden when \$0, lg: breakpoint only), “Active ”, gear icon. Row click opens the drawer.
- **Detail drawer** (Sheet, sm:max-w-xl) with 5 tabs: **Status · Industries · Features · Quotas · Log**.

Inputs

- `useAdminAgencies()` → RPC `list_agencies_for_admin()`: id, name, slug, owner_user_id, owner_email (joined from `auth.users`), `access_status`, `effective_status` (via `agency_effective_status()` which flips `trial` → `trial_expired` past `trial_ends_at`), `trial_ends_at`, `suspension_reason`, `suspended_at`, `feature_flags`, `quotas`, `admin_notes`, member/client/project counts, `last_activity_at = GREATEST(agency.updated_at, MAX(projects.updated_at))`. Ordered by last activity desc. staleTime 30 s.
- `useAgencyAccessLog(agencyId, 50)` → RPC `get_agency_access_log()` (readable by super admins **or** the owning agency’s owner).
- `useAiUsageByAgency(rangeFromPreset("30d"))`, indexed by agency id; fails silently.
- Industries tab: `direct agencies.select("primary_industry, industries") + useIndustries()` (industries table).

Workflows — each button → `wrapAction(label, mutation)` → toast on success/failure; all RPCs SECURITY DEFINER, all log to `agency_access_log`.

Action	RPC + payload	DB effect
Suspend (disabled if already suspended/disabled)	<code>suspend_agency(_agency_id, _reason)</code>	<code>access_status='suspended'</code> ,

Action	RPC + payload	DB effect
		suspension_reason, suspended_at=now(), suspended_by=auth.uid() (); log suspended
Disable (behind an AlertDialog confirm)	disable_agency(_agency_id, _reason)	access_status='disabled', same suspension fields; log disabled
Reactivate (disabled when already active)	reactivate_agency(_agency_id)	access_status='active', clears reason/suspended_at/suspended_by; log reactivated
Set trial (<input type="date">)	set_agency_trial(_agency_id, _ends_at) — client sends new Date(yyyy-mm-dd).toISOString(), or null for open-ended	access_status='trial', trial_ends_at, clears suspension fields; log trial_set with metadata
Save feature flags (JSON textarea, parse-validated)	update_agency_feature_flags(_agency_id, _flags)	replaces agencies.feature_flags wholesale; log feature_flags_updated
Quick toggles (Switches for generate, export, rag, image_polish, video; checked = flags[flag] != false, i.e. absent ⇒ on)	same RPC with {...existing, [flag]: v}	immediate write, own toast
Save quotas (JSON textarea; hint {max_seats, max_projects, ai_compute_monthly})	update_agency_quotas(_agency_id, _quotas)	replaces agencies.quotas; log quotas_updated
Save notes (“private to super admins”)	update_agency_admin_notes(_agency_id, _notes)	sets agencies.admin_notes; no log entry written
Save industries (primary radio-grid + multi-select chips; primary auto-included and locked in the chip list; Save disabled unless dirty)	admin_set_agency_industries(_agency_id, _primary_industry, _industries)	admin escape hatch — agencies cannot change their own industries (locked at trigger level per 20260427233000_lock_industry_isolation.sql) ; invalidates activation-types

Status tab also shows three read-only count tiles (Members / Clients / Projects) and the “Suspension reason (visible to the agency)” textarea, which is the *input* consumed by both suspend and disable.

Log tab — append-only feed: humanized action, relative time, `performer_email`, quoted reason. `before_state/after_state` snapshots are stored by the RPCs but **not rendered**.

Outputs & side effects — writes to `agencies` (status/trial/flags/quotas/notes/industries) and `agency_access_log`; invalidates `["admin", "agencies"]`, `["agency"]`, `["admin", "agency-access-log", id]`, `["activation-types"]`. No navigation.

Current-state gaps

- The `useMemo` at line 150 is used as a side-effecting reset (`setReason/setTrialEndsAt/...` inside a memo keyed on `agency?.id`) — a `useEffect` pattern shoehorned into `useMemo`.
 - **Trial date is timezone-lossy**: `new Date("2026-08-12")` parses as UTC midnight, so the effective expiry can land a day early for users west of UTC.
 - **No “clear trial” affordance** — leaving the date blank and clicking “Set trial” sends `null` which the RPC treats as open-ended trial, but the button is also the only way to *enter* trial state; there’s no way to go trial → active except “Reactivate”.
 - **Feature flags are written but never read anywhere in the app**. Grep shows no consumer of `agencies.feature_flags` outside this admin screen — the quick toggles are currently inert with respect to product behavior.
 - **Quotas are likewise never enforced**. No reader of `agencies.quotas` exists in `src/`.
 - `update_agency_admin_notes` doesn’t write an audit entry, unlike every sibling RPC.
 - Industries picker depends on `useIndustries()` which has **no fallback** to `BUILTIN_INDUSTRIES` — if the `industries` table is missing, the tab renders an empty picker and Save is unreachable (whereas `/admin/industries` does fall back).
 - The summary tile labeled “Suspended” silently folds in `trial_expired`, so the numbers don’t reconcile with the status pills in the list.
-

3. `/admin/industries` + `/admin/industries/:slug` + built-in registry

3a. `/admin/industries` — Industry list

`src/pages/AdminIndustries.tsx` · `src/hooks/useAdminIndustries.tsx`

Route & access — `/admin/industries`; in-component `useIsSuperAdmin()` guard → `<Navigate to="/projects" />`.

Intent — Manage the verticals Canopy serves. An industry is the top-level taxonomy that determines an agency’s vocabulary, which activation/project types they see, which brief sections render, and which global KB documents enter RAG.

UI anatomy — Header (Crown, search, “New industry”); optional **SchemaSetupBanner**; 4 summary tiles (Industries, Built-in, Custom, Total project types); “All industries” card listing rows.

Row anatomy — gradient icon (from ICON_MAP:

Sparkles/Building2/TreePine/Film/Sofa/Speaker/Layers), label, “Built-in” star badge, slug, “N <vocabulary.project_types>”, “N agencies (M primary)”, “N KB docs” — the whole block is a <Link> to the detail page — plus a trash icon opening a delete AlertDialog.

Inputs — useAdminIndustries() returns { rows, isSchemaReady } with a three-stage fallback:

1. RPC list_industries_for_admin() (single round-trip incl. project_type_count, agency_count, primary_agency_count, knowledge_doc_count) → isSchemaReady: true.
2. Direct industries table select; missing-table error (regex on “could not find the table / does not exist / undefined_table / relation ... does not exist”) → constants + isSchemaReady: false.
3. Table exists but empty, or exists without the RPC → constants / synthesized rows with **zero counts**, isSchemaReady: true.

Workflows

1. **Auto-seed on first super-admin visit** — a useEffect guarded by ensuredRef, isSuper, !isLoading, isSchemaReady, and “no real rows yet” upserts all BUILTIN_INDUSTRIES into industries (onConflict: “slug”, ignoreDuplicates: true), then invalidates [“admin”, “industries”] and [“industries”]. Failures are console.warn-only.
2. **New industry** dialog — slug (auto-slugified to [a-z0-9_]), label, description, vocabulary JSON (pre-filled with the 4 project keys) → useCreateIndustry() → RPC admin_create_industry(_slug, _label, _description, _icon, _vocabulary, _sort_order=100) → success toast, then **window.location.href = /admin/industries/<slug>** (full page reload, not a router navigate).
3. **Delete** — AlertDialog warns with the live agency count when agency_count > 0, then calls useDeleteIndustry({ slug, force: true }) → RPC admin_delete_industry(_slug, _force).
4. **Schema setup banner** — Copy setup SQL (from src/lib/industriesSetupSql.ts) to clipboard, Show/Hide SQL, deep link to the Supabase SQL editor.

Outputs & side effects — writes to industries (upsert/create/delete); invalidates admin/industries, industries, activation-types.

Current-state gaps

- **The auto-seed is dead in the exact case it exists for.** BUILTIN_PLACEHOLDER_UUIDS (useAdminIndustries.tsx:17-23) has only 5 entries and **omits interior_design**, so that fallback row gets id 00000000-0000-0000-0000-000000000000. The guard industries.some(i => !i.id.startsWith(“00000000-0000-4000-8000”)) therefore evaluates true on

the constants-fallback set and returns early — the upsert never runs when the table is empty.

- The registry has **6** industries but the code comments, the banner copy (“the 5 verticals below”), and the placeholder map all say 5.
- **Delete always passes force: true**, so the `_force` parameter is never exercised as a safety valve; the confirm dialog is the only guard against detaching live agencies. Built-in industries are deletable with no extra friction.
- Create dialog exposes no `icon` or `sort_order` field despite the RPC accepting both (`sort_order` silently defaults to 100).
- Post-create uses `window.location.href`, discarding the SPA state and all react-query cache.

3b. /admin/industries/:slug — Industry dashboard

`src/pages/AdminIndustryDashboard.tsx`

Route & access — `/admin/industries/:slug`; super-admin guard; “Industry not found” card when the slug isn’t in the list.

Intent — Configure everything an industry contributes platform-wide: identity/ordering, which activation types belong to it, its globally-curated knowledge corpus, and its vocabulary swaps.

UI anatomy — Back link → PageHeader (icon, label, “Built-in” chip, description) → 4 stat tiles (`<vocab.project_types>` count, Total agencies, As primary, KB documents) → 4 tabs.

Tab	Contents	Writes
Overview	Label, Sort order (number), Description, Icon (free-text Lucide name; falls back to Layers). Save disabled unless dirty.	<code>admin_update_industry(_slug, _label, _description, _icon, _vocabulary=null, _sort_order)</code>
Project Types	List of activation types tagged to this industry (label, Built-in badge, slug, category, “also in: ...”); X to untag. “Add existing” toggles a searchable picker of all untagged types; clicking one tags it.	<code>admin_set_activation_type_industries(_activation_type_id, _industries)</code> — the client computes the new array (union to add, filter to remove)
Knowledge	<code><KnowledgeBasePanel scope="industry" scopeId={industry.id} /></code> — “Global, super-admin curated knowledge. Every agency working in this industry sees these documents in their RAG retrieval.”	<code>knowledge_documents</code> with <code>scope='industry'</code> , <code>agency_id IS NULL</code> ; storage path <code>industry/<id>/<ts>_<file></code> in the <code>knowledge-documents</code> bucket

Tab	Contents	Writes
Vocabulary	JSON textarea + a reference card of the 9 recognized keys (project_type, project_types, project, projects, deliverable, render, spatial_plan, brief, client) with per-key hints.	admin_update_industry (_slug, _vocabulary)

Inputs — useAdminIndustries() (finds by slug), useActivationTypesByIndustry(slug) → RPC list_activation_types_by_industry, useAllActivationTypes() → direct activation_types select ordered builtin-first/category/label, useKnowledgeDocuments({scope:"industry", scopeId}).

Current-state gaps

- **ICON_MAP here omits Sofa** (present in AdminIndustries.tsx), so Interior Design renders the generic Layers icon on its own detail page but the correct sofa icon in the list.
- **KB uploads can attach to a fake industry id.** When the schema isn't ready, industry.id is a placeholder UUID (or all-zeros for interior_design), and the Knowledge tab still renders and accepts uploads against it.
- Project Types tab can only *cross-tag existing* types; the empty state directs the user to “the agency-side Activation Types page” to create new ones — there's no create affordance here.
- No delete/rename of the industry from its own dashboard (only from the list row).

3c. Built-in industries registry

src/lib/builtinIndustries.ts (+ src/lib/industryFields.ts, tested by builtinIndustries.test.ts)

Six canonical verticals shipped as a TS constant so the platform never depends on migration timing: **experiential** (Experiential & Trade Show, sort 10), **architecture** (20), **landscape** (30), **interior_design** (35), **entertainment** (40), **audio_visual** (50).

Each entry declares four things that shape product behavior:

1. **vocabulary** — the 9-key term swap (e.g. experiential renders “Activation / Activations / Booth render / Floor plan”; audio_visual renders “Install / Installs / Equipment list & layout / Scope of work”). Resolved at runtime by useVocabulary() from the agency's primary_industry, defaulting to experiential.
2. **briefSections** — which BriefSectionIds appear in Brief Review for the vertical (e.g. existing-space only for architecture/interior_design; hero-installation/hanging-elements only for experiential/entertainment; furniture-inventory only for interior_design).

3. **inputMode** — `spatial-canvas | existing-space-photo | hybrid`; drives which Spatial-step UI mounts at project creation.
4. **defaultRenderAngles** — seeds the Prompts step's view list (`hero_34, front, iso, wide_shot, before_after, ...`).

Note `briefSections`, `inputMode`, and `defaultRenderAngles` **exist only in the TS constant** — they are not columns on `industries`, not exposed in the admin dashboard, and not editable for custom industries. A super admin who creates a new industry via the UI gets vocabulary only; the brief schema, input mode, and render angles fall to whatever the code defaults are.

Related public surface — `/industries/:slug` (`src/pages/IndustryDetail.tsx`) is the public marketing deep-dive, reading the live `industries` table with a `BUILTIN_INDUSTRIES` fallback for anonymous visitors. Its `HERO_BY_SLUG` map has **5 images and omits interior_design**, so that vertical's landing page renders without a hero.

4. `/admin/super-admins` — Super Admin roster

`src/pages/SuperAdmins.tsx` · `src/hooks/useSuperAdmins.tsx`

Route & access — `/admin/super-admins`; spinner while `useIsSuperAdmin()` resolves, then `<Navigate to="/projects" replace />` for non-super-admins.

Intent — Grant and revoke platform-level administrator access, which confers visibility into (and RLS bypass over) every agency.

UI anatomy — Header (Crown, “Platform · N accounts”, “Invite super admin” button) → **Active** card (list of current super admins) → **Pending invites** card (dashed-border rows) → an amber warning card: “Super admins can view and impersonate any agency. Grant this role with care.” → invite Dialog (single email field).

Inputs

- `useSuperAdmins()` → RPC `list_super_admins()` (`user_id`, email from profiles, `created_at`; self-gated by `AND public.is_super_admin()` in the SQL, ordered oldest-first).
- `useSuperAdminInvites()` → `pending_invites` where `invite_type='super_admin' AND status='pending'`, newest first.

Workflows

1. **Invite** — `useInviteSuperAdmin(email)` lowercases/trims, validates `includes("@")`, inserts `pending_invites { email, invite_type:'super_admin', invited_by: me, agency_id: null, role: null }`. RLS (`agency_admins_can_invite_members`) permits `invite_type='super_admin'` only for super admins. Toast: “ will become a super admin on sign-up.”
2. **Cancel invite** — hard DELETE from `pending_invites` by id; no confirmation.

3. **Revoke** — `window.confirm` with two variants (self: “Revoke YOUR own super admin access?...”; other: “Revoke super admin access for ?”), then RPC `revoke_super_admin(_target_user_id)` which re-checks `is_super_admin()` and deletes the `user_roles` row.

Safeguards observed

- Server: `list_super_admins` and `revoke_super_admin` both self-gate on `is_super_admin()`; RLS gates `pending_invites` inserts.
- Client: `revoke` button disabled when `admins.length === 1`, with tooltip “Cannot revoke the last super admin”; self-revoke gets a distinct scarier confirm; a “You” badge marks your own row.

Outputs & side effects — writes/deletes `pending_invites`, deletes `user_roles` rows; invalidates `super-admins` / `super-admin-invites`. No navigation, no sign-out of the revoked user.

Current-state gaps

- **The last-super-admin protection is client-only.** `revoke_super_admin()` has no `COUNT(*) > 1` check — a direct RPC call (or two concurrent revokes) can empty the roster and permanently lock the platform out of every super-admin surface. Compare `admin-manage-role`, which at least blocks self-revocation server-side; this path allows it.
 - **The invite copy is aspirational.** “Users become super admins automatically when they sign up with these emails” is not implemented: there is no `apply_pending_invites` trigger in `supabase/migrations/` (it is only *mentioned* in a comment in `20260427150000_mandatory_agency_onboarding.sql:8`). The role is granted only when the invitee reaches `/onboarding/create-agency`, which calls `useMyPendingInvites()/useAcceptInvite() → accept_pending_invite()`. A super-admin invitee who already belongs to an agency never hits that page and never gets the role.
 - **No email is sent.** The row is inserted directly from the browser; nothing notifies the invitee.
 - Pending invites show no `expires_at` (14-day default) and there is no resend; expired invites remain in the “pending” list until the `status` filter excludes them (nothing sets `status='expired'`).
 - No audit trail for grant/revoke of super admin (unlike agency actions, which log to `agency_access_log`).
-

5. `/account/:userId` — User account drill-in

`src/pages/AgencyAccount.tsx`

Route & access — `/account/:userId` under `ProtectedRoute`. **No role guard in the component** — access control is entirely delegated to `get_all_user_profiles()`, whose

WHERE clause returns zero rows for non-admins, causing the page to throw “User not found” and render the “Account not found.” state.

Intent — Give the platform owner a single-user dossier: who they are, what role they hold, which agency they own, their roster, their project activity, and (eventually) their plan.

UI anatomy

1. “Back to Accounts” ghost button → `navigate("/admin")`.
2. **Header** — 14×14 avatar (amber+Crown for super admin, primary for admin, muted otherwise), display name, **role badge** computed as: `is_super_admin` → “Platform Owner” · owns an agency → “Agency Owner” · `is_admin` → “Agency Admin” · else “Member”; email + “Joined”; owned-agency line with slug badge. Right-aligned **“Preview as Agency Admin”** button.
3. **4 StatCards** — Projects (sub: N complete), Team Members (sub: “roster” or “No agency”), Last Active (relative to newest project `updated_at`), Member Since.
4. **Subscription card** — 5 pills (Free / Starter / Professional / Agency / Enterprise) + “Billing integration coming soon”.
5. **Team Roster card** — per member: initials avatar, email (or User ...<last6>), “Account owner for ” sub-line, role badge (owner when `is_primary_owner`, else `member.role`), a hardcoded green “Active”.
6. **Recent Projects card** — first 8 projects: name, status pill (draft/reviewed/generating/complete color map), activation/project type, `updated_at` date.

Inputs — one composite `useQuery(["agency-account", userId])`:

- `RPC get_all_user_profiles()`, then `.find(p => p.user_id === userId)` client-side.
- projects where `user_id = :userId`, ordered `updated_at` desc.
- agencies where `owner_user_id = :userId`, limit 1 — i.e. **only agencies this user owns**.
- If an owned agency exists: `RPC list_agency_members(_agency_id)` → `user_id`, `email`, `role`, `joined_at`, `is_primary_owner`.

Workflows — exactly one mutation-free action: **Preview as Agency Admin** → `setPreviewMode(true)` → `navigate("/projects")` → sonner toast “Preview mode active — browsing as Agency Admin / Your navigation has switched to agency view. Use the banner to exit.”

Outputs & side effects — no DB writes at all. Navigation to `/admin` or `/projects`; mutation of the in-memory `previewMode` flag.

Current-state gaps

- **The subscription tier selector is a decorative stub.** The five pills are `<button>`s with no `onClick`; the “selected” state is hardcoded (`tier.id === "free"`) with the

comment “For now default to ‘free’ — this will be wired to real billing later”. There is no subscription column being read or written.

- **No access-status surface.** Suspension state, trial window, feature flags, and quotas of the user’s agency are not shown here even though the drill-in is the natural place for them — you must cross-reference /admin/agencies.
 - **“Owned agencies” is singular and owner-only.** `limit(1)` on `owner_user_id`; a user who is an *admin* or *member* of an agency (rather than owner) shows “No agency” and an empty roster, and multi-agency owners show only one.
 - **No role controls.** Grant/revoke admin exists only on the list row in `UserAccountsManager`, not on the detail page.
 - “Active” on every roster member is a literal string, not a computed status.
 - The Preview button does **not** bind to this user’s agency (see §7) — despite the toast implying you’re now browsing as them.
 - `is_admin` from the RPC is a global `user_roles` fact, while `agency_members.role` is the per-agency fact; the badge mixes the two vocabularies (“Agency Admin” derived from a platform-level role row).
-

6. Platform invites (/platform-invites)

Route & access — **The route does not exist.** `src/App.tsx` defines no `/platform-invites` path; it falls through to `<Route path="*" element={<NotFound />} />`.

The dead link — `src/components/layout/AppSidebar.tsx:57` lists it in `platformOwnerNavItems`:

```
{ path: "/platform-invites", label: "Invites", icon: Mail },
```

So every super admin sees an **“Invites” item in the platform sidebar that renders the 404 page**. This is the only broken entry in either nav set — the other four (`/admin`, `/admin/agencies`, `/admin/industries`, `/admin/super-admins`) all resolve.

What the surface would have shown — the data layer is fully built and is currently reachable only as a sub-tab inside `/admin` → All Accounts → **Invitations**:

- `usePlatformInvites()` (`useAdminRole.tsx:160`) → `platform_invites` table, ordered `created_at` desc, enabled when `useIsAdmin()`.
- `InvitesTab` in `UserAccountsManager.tsx` renders the list with Accepted / Expired / Pending derivation and a “New Invite” button.

Invite tables & functions — there are two parallel systems

	platform_invites	pending_invites
Created in	20260322000633_*.sql	20260423155547_*.sql
Columns	id, email, role, invited_by, accepted_at, expires_at (default	id, email, invite_type ('agency_member' 'super_admin'),

	platform_invites	pending_invites
	now()+7d), created_at	agency_id, role, invited_by, status ('pending' 'accepted' 'revoked' 'expired'), created_at, accepted_at, expires_at (default now()+14d)
RLS	“Admins can manage platform invites” — FOR ALL to has_role(admin) OR has_role(super_admin) (widened in 20260323193024_*.sql)	Per-operation policies; agency_admins_can_invite_members (INSERT) additionally requires agency_has_access(agency_id) so suspended agencies can’t invite
Written by	edge function admin-invite-user (service role)	client inserts from useSuperAdmins / useAgencyTeam
Consumed by	usePlatformInvites() — display only	RPCs my_pending_invites(), accept_pending_invite(_invite_id) (grants agency_members row or user_roles.super_admin)

Edge functions

- supabase/functions/admin-invite-user/index.ts — auth-header required → auth.getUser() → role check (role='admin' only, see gap in §1) → dedupe against auth.admin.listUsers() → either record already-accepted invite + upsert profile, or auth.admin.inviteUserByEmail(email, { data: { invited_role }, redirectTo: <origin>/auth }) + insert platform_invites + pre-create profile. Returns { message, user_id }.
- supabase/functions/admin-manage-role/index.ts — the four role actions (§1 workflow 4).
- There is **no** dedicated invite-acceptance edge function; acceptance is the accept_pending_invite RPC.

Current-state gaps

- Dead sidebar link → 404 (above).
- Two invite tables with no reconciliation, different expiry defaults (7d vs 14d), different lifecycle vocabularies (accepted_at nullable vs a status enum), and only one of them (pending_invites) actually grants anything.

- Nothing ever transitions a `pending_invites` row to `status='expired'`; expiry is enforced only by the `expires_at > now()` predicate inside `my_pending_invites()/accept_pending_invite()`.
 - `platform_invites` has no revoke/resend UI, and its `accepted_at` is set eagerly to `now()` for pre-existing users (so the ledger reads “Accepted” for people who were never actually invited).
 - `AcceptInvite.tsx` at `/invite/:token` is an unrelated **project-level** invite flow (use `AcceptInvite` from `useTeam.tsx`, `project_invites.token`), branded “BriefEngine” rather than Canopy — not connected to either platform invite table.
-

7. Preview-as-agency mode

`src/contexts/PlatformOwnerContext.tsx` (24 lines) · `AppSidebar.tsx` · `AdminSettings.tsx` · `AgencyAccount.tsx`

Route & access — Not a route. A React context boolean provided app-wide in `App.tsx` (`<PlatformOwnerProvider>` wraps the router).

Intent — Let a platform owner flip their own shell from the platform console into the agency-operator experience, to see what agency users see.

Mechanics — the entire implementation

```
const [previewMode, setPreviewMode] = useState(false);
```

There is no agency id, no target user, no session storage, no server call, no JWT change. Only three files consume it:

1. `AppSidebar.tsx` — `showPlatformNav = isSuperAdmin && !previewMode` chooses `platformOwnerNavItems` vs `agencyNavItems`; swaps the Crown logo for the Canopy mark; drops the “Platform Admin” eyebrow; changes the workspace caption from “Platform” to `agency?.name`; adds an “Admin Settings” item when `isSuperAdmin && previewMode`.
2. `AdminSettings.tsx` — `isPlatformView = isSuperAdmin && !previewMode` selects the 6-tab platform surface vs the 5-tab agency surface.
3. `AgencyAccount.tsx` — the “Preview as Agency Admin” button sets it to `true`.

Entering — two affordances, both `setPreviewMode(true); navigate("/projects")`:

- Sidebar footer button “Preview as Agency Admin” (dashed-border, expanded; eye icon with tooltip, collapsed) — rendered only when `isSuperAdmin && !previewMode`.
- `/account/:userId` header button (adds the 5-second sonner toast).

Exiting — `setPreviewMode(false); navigate("/admin")` from either the amber “Preview Mode” banner’s “Exit” link (expanded sidebar) or the amber eye button with tooltip “Exit Preview Mode” (collapsed sidebar). Both live in `SidebarContent`, so they’re only present on `AppLayout` pages.

What is read-only vs writable in preview — Nothing is read-only. The sidebar banner reads “Read-only agency view”, but no query, mutation, RLS policy, or component consults `previewMode`. A super admin in preview mode retains full `is_super_admin()` RLS bypass on `clients`, `projects`, `knowledge_documents`, `agency_members`, and every other gated table, and every Create/Edit/Delete control in the agency surfaces is live.

Current-state gaps

- **The label is factually wrong:** “Read-only agency view” describes an enforcement that does not exist anywhere in the codebase.
 - **Preview is not scoped to any agency.** Because `useAgency()` resolves the *current user’s own* highest-role `agency_members` row, entering preview from `/account/:userId` shows the super admin’s own agency (or nothing if they have no membership) — not the account they drilled into. The toast “browsing as Agency Admin” and the `/account/:userId` placement both imply a per-account impersonation that isn’t implemented.
 - **State is not persisted.** A refresh, a hard navigation (`window.location.href`, used by the industry create flow), or a new tab silently drops preview mode and snaps the shell back to the platform nav.
 - **Exit is unreachable outside AppLayout.** Any page not wrapped in `AppLayout` (e.g. `/access-suspended`, `/onboarding/*`, `/pricing` sub-flows without the shell) has no sidebar and therefore no exit affordance; the only recovery is navigating back to a shell page.
 - A super admin with **no** agency membership entering preview mode sees an agency nav pointed at empty data, with the workspace caption falling back to “Workspace”.
 - `ProtectedRoute`’s onboarding gate exempts super admins unconditionally (`!isSuperAdmin`), so preview mode never triggers the onboarding redirect — but it also means a super admin can’t preview the onboarding experience.
-

8. Suspension / disable enforcement path

Admin action → **DB state** (from §2): `suspend_agency / disable_agency / set_agency_trial` write `agencies.access_status ∈ {active, trial, suspended, disabled}` plus `trial_ends_at`, `suspension_reason`, `suspended_at`, `suspended_by`, and append to `agency_access_log`.

Server-side enforcement (authoritative) — `public.agency_has_access(_agency_id)` returns true only when `access_status IN ('active', 'trial') AND (trial_ends_at IS NULL OR trial_ends_at > now())`. It is embedded in the `FOR ALL` RLS policies (both `USING` and `WITH CHECK`) on:

- `public.clients` — `agency_members_can_write_clients`
- `public.projects` — `agency_members_can_write_projects`
- `public.knowledge_documents` — `agency_members_can_write_knowledge_documents`

- `public.pending_invites` (INSERT) — so a suspended agency cannot invite new members

Each policy short-circuits on `is_super_admin(auth.uid())` first, so platform owners always bypass.

Client-side derivation — `deriveAccessState()` in `src/hooks/useAgency.tsx` reads `agencies.access_status + trial_ends_at` off the embedded agency row from `agency_members` and computes:

- `effectiveStatus` — flips `trial` → `trial_expired` once `trial_ends_at` ≤ `Date.now()`
- `canWrite` = active, or unexpired trial
- `isLockedOut` = disabled **or** `trial_expired`
- `isRestricted` = anything other than active
- `suspensionReason`, `trialEndsAt`, `trialDaysRemaining` (`Math.ceil` of ms/day)

What the suspended user experiences

Effective status	Route gate (ProtectedRoute)	Banner (SuspensionBanner, top of every AppLayout page)	Writes
active	pass	none	allowed
trial (>14 days left)	pass	none	allowed
trial (≤14 days)	pass	amber “Trial: N days remaining / Convert to a paid plan...” + CheckCircle icon	allowed
suspended	pass — <code>isLockedOut</code> is false	red “Your access is currently suspended.” + the admin-authored <code>suspension_reason</code> + “Details” link → <code>/access-suspended</code>	blocked by RLS (writes fail with a policy error)
trial_expired	redirect → <code>/access-suspended</code>	amber “Your trial has ended.”	blocked
disabled	redirect → <code>/access-suspended</code>	red “Your access has been disabled.”	blocked

ProtectedRoute skips the gate entirely for super admins and for the `/access-suspended` path itself (and `enforceAccessGate={false}` is set on `/access-suspended` and `/onboarding/create-agency` to prevent redirect loops).

/access-suspended landing (`src/pages/AccessSuspended.tsx`) — bounces back to `/projects` via `window.location.href` if access is no longer restricted. Otherwise renders: Canopy logo, status icon (Lock/AlertTriangle/Clock), title (“Account disabled” / “Access suspended” / “Trial ended”), headline, “ can no longer create or modify data in Canopy.”, a **Reason** panel echoing `suspension_reason`, a **Trial ended** panel with the relative expiry, reassurance that data is preserved, a `mailto:hello@exhibit.us.com` contact link, and Home / Sign out buttons.

Current-state gaps

- **suspended is a soft state with a hard failure mode.** The route gate treats only `disabled` and `trial_expired` as lockout, so a suspended user browses the whole app normally and only discovers the restriction when a save silently fails with a Postgres RLS error — no `canWrite` check disables a single button anywhere (`canWrite` is computed and never consumed).
 - **The comment in `disable_agency` overstates behavior:** “agency members are signed out (frontend)”. No sign-out is triggered; the user is merely redirected and can still hold a valid session indefinitely.
 - **RLS coverage is partial.** Only `clients`, `projects`, `knowledge_documents`, and `pending_invites` carry `agency_has_access()`. Other agency-scoped tables (e.g. `activation_type_overrides`, `agencies` itself via `useUpdateAgency`, pricing tables) are not gated, so a suspended agency can still mutate them.
 - **Trial expiry is not proactive.** `access_status` stays literally `'trial'` in the DB forever; expiry is recomputed on every read (`agency_effective_status()` server-side, `deriveAccessState()` client-side). There is no scheduled job, no notification, and no audit-log entry when a trial lapses.
 - **Enforcement uses the *primary* agency only.** `useAgency()` picks the highest-role membership (`owner > admin > member > viewer`); a user who belongs to both a suspended and an active agency is gated by whichever one wins that sort, not by the agency whose data they’re touching.
 - `SuspensionBanner` uses dark-theme token classes (`text-red-200`, `text-amber-100`) inside what the codebase describes as a “Flow C light everywhere” shell — likely low-contrast in practice.
 - `AccessSuspended.tsx` performs its redirect with a raw `window.location.href` inside render rather than a router `<Navigate>`.
-

Cross-cutting observations for the FRD

1. **Two admin concepts share one route.** `/admin` is both Platform Admin and Agency Settings, disambiguated by a client-side boolean. There is no dedicated platform-admin component, no role guard on the route, and the initial tab state race (§1) means the default landing is currently broken for super admins.
2. **The dedicated platform pages (`/admin/agencies`, `/admin/industries*`, `/admin/super-admins`) are consistently guarded** — in-component `useIsSuperAdmin()` + `<Navigate to="/projects">` + server-side

_require_super_admin()). /admin and /account/:userId are the two exceptions, relying entirely on RPC-level filtering.

- 3. **Configuration written by admins is largely not yet consumed.** feature_flags, quotas, and the subscription tier have complete authoring UIs and zero readers. The FRD should treat these as authoring-only today.
- 4. **Three separate invite mechanisms** coexist: platform_invites (edge function, display-only ledger), pending_invites (agency member + super admin, actually grants via accept_pending_invite), and project_invites (token links at /invite/:token). Only the middle one grants roles.
- 5. **Preview mode is a nav skin, not an access mode.** Any FRD requirement around “view as agency” needs to specify agency binding, persistence, and actual read-only enforcement — none of which exist.
- 6. **The interior_design industry is a consistent second-class citizen:** missing from the placeholder-UUID map (which breaks auto-seed), missing from the detail-page icon map, missing from the public hero-image map, and contradicted by “5 industries” copy throughout.

\newpage

I have everything I need. Here is the full report.

Appendix E — Backend, Data Model & AI Generation Pipeline

Supabase project: kjbamfitkaxnfypplaq. Generated types at /Users/bryanscott/Desktop/Brief to Booth/src/integrations/supabase/types.ts (PostgREST 14.1) are treated as the ground truth for the hosted schema; supabase/migrations/*.sql are treated as *intent only* (the repo’s own docs state hand-authored migrations are not applied to the hosted DB).

1. Data model

34 tables, one enum (app_role: admin | member | super_admin), no views, ~68 RPC functions.

Identity / agency / access

Table	Purpose	Key columns	Relationships
agencies	Tenant root. Owns clients, projects, KB, quotas, render-model preference.	owner_user_id, slug, access_status (active/trial/suspended/disabled),	referenced by nearly everything

Table	Purpose	Key columns	Relationships
		trial_ends_at, feature_flags jsonb, quotas jsonb, image_model, industries[], primary_industry , brand_colors, logo_url, suspended_at/_by , suspension_reaso n, admin_notes	
agency_members	Membership + role within an agency.	agency_id, user_id, role, invited_by, joined_at	→ agencies
agency_access_log	Immutable audit of admin actions on an agency (suspend/reactivate/quota/flag changes).	action, before_state/after_state jsonb, performed_by, reason	written only by the _log_agency_access RPC
user_roles	Platform-level roles, separate from agency roles.	user_id, role (app_role enum)	—
profiles	Display name / email / avatar mirror of auth.users.	user_id, email, display_name, avatar_url	—
team_members	Legacy pre-agency team model (owner + invited members).	team_owner_id, user_id, role, invited_email, accepted_at	superseded by agency_members; still read in src/hooks/useTeam.tsx
company_profiles	Per-user agency-branding profile used in exports/decks.	company_name, logo_url, logo_dark_url, brand_color, secondary_color, tagline, default_booth_sizes[], contact fields	—
industries	Admin-managed industry verticals with a vocabulary jsonb.	slug, label, is_builtin, sort_order, vocabulary	mirrored client-side by src/lib/builtinIndustries.ts

Clients & brand

Table	Purpose	Key columns	Relationships
clients	The brand a project is for.	agency_id, name, industry, website, logo_url, primary_color, secondary_color	→ agencies
brand_guidelines	One structured row per client — deterministic (non-RAG) brand facts.	client_id, color_system, typography, logo_rules, tone_of_voice, photography_style, materials_finishes (all jsonb), guidelines_version	FK not declared in types (loose client_id)
brand_intelligence	Per-client knowledge entries — the approval-gated memory core.	client_id, category, title, content, tags[], source (manual/ai_extracted/feedback), confidence_score, is_approved, approved_at, source_project_id	→ clients
venue_intelligence	Show/venue knowledge (design tips, traffic, union labor).	show_name, venue, city, industry, design_tips[], traffic_patterns, audience_notes, logistics_notes, booth_placement_tips, typical_booth_sizes[], union_labor_required, source, source_project_id	→ projects
show_costs	Per-show/city cost baselines.	show_name, city, venue,	—

Table	Purpose	Key columns	Relationships
		estimated_booth_cost_per_sqft, estimated_drayage_per_cwt, estimated_electrical_per_outlet, estimated_labor_rate_per_hr, badge_scan_cost, union_labor_required, is_preset	
brand_assets	Not in types.ts. Read/written in src/hooks/useBrandAssets.tsx via .from("brand_assets" as any). Client brand files in the brand-assets bucket.	—	schema drift

Projects & briefs

Table	Purpose	Key columns	Relationships
projects	The central record: brief text + parsed brief + all 8 generated elements as JSONB columns.	user_id, client_id, parent_id (suite hierarchy), name, project_type, activation_type, status, is_suite, inherits_brief, inherits_brand, scale_classification, footprint_sqft, suite_notes, brief_text, brief_file_url/name, brand_website_url, parsed_brief jsonb, render_prompts jsonb, hero_prompt, hero_style_conf	self-FK parent_id, → clients

Table	Purpose	Key columns	Relationships
		rmcd, element columns: big_idea, experience_frame work, interactive_mech anics, digital_storytel ling, human_connection , adjacent_activat ions, spatial_strategy , budget_logic	
custom_project_t ypes	User/AI-authored project types beyond the builtins.	type_id, label, short_label, render_context, spatial_unit, default_size, is_ai_detected, confirmed_by_use r, source_brief_id	→ projects
project_type_con figs	Per-user enable/label/override of a project type.	project_type_id, is_enabled, label, tagline, render_context, element_override s, cost_category_ov errides, sort_order	no FK
activation_types	Catalog of activation formats (booth, popup, premiere...).	slug, label, category, industries[], is_builtin, default_scale, default_sqft, element_emphasis jsonb, parent_type_affi nity[], render_context_o verride	—
activation_type_	Per-agency template	agency_id,	→ both

Table	Purpose	Key columns	Relationships
overrides	override of an activation type.	activation_type_id, template jsonb, description	

Elements / spatial

There is **no dedicated elements or zones table**. The 8 generated elements are JSONB columns on projects (mapped in `src/hooks/useProjectSync.tsx: ELEMENT_DB_KEYS`). Spatial geometry (zones, features, materials catalog, hanging elements) lives inside `projects.spatial_strategy jsonb`, modeled client-side by `src/lib/geometryModel.ts`.

Renders / images

Table	Purpose	Key columns
project_images	Every saved render. The outcome record.	project_id, user_id, angle_id (version+config-suffixed), angle_name, storage_path, public_url, is_current, prompt_artifacts jsonb
rhino_renders	Uploaded Rhino/3D screenshots + their AI-polished counterpart.	project_id, original_storage_path/_public_url, polished_storage_path/_public_url, polish_status, polish_prompt, polish_feedback, view_name, notes

`project_images.prompt_artifacts` is the union of three payloads (see §3): the client's transparency payload (prompt, negative, geometrySummary, compliance, references[], model, generatedAt), the badge fields (modelUsed, primaryError), the config tags (configKey, configLabel), plus hangingApproved written later by the approval gate.

Pricing / BOM

Table	Purpose	Key columns
plan_items	The project BOM line items.	project_id, agency_id, item_key, description, quantity, unit, quality_tier, category, csi_division, unformat_class, manufacturer, model_number, position

Table	Purpose	Key columns
		jsonb, override_unit_price/_c urrency/_reason, metadata
pricing_sources	Rate-card / vendor feed registry.	agency_id, source_type, vendor_name, region, config jsonb, is_active, last_refreshed_at
pricing_quotes	Individual priced rows resolved from a source.	source_id, agency_id, item_key, unit_price, currency, unit, quality_tier, region, confidence, fetched_at, valid_until, source_url
regional_factors	Cost multipliers per region/category.	region, region_kind, category, factor, effective_at, source

Resolution order is encoded in the price_plan(_project_id, _region, _quality_tier) RPC: override > agency rate card > global feed > unpriced, with regional_factor applied to non-overrides. project_pricing_summary rolls up by csi_division. Called from src/hooks/usePricing.tsx.

Knowledge / RAG

Table	Purpose	Key columns
knowledge_documents	The RAG corpus. One row per uploaded doc.	agency_id, scope (agency/client/activati on_type/project/indust ry), scope_id, filename, storage_bucket (default knowledge-documents), storage_path, mime_type, file_size_bytes, status (pending/processing/embedde d/failed), processing_error, chunk_count, extracted_text (capped 50k), summary, doc_type, auto_tags[], user_tags[], priority_weight , is_pinned , metadata

Table	Purpose	Key columns
knowledge_chunks	Chunked + embedded text.	document_id, chunk_index, content, embedding (pgvector 768), scope, scope_id, agency_id, token_count, metadata
rag_query_log	Retrieval analytics.	agency_id, user_id, source, query, scopes[], scope_ids[], top_k, result_chunk_ids[], result_doc_ids[], reranked, pinned_doc_ids[], duration_ms
knowledge_base_files	Legacy per-project KB files (pre-RAG).	project_id, file_name, storage_path, public_url, extracted_text, folder, layer, scope, doc_type, topics[]
activation_type_kb_files	Legacy per-activation-type KB files.	same shape, activation_type_id
kb_migration_log	Idempotency ledger for the legacy → knowledge_documents backfill.	source_table, source_row_id, document_id, status, error

match_knowledge_chunks(_agency_id, _query_embedding, _query_text, _scopes[], _scope_ids[], _match_count, _vector_weight) is a SECURITY DEFINER plpgsql function that gates on is_agency_member() OR is_super_admin(), then computes $\text{hybrid_score} = (\text{_vector_weight} * \text{cosine_similarity} + (1 - \text{_vector_weight}) * \text{ts_rank_cd}(\text{bm25})) * \text{knowledge_documents.priority_weight}$ and returns priority_weight + is_pinned alongside.

Invites / admin / telemetry

Table	Purpose
pending_invites	Agency-scoped invite (email, invite_type, role, expires_at, accepted_at, status). Accepted via accept_pending_invite, listed by my_pending_invites.
platform_invites	Platform-level invite issued by admin-invite-user.

Table	Purpose
project_invites	Share-link invite to one project: token, scope, label, expires_at, accepted_by. Redeemed by accept_project_invite(_token).
ai_usage_events	AI telemetry: feature, model, provider, user_id, agency_id, project_id, input_tokens/output_tokens/total_tokens, cost_usd, duration_ms, status, error_message, metadata. Service-role insert only (_shared/usage-logger.ts); read via ai_usage_by_agency/_by_feature/_by_user/_fleet_totals RPCs.

Storage buckets (referenced in code)

Bucket	Path convention	Written by
project-images	{projectId}/ {configKey}_{angleId} _{ts}.{ext} (save-render-image); {projectId}/hero_34_{ts}_gen.{ext} and {projectId}/{safeAngle}_{ts}_gen.{ext} (server-side pre-upload in generate-hero/view); {projectId}/existing-space/{ts}.{ext} (BriefExistingSpace)	edge functions + client
knowledge-documents	{agencyId}/{scope}/ {scopeId}/{ts} _{safeName} or industry/{industryId} /{ts}_{safeName}	useKnowledgeDocuments
knowledge-base (legacy)	{userId}/ {projectId}/{ts} _{fileName}	useKnowledgeBase, ActivationTypeKnowledgeBase, ClientBrandKnowledgeBase, AddClientWizard, BrandGuidePrompt, BrandIntelligencePanel (clients/{clientId}/brand-pdf/{ts}_{name})
brand-assets	{userId}/{clientId}/	useBrandAssets

Bucket	Path convention	Written by
rhino-renders	{ts}_{safeName} per-file name from useRhinoRenders	useRhinoRenders
company-assets	{userId}/logos/ {variant}_{ts}.{ext}	src/pages/ CompanyProfile.tsx
briefs	{userId}/ {projectId}/original. {ext}	BriefUpload

Observed data-model gaps

- **projects.agency_id is never stamped on insert.** src/pages/Clients.tsx:36 documents this explicitly (“only backfilled when an agency is created and is never stamped on new project inserts, so filtering on it returned 0 for every client”). It is also absent from types.ts entirely, yet price_plan does SELECT p.agency_id FROM projects p and raises if null — so **price_plan will raise 42704 for any project created after agency onboarding.**
- **projects.prompt_versions does not exist.** src/lib/promptVersions.ts writes to it and silently falls back to localStorage on the missing-column error. Prompt versions are therefore **device-local only.**
- Tables in types.ts with **no client writers:** knowledge_chunks, rag_query_log, ai_usage_events, kb_migration_log, agency_access_log, regional_factors (edge/RPC only, by design); pricing_sources and pricing_quotes have **no writer anywhere in the repo** — the pricing engine can only ever resolve override or no_quote.
- Tables used in code but **absent from types.ts** (cast through as any/as never): brand_assets, beta_waitlist.
- brand_intelligence has **no key/value/relevance_weight/metadata columns**, but summarize-document inserts rows with exactly those columns and brand-compliance-check selects category, key, value. Both are broken against the live schema (the insert is best-effort and only warns).
- activation_type_overrides, project_type_configs, team_members are read/written from the UI but are not wired into any generation path.

2. Edge functions

29 functions. Shared infra lives in supabase/functions/_shared/:

- **ai-gateway.ts** — callGemini, callAnthropic, callOpenAIImage, generateImageWithFallback. Despite the header comment claiming Lovable was replaced, **the Lovable gateway is still the preferred Gemini path:**
_callGeminiInner uses
https://ai.gateway.lovable.dev/v1/chat/completions whenever
LOVABLE_API_KEY is set, and only falls back to
https://generativelanguage.googleapis.com/v1beta/models/{model}:g

enerateContent with GOOGLE_AI_API_KEY. Model-name map:
google/gemini-2.5-flash|flash-lite|pro, google/gemini-3-pro-image-preview → gemini-2.0-flash-exp, google/gemini-3-flash-preview → gemini-2.0-flash.

- Image strategy (generateImageWithFallback): **gpt-image-2 primary** (<https://api.openai.com/v1/images/generations> with no refs, /v1/images/edits multipart with refs, max 4 refs, optional alpha mask, 220 s timeout, one retry on transient errors with 8 s backoff for rate-limit/overloaded and 3 s otherwise) → **Gemini gemini-3-pro-image-preview fallback** (100 s timeout, retries once on gemini-3.1-flash-image-preview when the response has no image). Returns modelUsed + primaryError.
- Lovable image path has its own 402 fallback chain: requested model → google/gemini-3.1-flash-image-preview → google/gemini-2.5-flash-image, plus one retry on “upstream returned empty body”.
- Anthropic: <https://api.anthropic.com/v1/messages>, default claude-sonnet-4-20250514, key resolved across ANTHROPIC_API_KEY → LOVABLE_API_KEY → ANTHROPIC_KEY → CLAUDE_API_KEY with a 401-triggered retry on a *different* candidate key.
- **rag-helper.ts** — buildRagContext, described in §4.
- **usage-context.ts** / **usage-logger.ts** — resolves user_id from the JWT and agency_id from the first agency_members row, then fire-and-forget inserts into ai_usage_events with cost estimated from a hardcoded price table in pricing.ts.
- **streaming-response.ts** — emits a keep-alive space every 20 s to defeat Supabase’s 150 s idle timeout, then flushes the JSON body. (generate-hero and generate-view each inline their own copy; generate-view’s pads to 2 KB and sets X-Accel-Buffering: no.)
- **access-gate.ts** — checkAgencyAccess() returns 403 for suspended/disabled/trial-expired/feature-disabled agencies. **It is imported by zero functions** — dead code.

Function	Purpose	Model	Writes / side effects
parse-brief	Extract a full structured brief from raw text / DOCX (JSZip XML walk) / PDF (Gemini vision). Retries 3×, repairs truncated JSON.	google/ gemini-2.5-flash (text), google/gemini-2.5-pro (PDF vision)	none (returns {data})
synthesize-brief	Turn the guided Q&A wizard answers into the same ParsedBrief shape + narrative briefText.	google/ gemini-2.5-flash, temp 0.7	none
suggest-brief-	One-field	google/	none

Function	Purpose	Model	Writes / side effects
field	autocomplete suggestion in the guided builder.	gemini-2.5-flash-lite, temp 0.8	
generate-element	Generate any of the 8 strategy elements via forced tool call. Per-element temperature (0.4 for budgetLogic/spatialStrategy, 1.2 on regenerate, else 0.9). Post-processes spatialStrategy zone positions (ratio → %, clamp, bounds).	google/gemini-2.5-pro	none (client persists to projects.<element>)
enrich-spatial	Fill structuralForm/featureDescription/intent/materialIds per zone and propose 3–6 BoothFeatures bound to the materials catalog.	google/gemini-2.5-pro	none
generate-hero	The primary render. Uses the client's composedPrompt.renderer verbatim when present; else EDIT MODE (previousImage+feedback) or a legacy edge-side markdown builder.	gpt-image-2 → gemini-3-pro-image-preview, 1536x1024, quality high	Storage project-images/{projectId}/hero_34_{ts}_gen.{ext}; returns imageUrl, modelUsed, promptUsed, primaryError
generate-view	Same for one auxiliary angle. Hero image is the single reference in composer mode; multi-anchor only in the legacy path. Requires a Bearer JWT.	same	Storage project-images/{projectId}/{safeAngle}_{ts}_gen.{ext}

Function	Purpose	Model	Writes / side effects
save-render-image	Persist a render: upload bytes (base64 or fetched URL) then insert the metadata row and flip previous rows for that angle to <code>is_current=false</code> .	none	Storage <code>project-images</code> ; writes <code>project_images</code>
generate-panorama	Ultra-wide interior panorama for the VR/explorer view. No fallback model.	<code>gpt-image-2</code> only, 1536x1024	none
polish-rhino-render	Turn a Rhino/SketchUp screenshot into a photoreal viz (geometry preserved, materials/lighting/people added). No fallback.	<code>gpt-image-2</code> only, 1536x1024	none
generate-video	Image → video. Routes on <code>VIDEO_API_PROVIDER</code> .	Runway <code>gen4_turbo</code> , Kling <code>kling-v2</code> , or <code>google/veo-2</code> via the gateway	none (returns a poll <code>taskId</code>)
generate-materials	Cost-estimated materials list by category. RAG-enabled (<code>topK: 8</code>).	<code>google/gemini-3-flash-preview</code>	none
generate-3d-brief	Meshy.ai prompts + a Rhino/3ds-Max modeling brief (layers, materials, construction notes). RAG-enabled (<code>topK: 6</code>).	<code>google/gemini-3-flash-preview</code>	none
generate-presentation	Multi-mode router: ping (deep key validation with <code>claude-3-5-haiku-20241022</code>), slides (<code>pptxgenjs</code> structures), designed-deck	<code>claude-sonnet-4-20250514</code> (16384 max tokens, temp 0.7)	none

Function	Purpose	Model	Writes / side effects
	(standalone 1920×1080 HTML per slide, incl. per-slide regenerate). RAG-enabled.		
analyze-existing-space	Vision pass over an uploaded room photo → dimensions, features, materials, lighting, summary. Raises 502 on unparseable model JSON.	google/gemini-2.5-pro, temp 0.3	none
deep-dive-brand	Build a brand profile from a URL (Firecrawl scrape + map for about/mission/values, branding signals merged) or a brand-book PDF (inline base64 / Storage download / client-rasterized pageImages[] / pageStoragePaths []). Emits brand_intelligence-ready entries + a brand_guidelines payload + logo/colors.	google/gemini-2.5-pro, temp 0.3	Storage reads only; returns entries for the client to insert
scrape-brand-guidelines	Thin Firecrawl branding+markdown scrape.	none	none
scrape-venue-info	Firecrawl a show/venue page → structured venue intelligence.	google/gemini-2.5-pro, temp 0.3	none (client writes venue_intelligence)
extract-learnings	Project-close distillation into 3–8 categorized intelligence entries; validates against the	google/gemini-2.5-flash, temp 0.4	none (client inserts brand_intelligence)

Function	Purpose	Model	Writes / side effects
embed-document	6-category enum. Extract (unpdf / mammoth / UTF-8 / synthetic image caption), sanitize for PG (NUL, C0, lone surrogates), chunk (~1000 chars, 1500 max, 100 overlap, paragraph-preferred), embed sequentially with 100 ms spacing. Idempotent — deletes prior chunks first.	gemini-embedding-001, 768 dims, RETRIEVAL_DOCUMENT	knowledge_chunks insert; knowledge_documents status/chunk_count/extracted_text/processing_error
rag-retrieve	Standalone retrieval endpoint: embed → per-scope match_knowledge_chunks → scope-weight → dedupe → optional rerank → pin force-include → enrich with doc metadata.	gemini-embedding-001 + google/gemini-2.5-flash (rerank)	rag_query_log insert
auto-tag-document	Classify doc_type + extract 3–8 tags + one-sentence summary. (Comments say Haiku; code uses Gemini.)	google/gemini-2.5-flash, temp 0	knowledge_documents : doc_type, auto_tags, title, summary, metadata
summarize-document	Rich summary + 5–20 structured key facts with confidence.	claude-sonnet-4-20250514, temp 0.2	knowledge_documents : summary, metadata; optional brand_intelligence insert (schema-mismatched — see gaps)
extract-pricing	Pull rate-card line items from a KB doc.	claude-sonnet-4-20250514, temp 0	knowledge_documents.metadata.pricing
brand-compliance-check	Score a proposed design against brand_guidelines	claude-sonnet-4-20250514, temp 0.1	none

Function	Purpose	Model	Writes / side effects
	+ activation-type emphasis → pass/warn/fail + checks + must-have coverage.		
best-practices-suggest	Synthesize best practices + pitfalls from up to 20 past agency projects.	claude-sonnet-4-20250514	none
migrate-legacy-kb	Super-admin-only idempotent backfill of knowledge_base_files + activation_type_kb_files into knowledge_documents, then triggers embedding.	none	knowledge_documents, kb_migration_log
admin-invite-user	Admin-gated auth.admin.inviteUserByEmail.	none	platform_invites , profiles upsert
admin-manage-role	grant/revoke admin / super_admin; blocks self-demotion.	none	user_roles

Notable hardening

- save-render-image **PGRST204 retry**: if the insert is rejected because prompt_artifacts is missing (migration 20260514000000 not applied), it deletes the field and re-inserts. Rationale is in-code: without it the image uploaded to Storage but no row was written, so renders vanished on reload. It also caps client artifacts at 100 KB via sanitizePromptArtifacts.
- generate-hero explicitly **removed** its own project_images.prompt_artifacts update because it raced save-render-image and clobbered the *previous* hero's prompt history.
- Both render functions **deliberately drop the floorplan/isometric PNGs** from the model input — the canvas captures bake zone-name text into pixels which gpt-image-2 reproduced on booth walls. They also strip .svg references (image models 400 on them).
- generate-hero/generate-view upload server-side and return a short URL because multi-MB base64 round-trips were silently failing.
- generate-element wraps budgetLogic's schema with only totalPerShow + allocation required, because 60+ nested required fields caused truncated tool calls read by the UI as "stuck generating".

- `parse-brief` has a bracket-balancing JSON repair for truncated model output.
- `analyze-existing-space` distinguishes upstream model failures (502) from its own (500).

Never invoked from the client: `rag-retrieve`, `best-practices-suggest`, `brand-compliance-check`, `extract-pricing`, `summarize-document`. All five are deployed and JWT-gated (`supabase/config.toml`), but no call site exists in `src/`.

3. Prompt composition pipeline (client side)

`src/lib/geometryModel.ts` is the absolute-unit spatial model that replaced the legacy 0–100 % zone shape. `BoothGeometry = { width, depth, ceilingHeightFt, measurementSystem, zones[], features?[], materialsCatalog?[], hangingElements?[] }`. `AbsoluteZone` carries real-unit `x/y/width/depth, heightFt`, a footprint shape (`rect | L | circle | diamond + shapeParams`), a `structuralForm` (`open | enclosed | canopy | alcove | platform | tower`), and `materialIds[]`. `BoothFeature` adds non-zone sculptural objects with `formType` (`tower/ribbon/archway/canopy/sculpture/screen/totem/platform/bar/kiosk`) and `path/polygon` shapes. `AbsoluteHangingElement` is center-anchored (unlike zones) with `suspensionDropFt`. Two-way converters (`boothGeometryFromLegacy`, `normalizedFromAbsoluteZone`) keep AI-generated percentage zones working.

`src/lib/normalizedBrief.ts` is the single source of truth.
`normalizeBrief({project, parsedBrief, geometry, elements}) → NormalizedBrief`;
`validateBrief() → {failures, gaps}`;
`composePrompt() → the 5 stages {briefJson, geometrySummary, renderer, negative, compliance}`.

Normalizer behavior worth noting: `safeBrief()` fills every required field defensively; colors parse an embedded `"orange (#E67E22)"` hex written by the clarification UI; zone names map to *functional purposes* via `zoneNameToPurpose` (`hero/lounge/meeting/reception/demo/merch/narrative/service/media/supporting`) so poetic zone names never reach the model; `hero.physicalForm` falls back from `elements.interactiveMechanics.data.hero.physicalForm.structure` to `parsedBrief.experience.hero.description` (this dual source is what makes the clarification answer actually close the gap); hanging elements preserve incoming ids or mint UUIDs and use **pure geometric defaults** (`ring`, booth center, `width/3`, 1 ft thick, 3 ft drop) — the parser's `estimatedDimensions/suspensionHint` free text is captured but ignored. Fixed constants: `maxObjectSizePctOfFootprint = 0.30`, `minCirculationWidth = 3 ft / 0.9 m`, `humanScale = 5.58 ft / 1.7 m`, staffing hardcoded to `{count: 4, roles: ["sales","engineer"], attire: "business"},timeOfDay: "controlled"`, venue type always `convention_center / controlled_indoor`, `interactiveTech: []`.

Renderer section order (`rendererPrompt`) — spatial-canvas path:

1. # SCENE — from `PROJECT_TYPE_SCENE[type]`; 16:9, eye-level 1.7 m, front-left 45°, per-type photographic lineage (Snøhetta/Foster, Iwan Baan/Hufton+Crow).

2. # SPACE — floor footprint stated as an explicitly **RECTANGULAR** carpet allocation (fixes the “oval carpet” bug), max height, open sides, human scale, min circulation, then “above the floor, design organically”. **The old # COORDINATE LAYOUT per-zone (x,y,w,d) grid was deliberately removed** — it forced a layout-solver mindset producing rectangular pavilions.
3. # HANGING ELEMENTS — only when elements exist. Teaches that they hang from venue rigging and are **not** load-borne by the booth. Per element: name, Form:, geometry line (w × d in booth units × thickness in ft, shape, positionPhraseFor() natural-language placement, drop below ceiling), materials, surfaces, lighting, printed. creativeDirection is emitted with explicit lock language (“EXACT — follow precisely... NOT inspiration”).
4. # STRUCTURAL APPROACH — only when visualLanguage / hero.physicalForm / embrace is non-empty. Asserts brand language must be *architecture*, not decoration; adds reference themes, authored hero form, embrace list, an explicit anti-default list, and (when hanging exists) the “open air between them” separation note.
5. # ZONE PROGRAM — **deduped functional purposes only**, framed as “what the booth needs to contain — let the design place them organically”. Zone coordinates stay on NormalizedBrief.zones for docs/views but never enter the image prompt.
6. # BRAND — name + descriptor, colors by role with hex, required signage with visibilityRequirement (all_sides when openSides ≥ 3 and zones exist, else front_and_back; tagline → front_only).
7. # BUDGET REALITY — tier + ~\$N/sq ft, then BUDGET_TIER_VOCAB: **standard** = laminate / fabric graphics / vinyl / stock aluminum extrusion / functional lighting; **premium** = custom millwork / backlit SEG fabric / wood veneer / metal trim / integrated AV / designed lighting; **ultra** = sculptural forms / kinetic elements / natural stone / LED-integrated panels / theatrical lighting. Plus the **upsell rule**: one accent moment may hint one tier above, never the whole booth (collapses to “design within the ultra tier” at the top). Tier is inferred by inferBudgetTier: ≥ \$400/sqft → ultra, ≥ \$250 → premium, else standard; **no budget** → **premium**.
8. # CONTEXT — venue, audience, time of day, staffing, interactive tech.
9. # ENVIRONMENT — always emitted, per type: convention hall (venue floor, 10 ft aisles, adjacent booths out of focus), festival/outdoor grounds, or real architectural context. Closes with “structure occupies roughly 60-70 % of the frame width”.
10. # DESIGN INTENT — creative.designPhilosophy, capped 300 chars.
11. # HARD CONSTRAINTS — exact footprint, open sides, signage strings, hero ≤ 30 % of footprint, forbidden items, hanging-aloft rule.
12. # NEGATIVE — appended inline because gpt-image-2 has no negative input: creative.avoid + no overlaid annotations + no zone/room labels on fascia + no dimension/percentage callouts + no flat rectangular fascia/truss + no cartoon/oversaturation/AI artifacts + the **void ban** (“blank background, white void, studio backdrop, isolated product shot, floating booth with no floor”) which pairs with # ENVIRONMENT.

Existing-space path (composeExistingSpacePrompt) — dispatched by BUILTIN_INDUSTRIES[industrySlug].inputMode: existing-space-photo always, hybrid only when normalized.existingSpace is present, otherwise spatial-canvas.

Sections: # SCENE (reused) → # EXISTING SPACE (analysis summary, dimensions, features, materials, natural light) → # REGIONS TO PRESERVE → # REGIONS TO REDESIGN → # REDESIGN INTENT → # HARD CONSTRAINTS (same-room, proportions/openings identical) → # NEGATIVE (same string, no drift). # SPACE and # ZONE PROGRAM are intentionally omitted — they describe a void-to-design frame that is wrong here.

Validation gaps surfaced to the clarification UI: primary-color hex (helpful; fires only on colors[0] so it can actually close), venue name (blocking), audience (helpful), hero physical form (helpful), hanging elements (helpful, with a `_dismissedGaps` sentinel so “No — floor-only booth” doesn’t re-fire forever), and hero-scale-exceeds-30 % (blocking). `applyGapAnswer` writes each answer back to the exact `ParsedBrief` field the normalizer reads.

src/lib/promptVersions.ts — angle-id scheme. Base → hero_34; versioned → hero_34__v__{versionId}; version+config → hero_34__v__{versionId}__cfg__{sanitizedFootprint}. `sanitizeConfigKey` lowercases and dash-collapses (“20x40”). `parseVersionedAngleId` strips `__cfg__` first (outermost), then `__v__`. **Legacy fallback:** ids with no suffix belong to the project’s first config (`configs[0]`, the largest) and are claimed by a version flagged `claimsUnversioned` or with id “legacy” (`LEGACY_VERSION_ID`). `findOrphanedVersions()` scans saved images for version suffixes with no matching metadata — including a synthetic legacy bucket — so lost versions can be one-click recovered. Version metadata (`preset`, `label`, `customEmphasis`, `imageModel`, `notes`) persists to `projects.prompt_versions` with a **localStorage mirror**; since that column doesn’t exist, only the mirror is live.

src/lib/renderPromptArtifacts.ts — `buildRenderPromptArtifacts()` produces the transparency payload: `prompt` (capped at `MAX_PROMPT_CHARS = 20 000`, flagged `promptTruncated`), `negative`, `geometrySummary`, `compliance[]`, `references[]` (deduped by URL, **data: URLs filtered out** so no base64 or rasterized mask ever lands in the row, URLs > 2 000 chars dropped, labels defaulted), `model`, `generatedAt`. Empty optionals are omitted rather than stored blank. Returns null with no prompt text.

src/lib/hangingRefinement.ts — `hangingApprovalKey(configKey, heroImageUrl)` scopes approval to *this hero image within this booth size*, so refining or regenerating the hero automatically resets approval and switching size chips never bleeds it. `formatHangingSpecLines()` renders the canonical spec. `buildHangingEditInstruction()` builds the EDIT-MODE body: “modify ONLY the suspended element(s); keep booth, floor, furnishings, people, environment, lighting and camera IDENTICAL”, followed by the canonical spec block, the user’s refinement request, and the suspension/creative-direction reassertion.

The consistency model: hero-first → hanging approval → fan-out

1. `PromptGenerator` memoizes `normalizeBrief` → `composePrompt` → `validateBrief` (each wrapped in try/catch so a bad brief can’t trip the app error boundary).
2. **Hero.** `handleGenerateHeroImage` captures the geometry PNGs, computes the existing-space mask if applicable, and calls `renderStore.generateHeroImage` with `composedPrompt = { renderer, negative, artifacts:`

composerOutput }. The edge function uses renderer **verbatim**. On return, the store appends a HeroTurn to heroThread (branchable conversation), records heroModelUsed / heroPrimaryError, and calls onSave with buildRenderPromptArtifacts(...) — preferring the edge function’s echoed promptUsed over the local renderer text (critical in EDIT MODE, where the edge wraps the feedback in its own template).

3. **Hanging approval gate.** If normalizedBrief.hanging.elements.length > 0, the check panel shows the spec next to the hero. *Refine* calls generateHeroImage in EDIT MODE — previousImageUrl + feedback and **deliberately no composedPrompt**, because that branch takes priority in the edge function and would regenerate from scratch. Each refine saves a new hero version in the same config stack. *Approve* flips renderStore.hangingApprovals[key] and persists prompt_artifacts.hangingApproved = true on the hero’s project_images row so it survives reload. Generating views while unapproved is allowed but warns once — every view inherits the hero’s hanging element.
 4. **Fan-out.** composedViewPrompts builds a HeroSnapshot { composerOutput, normalizedBrief, imageUrl, generatedAt } once the hero exists, and runs composeViewPrompt(snapshot, angle, {zoneId}) for every non-hero angle. View prompts are deliberately **terse (~80–150 tokens)**: one viewInstruction sentence + a consistency guard (“treat the reference as canonical... only the camera angle changes”) + a restriction line. The in-code rationale: a long structured block made gpt-image-2 treat the request as a *fresh generation* and produce a different booth; short prompt + hero-as-only-reference flips it into edit-mode behavior. generateAllViews runs exteriors first then interiors, at **concurrency 3** (down from ~21 min serial to ~9 min; not higher because gpt-image-2 throttles).
 5. **Reference-image flow.** Hero: existing-space photo **exclusively** when present (plus optional alpha mask, transparent = editable) — otherwise [previousImage (edit source), brandLogo, ...extraReferenceUrls] sliced to 4, SVGs stripped. Views: existing-space photo exclusively; else **hero only** in composer mode; else the legacy multi-anchor list. Geometry PNGs are never forwarded. Per-angle attachments come from useRenderReferences and are cleared on success; project-wide visual references (inspiration + brand assets) persist across every view.
 6. **Save.** makeSaveHandler(configKey, configLabel) is a factory — “Generate all sizes” binds a handler per config so a mid-flight chip switch can’t misfile a render. It composes the final angle id (version + config suffixes, or bare for a legacy version), then save-render-image uploads, flips prior is_current, and inserts the row with merged artifacts.
-

4. Brand intelligence & knowledge / RAG

brand_guidelines / brand_intelligence flow. deep-dive-brand (URL or brand-book PDF) produces entries[] mapped into the 6 categories — strategic_voice (mission, vision, values, personality, tone, sentiment, target audience, messaging pillars, taglines, competitors), visual_identity (colors, typography, logo URL, photography style), process_procedure (dos/don’ts) — plus a structured guidelines payload

(colorSystem/typography/logoRules/photographyStyle). Firecrawl branding signals backfill any color/typeface/logo Gemini left blank. Entries arrive **unapproved**; the client inserts them with `is_approved: false, source: "ai_extracted"` (AddClientWizard, ClientsManager, BrandIntelligencePanel).

`src/lib/brandIntelligenceExtractor.ts` does the same locally from a parsed brief (`confidence_score: null`). `extract-learnings` produces the same shape at project close.

The approval gate is real and enforced client-side: `PromptGenerator.tsx:249` and `ElementDashboard.tsx:274` both filter `e.is_approved` before building the `brandIntelligence` payload. Inside `generate-hero/generate-view` a second filter narrows to `visual_identity | vendor_material | strategic_voice` (views drop `strategic_voice`). **confidence_score is never read anywhere in generation** — it is display-only.

Deterministic (non-RAG) brand injection: `useBrandRAG + src/lib/brandRAGBuilder.ts` assemble a plain-text `brandContext` string from `brand_guidelines + approved brand_intelligence + brand_assets + venue_intelligence + legacy knowledge_base_files` (agency KB via the sentinel project id `00000000-0000-0000-0000-000000000001` + project KB) + suite context, weighted by an `ELEMENT_CATEGORY_PRIORITY` map and truncated to a token budget. This string is passed as `brandContext/suiteContext` to `generate-element`, `generate-hero`, `generate-view` (capped at 600/500 chars in the hero/view prompts).

pgvector RAG. Upload → `knowledge-documents` bucket → `knowledge_documents` row (`status: pending`) → `embed-document` (`chunk + gemini-embedding-001 768-d`) → `auto-tag-document` fires 3 s later. Retrieval (`_shared/rag-helper.ts`, mirrored in `rag-retrieve`): embed the query with `RETRIEVAL_QUERY`, call `match_knowledge_chunks` **once per scope in parallel** with `perScopeCount = max(4, ceil(max(topK*3, 12)/scopes))` and `vectorWeight 0.7`, apply **scope weights** **project 1.0 > client 0.85 > activation_type 0.75 > agency 0.6** to `hybrid_score`, dedupe by chunk id keeping the highest weighted score, optionally LLM-rerank the top 20 with `gemini-2.5-flash` at temp 0 (JSON array of 0–1 scores), **force-include one chunk per pinned document**, fill remaining slots, group by scope, and format as — `RETRIEVED KNOWLEDGE BASE CONTEXT` — with per-scope headers and ★ **PINNED** markers that the prompt declares authoritative on conflict. Every retrieval fire-and-forget logs to `rag_query_log`. Failures always degrade to an empty context — retrieval never blocks generation.

Where retrieved context is injected: `parse-brief` (system prompt), `generate-element` (system prompt), `generate-hero` (last 600 chars of `# ADDITIONAL CONTEXT` — deliberately at the end because `gpt-image-2` weights opening tokens), `generate-view` (last 500 chars), `generate-materials` (topK 8), `generate-3d-brief` (topK 6), `generate-presentation` (both modes; skipped on slide regeneration).

Critical observed gap — the RAG path is effectively dead in the main flows. Every `buildRagContext` call site is guarded by `if (agency_id)`, and **no client call site sends agency_id** to `parse-brief` (`BriefUpload.tsx` posts only `briefText/fileBase64`),

generate-element (ElementDashboard.tsx), generate-hero, or generate-view (renderStore.ts has zero occurrences of agency_id). Only generate-materials, generate-3d-brief (ExportPackage.tsx:137,168) and generate-presentation (useDesignedDeck.tsx:143,202) pass it. parse-brief also never receives brandIntelligence/brandContext, so its cross-reference block is dead too.

Stated intent (docs/intelligence/.md) vs. code*

Capability	Docs claim	Code reality
brand_intelligence + approval gate, 6 categories	✓	LIVE — gate enforced at both generation call sites
brand_guidelines structured facts wired into generation	✓	PARTIAL — injected via brandRAGBuilder's prose block, and read by brand-compliance-check, but that function is never called
deep-dive-brand URL + brand-book extraction	✓	LIVE (four input modes: URL, inline PDF, Storage PDF, client-rasterized pages)
Scope-weighted RAG over documents	✓	PARTIAL — fully implemented and correct, but unreachable from the brief/element/render flows for lack of agency_id; live only for materials / 3D brief / decks
prompt_artifacts outcome records	✓	LIVE , with a PGRST204 fallback that silently degrades to no artifacts if the column is missing
extract-learnings at project close	🟡 manual button	matches — manual only, reads elements, not an event stream
learning_events capture	■ Phase 1	PLANNED — table does not exist; no writer anywhere
Embeddings on intelligence entries	■	PLANNED — no embedding column on brand_intelligence
“first 40 approved rows” retrieval for intelligence	stated as today's behavior	inaccurate — the client fetches <i>all</i> client-scoped entries (useClients.tsx select * ordered by category/created_at, no limit) and filters is_approved in memory. The limit(40)

Capability	Docs claim	Code reality
use_count / last_used_at / supersedes_id reinforcement	 Phase 3	exists only inside brand-compliance-check, which is never invoked. PLANNED
scope: agency on intelligence entries	 Phase 4	PLANNED — brand_intelligence has no scope column; agency- scope knowledge only exists in knowledge_documents PLANNED
Estimate calibration from final budgets	 Phase 4	PLANNED
Hanging approval as a durable signal	 on the image row	LIVE (prompt_artifacts.hang ingApproved)
“Migrations gotcha” warning	documented	corroborated — save- render-image’s PGRST204 retry and promptVersions’ localStorage fallback both exist for exactly this reason

docs/intelligence/data-model.md also claims generate-element has a
buildBrandIntelligenceBlock; the function does receive brandIntelligence in its
body but the named helper exists only in generate-hero/generate-view (and in
generate-hero the extracted helper is now dead — the structured builder inlines the filter).

5. Client state

Zustand — **three stores, none persisted.** No persist middleware, no createJSONStorage, no partialize anywhere.

- **src/store/projectStore.ts** (260 lines) — currentProject (id, name, projectType, clientId, hierarchy, rawBrief, parsedBrief, the 8 elements with status/data, renderPrompts), projects[], suiteContext, isLoading, activeStep(upload | review | generate | spatial | prompts | export). loadFromDb derives activeStep from what’s populated (brief → review, any complete element → generate, renderPrompts → prompts). Also exports ELEMENT_META for UI labels/icons.
- **src/store/renderStore.ts** (1038 lines) — the render orchestrator. Holds heroPrompt, heroImage, heroModelUsed, heroPrimaryError, heroFeedback,

heroIterations[], heroThread[] (branchable turns), generatedPrompts, generatedImages (per-angle {url, status}), progress flags, heroVersion/viewVersions (cache-busting counters), designContext, consistencyTokens, hangingApprovals, hydratedFromDb. Async actions generateHeroImage, generateAllViews (batch 3), regenerateView, cascadeRegenerateViews call the edge functions directly and invoke an injected onSave callback. Every action re-checks `get().projectId !== projectId` before writing, so a project switch mid-flight can't corrupt state. `resetForProject` wipes to `initialState`.

- **src/store/videoStore.ts** (337 lines) — CAMERA_MOTION_PRESETS + per-angle video generation state.

React Query — new `QueryClient()` in `src/App.tsx:56` with **all defaults** (no `staleTime`, no `gcTime`, no `retry` config). ~30 hooks in `src/hooks/`. Query-key conventions: `["project", id]`, `["project-images", projectId]`, `["kb-files", label, projectId]`, `["knowledge-documents", scope, scopeId, agencyId|"global"]`, `["client-project-counts", agencyId]`. Mutations invalidate their own key on success.

The bridge: `useProjectSync` reads `?project=` from the URL, hydrates Zustand from the React Query result exactly once (`hasHydrated` ref), and on project change calls `resetProject()` + `queryClient.removeQueries` for `["project", prev]` and `["kb-files", prev]`. Write-back is one-field-at-a-time via `saveProjectField(projectId, field, value) → projects.update({[field]: value})`. Element generation persists immediately after each element so navigation can't lose progress.

Persistence outside the DB: `localStorage` holds the Supabase auth session and prompt-version metadata (`canopy:prompt-versions:{projectId}`). There's a `useClearCacheOnUserChange` hook to avoid cross-user cache bleed.

6. Integrations & config

AI providers, per task

Task	Model	Route
Brief parse (text), synthesize, learnings	google/gemini-2.5-flash	Lovable gateway if <code>LOVABLE_API_KEY</code> , else Google direct
Field suggestion	google/gemini-2.5-flash-lite	same
Brief parse (PDF vision), elements, spatial enrichment, brand deep-dive, venue scrape, existing-space analysis	google/gemini-2.5-pro	same
Materials list, 3D brief	google/gemini-3-	same

Task	Model	Route
	flash-preview	
Document auto-tagging	google/gemini-2.5-flash	same
Embeddings	gemini-embedding-001 (768-d)	Google direct only (GOOGLE_AI_API_KEY required — no Lovable path)
RAG rerank	google/gemini-2.5-flash @ temp 0	Lovable then Google
Decks, doc summarize, pricing extract, compliance, best practices	claude-sonnet-4-20250514	Anthropic direct
Deck key ping	claude-3-5-haiku-20241022	Anthropic direct
Hero / view renders	gpt-image-2 → gemini-3-pro-image-preview (→ gemini-3.1-flash-image-preview)	OpenAI direct / gateway
Panorama, Rhino polish	gpt-image-2 only, no fallback	OpenAI direct
Video	Runway gen4_turbo / Kling kling-v2 / google/veo-2	direct or gateway

Client-facing model naming (src/lib/imageModels.ts) deliberately hides providers: Signature (gemini-3-pro-image-preview), Studio (gemini-3.1-flash-image-preview), Draft (gemini-2.5-flash-image), Typographic (gpt-image-2). useAgencyImageModel collapses agencies.image_model to the coarse "gemini" | "openai" flag the edge functions expect — but **both render functions void imageModel** and always run the gpt-image-2-first fallback chain, so the agency preference is currently inert. Its default (imageModelToProvider → "openai") disagrees with DEFAULT_IMAGE_MODEL (google/gemini-3-pro-image-preview).

Secrets referenced in edge functions: SUPABASE_URL, SUPABASE_ANON_KEY, SUPABASE_SERVICE_ROLE_KEY, LOVABLE_API_KEY, GOOGLE_AI_API_KEY, OPENAI_API_KEY (+ OPENAI_KEY, GPT_IMAGE_API_KEY), ANTHROPIC_API_KEY (+ ANTHROPIC_KEY, CLAUDE_API_KEY, and LOVABLE_API_KEY as an Anthropic candidate), FIRECRAWL_API_KEY, VIDEO_API_KEY, VIDEO_API_PROVIDER.

Frontend env: VITE_SUPABASE_URL, VITE_SUPABASE_PUBLISHABLE_KEY, VITE_SUPABASE_PROJECT_ID. src/integrations/supabase/client.ts **hardcodes the project URL and anon key as fallbacks** — the app works with no .env at all. .env.example also lists GOOGLE_AI_API_KEY / ANTHROPIC_API_KEY, which are server-side only and are not read by the frontend.

JWT config (supabase/config.toml): `verify_jwt = false` for `parse-brief`, `generate-hero`, `generate-view`, `generate-element`, `save-render-image`, `generate-materials`, `enrich-spatial`, `generate-3d-brief`, `generate-presentation`, `generate-video`, `extract-learnings`, `polish-rhino-render`, `generate-panorama`, `deep-dive-brand`. `verify_jwt = true` for `scrape-brand-guidelines`, `embed-document`, `rag-retrieve`, `auto-tag-document`, `summarize-document`, `extract-pricing`, `brand-compliance-check`, `best-practices-suggest`, `analyze-existing-space`. Not listed (default true): `admin-invite-user`, `admin-manage-role`, `migrate-legacy-kb`, `scrape-venue-info`, `suggest-brief-field`, `synthesize-brief`. Several `verify_jwt = false` functions do their own auth: `generate-view` and `generate-panorama` require a Bearer token and call `auth.getUser()`; `save-render-image` requires an auth header and derives `user_id` from it; `generate-hero` does **not** authenticate at all (it only builds a best-effort usage context).

Webhooks / cron: none. No `pg_cron`, no `pg_net`, no HTTP triggers in migrations, no Vercel cron in `vercel.json` (which only sets SPA rewrites and asset cache headers). 22 DB triggers exist in migrations (`updated_at` stamps, agency onboarding backfill). All async work is fire-and-forget invokes from the client or in-function `setTimeout`.

Cost accounting is a hardcoded per-model table in `_shared/pricing.ts` (dated 2026-05): Gemini Pro \$1.25/\$10 per Mtok, Flash \$0.075/\$0.30, image models priced per-image (`gpt-image-2` \$0.19, `gemini-3-pro-image-preview` \$0.039), Sonnet \$3/\$15. Models absent from the table (e.g. `google/veo-2`, `claude-3-5-haiku-20241022`) `log cost_usd = 0`.